

铁路多目标识别与人工复核软件源代码

软件名称：铁路多目标识别与人工复核软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. backend/app/__init__.py
2. backend/app/api/__init__.py
3. backend/app/api/ai.py
4. backend/app/api/annotations.py
5. backend/app/api/assets.py
6. backend/app/api/auth.py
7. backend/app/api/export.py
8. backend/app/api/image_tasks.py
9. backend/app/api/images.py
10. backend/app/api/labels.py
11. backend/app/api/tasks.py
12. backend/app/api/video_captions.py
13. backend/app/api/vision.py
14. backend/app/core/__init__.py
15. backend/app/core/config.py
16. backend/app/core/dependencies.py
17. backend/app/core/security.py
18. backend/app/main.py
19. backend/app/models/__init__.py
20. backend/app/models/schemas.py
21. backend/app/repositories/__init__.py
22. backend/app/repositories/postgres_store.py
23. backend/app/services/__init__.py
24. backend/app/services/export_service.py
25. backend/app/services/inference_service.py
26. backend/app/services/storage_service.py
27. backend/app/services/task_service.py
28. backend/app/services/task_worker.py
29. backend/app/services/vision_service.py
30. backend/models/deepseek-vl2-tiny/config.json
31. backend/models/deepseek-vl2-tiny/processor_config.json
32. backend/pyproject.toml
33. backend/requirements-gpu.txt
34. backend/scripts/export_tensorrt.py
35. backend/scripts/process_video_captions.py
36. backend/scripts/update_video_caption_titles.py

backend/app/__init__.py

```
"""Railway recognition backend package."""
```

backend/app/api/__init__.py

```
"""HTTP route modules."""
```

backend/app/api/ai.py

```
from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException

from app.api.auth import require_user
from app.core.dependencies import get_store, get_vision_service
from app.models.schemas import AiInferRequest, ApiResponse, AuthUser, ImageTaskResult
from app.repositories.postgres_store import PostgresStore
from app.services.vision_service import VisionService

router = APIRouter(prefix="/api/ai", tags=["image-workspace"])

@router.post("/infer", response_model=ApiResponse)
def infer_image_task(
    request: AiInferRequest,
    store: PostgresStore = Depends(get_store),
    service: VisionService = Depends(get_vision_service),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    if current_user.role == "reviewer":
        raise HTTPException(status_code=403, detail="审核员不能执行模型或修改任务结果")
    image = store.get_accessible_image_asset(request.image_id, current_user.user_id)
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")

    session_id = request.session_id or image.session_id or "default"
    task = store.create_or_get_image_task(request.image_id, request.task_type, session_id, current_user.user_id)
    existing_result = store.get_image_task_result_by_task_id(task.task_id)
    if existing_result is not None and existing_result.review_status == "approved":
        raise HTTPException(status_code=409, detail="审核通过的任务已经锁定，不能重新执行模型")
    store.update_image_task_status(task.task_id, "processing")

    try:
        analysis = service.analyze_path(Path(image.file_path), request.task_type, request.model_name)
        result_json = _result_from_analysis(request.task_type, analysis)
    except Exception as exc:
        store.update_image_task_status(task.task_id, "failed")
        raise HTTPException(status_code=503, detail=f"通用视觉模型执行失败: {exc}") from exc

    description = result_json.get("description", "") if isinstance(result_json, dict) else ""
    model_id = str(analysis.get("model") or request.model_name or "unknown")
    result = ImageTaskResult(
        task_id=task.task_id,
        image_id=image.image_id,
        task_type=request.task_type,
        result_image_url="",
        result_image_path="",
        result_json=result_json,
        annotation_json=_annotation_from_result(request.task_type, result_json),
        description_text=description,
        source="ai",
        model_id=model_id,
    )
    saved, version = store.save_image_task_result(result)
```

```

return ApiResponse(
    data={
        "taskId": task.task_id,
        "imageId": image.image_id,
        "taskType": request.task_type,
        "status": "success",
        "resultId": saved.result_id,
        "versionId": version.version_id,
        "versionNo": version.version_no,
        "resultJson": result_json,
        "annotationJson": saved.annotation_json,
        "descriptionText": saved.description_text,
        "source": saved.source,
        "modelId": saved.model_id,
        "latestVersionId": version.version_id,
        "latestVersionNo": version.version_no,
    }
)

```

```

def _result_from_analysis(task_type: str, analysis: dict) -> dict:
    if task_type == "detection":
        return {"boxes": [_box_payload(item) for item in analysis.get("objects", [])]}
    if task_type == "segmentation":
        return {
            "segments": [
                {
                    "label": item.get("label", "object"),
                    "score": float(item.get("confidence", 0.0)),
                    "polygon": item.get("polygon", []),
                }
                for item in analysis.get("segments", [])
            ]
        }
    if task_type == "pose":
        poses = analysis.get("poses", [])
        return {"keypoints": _pose_keypoints(poses), "skeleton": _pose_skeleton(poses)}
    if task_type == "classification":
        return {
            "labels": [
                {"label": item.get("label", "unknown"), "score": float(item.get("confidence", 0.0))}
                for item in analysis.get("results", [])
            ]
        }
    return {"description": str(analysis.get("caption", ""))}

```

```

def _box_payload(item: dict) -> dict:
    bbox = item.get("bbox", {})
    return {
        "x": float(bbox.get("x", 0.0)),
        "y": float(bbox.get("y", 0.0)),
        "width": float(bbox.get("width", 1.0)),
        "height": float(bbox.get("height", 1.0)),
        "label": str(item.get("label", "object")),
        "score": float(item.get("confidence", 0.0)),
    }

```

```

KEYPOINT_NAMES = [
    "nose",
    "left_eye",
    "right_eye",
    "left_ear",
    "right_ear",
    "left_shoulder",
    "right_shoulder",
    "left_elbow",
    "right_elbow",
    "left_wrist",
    "right_wrist",

```

```

    "left_hip",
    "right_hip",
    "left_knee",
    "right_knee",
    "left_ankle",
    "right_ankle",
]

COCO_SKELETON = [
    ("left_shoulder", "right_shoulder"),
    ("left_shoulder", "left_elbow"),
    ("left_elbow", "left_wrist"),
    ("right_shoulder", "right_elbow"),
    ("right_elbow", "right_wrist"),
    ("left_shoulder", "left_hip"),
    ("right_shoulder", "right_hip"),
    ("left_hip", "right_hip"),
    ("left_hip", "left_knee"),
    ("left_knee", "left_ankle"),
    ("right_hip", "right_knee"),
    ("right_knee", "right_ankle"),
    ("nose", "left_eye"),
    ("nose", "right_eye"),
    ("left_eye", "left_ear"),
    ("right_eye", "right_ear"),
]

def _pose_keypoints(poses: list[dict]) -> list[dict]:
    keypoints: list[dict] = []
    for pose_index, pose in enumerate(poses):
        coordinates = pose.get("keypoints", [])
        confidences = pose.get("confidence", [])
        for point_index, coordinates_xy in enumerate(coordinates):
            if len(coordinates_xy) < 2:
                continue
            base_name = KEYPOINT_NAMES[point_index] if point_index < len(KEYPOINT_NAMES) else
f"point_{point_index}"
            keypoints.append(
                {
                    "name": f"person_{pose_index}_{base_name}",
                    "x": float(coordinates_xy[0]),
                    "y": float(coordinates_xy[1]),
                    "score": float(confidences[point_index]) if point_index < len(confidences) else 0.0,
                }
            )
    return keypoints

def _pose_skeleton(poses: list[dict]) -> list[dict]:
    skeleton: list[dict] = []
    for pose_index, pose in enumerate(poses):
        coordinates = pose.get("keypoints", [])
        confidences = pose.get("confidence", [])
        points: dict[str, dict] = {}
        for point_index, coordinates_xy in enumerate(coordinates):
            if len(coordinates_xy) < 2 or point_index >= len(KEYPOINT_NAMES):
                continue
            score = float(confidences[point_index]) if point_index < len(confidences) else 1.0
            if score <= 0.05:
                continue
            x = float(coordinates_xy[0])
            y = float(coordinates_xy[1])
            if x <= 0 and y <= 0:
                continue
            points[KEYPOINT_NAMES[point_index]] = {"x": x, "y": y, "score": score}

    for start_name, end_name in COCO_SKELETON:
        start = points.get(start_name)
        end = points.get(end_name)
        if not start or not end:

```

```

        continue
    skeleton.append(
        {
            "name": f"person_{pose_index}_{start_name}_to_{end_name}",
            "from": f"person_{pose_index}_{start_name}",
            "to": f"person_{pose_index}_{end_name}",
            "x1": start["x"],
            "y1": start["y"],
            "x2": end["x"],
            "y2": end["y"],
            "score": min(start["score"], end["score"]),
        }
    )
return skeleton

```

```

def _annotation_from_result(task_type: str, result_json: dict) -> dict:
    if task_type == "detection":
        return {"shapes": result_json.get("boxes", [])}
    if task_type == "segmentation":
        return {"shapes": result_json.get("segments", [])}
    if task_type == "pose":
        return {"keypoints": result_json.get("keypoints", []), "skeleton": result_json.get("skeleton", [])}
    if task_type == "classification":
        return {"labels": result_json.get("labels", [])}
    return {"description": result_json.get("description", "")}

```

backend/app/api/annotations.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.core.dependencies import get_store
from app.models.schemas import AnnotationsDocument, ReviewRequest, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/assets", tags=["annotations"])

@router.get("/{asset_id}/annotations", response_model=AnnotationsDocument)
def get_annotations(asset_id: str, store: PostgresStore = Depends(get_store)) -> AnnotationsDocument:
    annotations = store.get_annotations(asset_id)
    if annotations is None:
        raise HTTPException(status_code=404, detail="annotations not found")
    return annotations

@router.put("/{asset_id}/annotations", response_model=AnnotationsDocument)
def save_annotations(
    asset_id: str,
    annotations: AnnotationsDocument,
    store: PostgresStore = Depends(get_store),
) -> AnnotationsDocument:
    if store.get_asset(asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    if annotations.asset_id != asset_id:
        raise HTTPException(status_code=400, detail="asset_id mismatch")
    annotations.updated_at = now_utc()
    return store.save_annotations(annotations)

@router.post("/{asset_id}/annotations/review", response_model=AnnotationsDocument)
def review_annotations(
    asset_id: str,
    review: ReviewRequest,
    store: PostgresStore = Depends(get_store),
) -> AnnotationsDocument:
    annotations = store.get_annotations(asset_id)
    if annotations is None:

```

```

        raise HTTPException(status_code=404, detail="annotations not found")
    annotations.review_status = review.status
    for frame in annotations.frames:
        frame.review_status = review.status
        for obj in frame.objects:
            if review.status == "approved" and obj.status == "auto":
                obj.status = "confirmed"
            elif review.status == "rejected":
                obj.status = "rejected"
    annotations.reviewed_at = now_utc() if review.status in {"approved", "rejected"} else None
    annotations.updated_at = now_utc()
    return store.save_annotations(annotations)

```

backend/app/api/assets.py

```

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile
from fastapi.responses import FileResponse

from app.core.dependencies import get_storage_service, get_store
from app.models.schemas import Asset
from app.repositories.postgres_store import PostgresStore
from app.services.storage_service import StorageService

router = APIRouter(prefix="/api/assets", tags=["assets"])

@router.post("/upload", response_model=Asset)
def upload_asset(
    file: UploadFile = File(...),
    storage: StorageService = Depends(get_storage_service),
    store: PostgresStore = Depends(get_store),
) -> Asset:
    asset = storage.save_upload(file)
    return store.create_asset(asset)

@router.get("", response_model=list[Asset])
def list_assets(store: PostgresStore = Depends(get_store)) -> list[Asset]:
    return store.list_assets()

@router.get("/{asset_id}", response_model=Asset)
def get_asset(asset_id: str, store: PostgresStore = Depends(get_store)) -> Asset:
    asset = store.get_asset(asset_id)
    if asset is None:
        raise HTTPException(status_code=404, detail="asset not found")
    return asset

@router.get("/{asset_id}/frames/{frame_index}/image")
def get_frame_image(
    asset_id: str,
    frame_index: int,
    storage: StorageService = Depends(get_storage_service),
    store: PostgresStore = Depends(get_store),
) -> FileResponse:
    if store.get_asset(asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    frame_path = storage.frame_path(asset_id, frame_index)
    if not frame_path.exists():
        raise HTTPException(status_code=404, detail="frame image not found")
    return FileResponse(frame_path, media_type="image/jpeg")

```

backend/app/api/auth.py

```

from datetime import timedelta

```

```

from fastapi import APIRouter, Depends, HTTPException, Request, Response

from app.core.dependencies import get_store
from app.core.security import hash_password, hash_session_token, new_session_token, verify_password
from app.models.schemas import ApiResponse, AuthLoginRequest, AuthRegisterRequest, AuthUser,
AuthUserRoleUpdateRequest, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/auth", tags=["auth"])
AUTH_COOKIE_NAME = "railway_session"
SESSION_MAX_AGE_SECONDS = 7 * 24 * 60 * 60

def resolve_user(request: Request, store: PostgresStore) -> AuthUser | None:
    token = request.cookies.get(AUTH_COOKIE_NAME, "")
    if not token:
        return None
    return store.get_user_by_auth_session(hash_session_token(token))

def require_user(request: Request, store: PostgresStore = Depends(get_store)) -> AuthUser:
    user = getattr(request.state, "current_user", None) or resolve_user(request, store)
    if user is None:
        raise HTTPException(status_code=401, detail="登录状态已失效, 请重新登录")
    return user

def require_reviewer(current_user: AuthUser = Depends(require_user)) -> AuthUser:
    if current_user.role not in {"reviewer", "admin"}:
        raise HTTPException(status_code=403, detail="该操作需要审核员或管理员权限")
    return current_user

def require_admin(current_user: AuthUser = Depends(require_user)) -> AuthUser:
    if current_user.role != "admin":
        raise HTTPException(status_code=403, detail="该操作需要管理员权限")
    return current_user

@router.post("/register", response_model=ApiResponse)
def register(
    request: AuthRegisterRequest,
    response: Response,
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    user = store.create_user(request.username, request.display_name, hash_password(request.password))
    if user is None:
        raise HTTPException(status_code=409, detail="该账号已存在")
    _start_session(response, user, store)
    return ApiResponse(data=_user_payload(user))

@router.post("/login", response_model=ApiResponse)
def login(
    request: AuthLoginRequest,
    response: Response,
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    credentials = store.get_user_credentials(request.username)
    if credentials is None or not verify_password(request.password, credentials[1]) or not
credentials[0].is_active:
        raise HTTPException(status_code=401, detail="账号或密码错误")
    user = credentials[0]
    _start_session(response, user, store)
    return ApiResponse(data=_user_payload(user))

@router.get("/me", response_model=ApiResponse)
def current_user(request: Request, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    user = resolve_user(request, store)

```

```

    return ApiResponse(data=_user_payload(user) if user else None)

@router.post("/logout", response_model=ApiResponse)
def logout(request: Request, response: Response, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    token = request.cookies.get(AUTH_COOKIE_NAME, "")
    if token:
        store.delete_auth_session(hash_session_token(token))
        response.delete_cookie(AUTH_COOKIE_NAME, path="/", httponly=True, samesite="lax")
    return ApiResponse(data={"loggedOut": True})

@router.get("/users", response_model=ApiResponse)
def list_users(
    store: PostgresStore = Depends(get_store),
    _: AuthUser = Depends(require_admin),
) -> ApiResponse:
    return ApiResponse(data=[_user_payload(user) for user in store.list_users()])

@router.put("/users/{user_id}/role", response_model=ApiResponse)
def update_user_role(
    user_id: str,
    request: AuthUserRoleUpdateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_admin),
) -> ApiResponse:
    if user_id == current_user.user_id and request.role != "admin":
        raise HTTPException(status_code=400, detail="不能移除自己的管理员权限")
    user = store.update_user_role(user_id, request.role)
    if user is None:
        raise HTTPException(status_code=404, detail="用户不存在")
    return ApiResponse(data=_user_payload(user))

def _start_session(response: Response, user: AuthUser, store: PostgresStore) -> None:
    token = new_session_token()
    store.create_auth_session(
        hash_session_token(token),
        user.user_id,
        now_utc() + timedelta(seconds=SESSION_MAX_AGE_SECONDS),
    )
    response.set_cookie(
        AUTH_COOKIE_NAME,
        token,
        max_age=SESSION_MAX_AGE_SECONDS,
        httponly=True,
        samesite="lax",
        secure=False,
        path="/",
    )

def _user_payload(user: AuthUser) -> dict:
    return {
        "userId": user.user_id,
        "username": user.username,
        "displayName": user.display_name,
        "role": user.role,
    }

```

backend/app/api/export.py

```

from typing import Literal

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import JSONResponse, PlainTextResponse

from app.core.dependencies import get_export_service

```

```

from app.services.export_service import ExportService

router = APIRouter(prefix="/api/assets", tags=["export"])

@router.get("/{asset_id}/export")
def export_annotations(
    asset_id: str,
    format: Literal["json", "coco", "yolo"] = "json",
    export_service: ExportService = Depends(get_export_service),
):
    annotations = export_service.get_annotations(asset_id)
    if annotations is None:
        raise HTTPException(status_code=404, detail="annotations not found")
    if format == "json":
        return JSONResponse(annotations.model_dump(mode="json"))
    if format == "coco":
        return JSONResponse(export_service.to_coco(annotations))
    return PlainTextResponse(export_service.to_yolo_text(annotations), media_type="text/plain")

```

backend/app/api/image_tasks.py

```

from fastapi import APIRouter, Depends, HTTPException, Query

from app.api.auth import require_reviewer, require_user
from app.core.dependencies import get_store
from app.models.schemas import (
    ApiResponse,
    AuthUser,
    ImageTaskAnnotationUpdateRequest,
    ImageTaskCreateRequest,
    ImageTaskResult,
    ImageTaskResultSaveRequest,
    ImageTaskReviewRequest,
    ImageTaskResultVersion,
    ImageTaskType,
)
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/image-tasks", tags=["image-workspace"])

@router.post("/create", response_model=ApiResponse)
def create_image_task(
    request: ImageTaskCreateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    image = store.get_accessible_image_asset(request.image_id, current_user.user_id)
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")
    task = store.create_or_get_image_task(
        request.image_id,
        request.task_type,
        request.session_id or "default",
        current_user.user_id,
    )
    return ApiResponse(data=_task_payload(task))

@router.post("/result/save", response_model=ApiResponse)
def save_image_task_result(
    request: ImageTaskResultSaveRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:

```

```

    _require_edit_permission(current_user)
    if store.get_accessible_image_asset(request.image_id, current_user.user_id) is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")
    _ensure_result_editable(store, request.task_id)
    result = ImageTaskResult(
        task_id=request.task_id,
        image_id=request.image_id,
        task_type=request.task_type,
        result_image_url=request.result_image_url,
        result_image_path=request.result_image_path,
        result_json=request.result_json,
        annotation_json=request.annotation_json,
        description_text=request.description_text,
        source=request.source,
        model_id=request.model_id,
    )
    saved, version = store.save_image_task_result(result)
    return ApiResponse(data={"resultId": saved.result_id, "versionId": version.version_id, "versionNo":
version.version_no})

```

```

@router.get("/result/versions", response_model=ApiResponse)
def list_image_task_result_versions(
    image_id: str = Query(alias="imageId"),
    task_type: ImageTaskType = Query(alias="taskType"),
    session_id: str = Query(default="default", alias="sessionId"),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_image_access(store, image_id, current_user)
    versions = store.list_image_task_result_versions(image_id, task_type, session_id or "default")
    return ApiResponse(data=[_version_payload(version) for version in versions])

```

```

@router.get("/result/version/{version_id}", response_model=ApiResponse)
def get_image_task_result_version(
    version_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    version = store.get_image_task_result_version(version_id)
    if version is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    _require_image_access(store, version.image_id, current_user)
    return ApiResponse(data=_version_payload(version))

```

```

@router.post("/result/version/{version_id}/restore", response_model=ApiResponse)
def restore_image_task_result_version(
    version_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    existing = store.get_image_task_result_version(version_id)
    if existing is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    _require_image_access(store, existing.image_id, current_user)
    _ensure_result_editable(store, existing.task_id)
    restored = store.restore_image_task_result_version(version_id)
    if restored is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    result, version = restored
    return ApiResponse(data={*_result_payload(result), "versionId": version.version_id, "versionNo":
version.version_no})

```

```

@router.get("/result", response_model=ApiResponse)
def get_image_task_result(
    image_id: str = Query(alias="imageId"),
    task_type: ImageTaskType = Query(alias="taskType"),

```

```

    session_id: str = Query(default="default", alias="sessionId"),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_image_access(store, image_id, current_user)
    result = store.get_image_task_result(image_id, task_type, session_id or "default")
    if result is None:
        return ApiResponse(message="no result", data=None)
    return ApiResponse(data=_result_payload(result))

@router.put("/annotation", response_model=ApiResponse)
def update_image_task_annotation(
    request: ImageTaskAnnotationUpdateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    _require_image_access(store, request.image_id, current_user)
    _ensure_result_editable(store, request.task_id)
    result = store.update_image_task_annotation(
        request.task_id,
        request.image_id,
        request.task_type,
        request.annotation_json,
    )
    if result is None:
        raise HTTPException(status_code=404, detail="当前任务结果不存在")
    return ApiResponse(data=_result_payload(result))

@router.post("/result/{task_id}/submit", response_model=ApiResponse)
def submit_image_task_result(
    task_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    result = store.submit_image_task_result(task_id, current_user.user_id)
    if result is None:
        raise HTTPException(status_code=409, detail="任务不存在、不属于当前用户或当前状态不能提交")
    return ApiResponse(data=_result_payload(result))

@router.post("/result/{task_id}/review", response_model=ApiResponse)
def review_image_task_result(
    task_id: str,
    request: ImageTaskReviewRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_reviewer),
) -> ApiResponse:
    result = store.review_image_task_result(task_id, request.status, request.comment, current_user.user_id)
    if result is None:
        raise HTTPException(status_code=409, detail="只有其他用户提交的待审核任务可以执行审核")
    return ApiResponse(data=_result_payload(result))

def _require_image_access(store: PostgresStore, image_id: str, current_user: AuthUser) -> None:
    image = (
        store.get_image_asset(image_id)
        if current_user.role in {"reviewer", "admin"}
        else store.get_accessible_image_asset(image_id, current_user.user_id)
    )
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在或无权访问")

def _require_edit_permission(current_user: AuthUser) -> None:
    if current_user.role == "reviewer":
        raise HTTPException(status_code=403, detail="审核员只能查看和审核任务，不能修改标注")

```

```

def _ensure_result_editable(store: PostgresStore, task_id: str) -> None:
    result = store.get_image_task_result_by_task_id(task_id)
    if result is not None and result.review_status == "approved":
        raise HTTPException(status_code=409, detail="审核通过的任务已经锁定, 不能继续修改")

def _task_payload(task) -> dict:
    return {
        "taskId": task.task_id,
        "imageId": task.image_id,
        "taskType": task.task_type,
        "status": task.status,
        "createdAt": task.created_at.isoformat(),
        "updatedAt": task.updated_at.isoformat(),
    }

def _result_payload(result: ImageTaskResult) -> dict:
    return {
        "resultId": result.result_id,
        "taskId": result.task_id,
        "imageId": result.image_id,
        "taskType": result.task_type,
        "status": result.status,
        "resultImageUrl": result.result_image_url,
        "resultImagePath": result.result_image_path,
        "resultJson": result.result_json,
        "annotationJson": result.annotation_json,
        "descriptionText": result.description_text,
        "source": result.source,
        "modelId": result.model_id,
        "latestVersionId": result.latest_version_id,
        "latestVersionNo": result.latest_version_no,
        "reviewStatus": result.review_status,
        "submittedBy": result.submitted_by,
        "submittedAt": result.submitted_at.isoformat() if result.submitted_at else None,
        "reviewedBy": result.reviewed_by,
        "reviewedAt": result.reviewed_at.isoformat() if result.reviewed_at else None,
        "reviewComment": result.review_comment,
        "createdAt": result.created_at.isoformat(),
        "updatedAt": result.updated_at.isoformat(),
    }

def _version_payload(version: ImageTaskResultVersion) -> dict:
    return {
        "versionId": version.version_id,
        "resultId": version.result_id,
        "taskId": version.task_id,
        "imageId": version.image_id,
        "taskType": version.task_type,
        "versionNo": version.version_no,
        "source": version.source,
        "modelId": version.model_id,
        "resultImageUrl": version.result_image_url,
        "resultImagePath": version.result_image_path,
        "resultJson": version.result_json,
        "annotationJson": version.annotation_json,
        "descriptionText": version.description_text,
        "createdAt": version.created_at.isoformat(),
    }

```

backend/app/api/images.py

```

import shutil
from pathlib import Path
from uuid import uuid4

from fastapi import APIRouter, Depends, File, Form, HTTPException, Query, UploadFile

```

```

from PIL import Image

from app.core.config import Settings, get_settings
from app.core.dependencies import get_store
from app.api.auth import require_user
from app.models.schemas import ApiResponse, AuthUser, ImageAsset, ImageHistoryResponse, ImageTaskType, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/images", tags=["image-workspace"])

SUPPORTED_IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp", ".bmp"}
MAX_IMAGE_BYTES = 20 * 1024 * 1024

@router.post("/upload", response_model=ApiResponse)
def upload_image(
    file: UploadFile = File(...),
    task_type: ImageTaskType = Form(..., alias="taskType"),
    session_id: str = Form(default="default", alias="sessionId"),
    settings: Settings = Depends(get_settings),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    del task_type
    original_name = Path(file.filename or "upload.png").name
    extension = Path(original_name).suffix.lower()
    if extension not in SUPPORTED_IMAGE_EXTENSIONS:
        raise HTTPException(status_code=400, detail="图片格式不支持，请上传 jpg、jpeg、png、webp 或 bmp")

    current = now_utc()
    target_dir = settings.upload_dir / "images" / "original" / f"{current.year:04d}" / f"{current.month:02d}"
    target_dir.mkdir(parents=True, exist_ok=True)
    file_name = f"{uuid4().hex}{extension}"
    target_path = target_dir / file_name

    with target_path.open("wb") as handle:
        shutil.copyfileobj(file.file, handle)

    file_size = target_path.stat().st_size
    if file_size > MAX_IMAGE_BYTES:
        target_path.unlink(missing_ok=True)
        raise HTTPException(status_code=400, detail="图片大小超过 20MB 限制")

    try:
        with Image.open(target_path) as image:
            width, height = image.size
            image.verify()
    except Exception as exc:
        target_path.unlink(missing_ok=True)
        raise HTTPException(status_code=400, detail="图片文件无法解析") from exc

    relative_url = "/" + target_path.relative_to(settings.data_dir).as_posix()
    image_asset = ImageAsset(
        original_name=original_name,
        file_name=file_name,
        file_url=relative_url,
        file_path=str(target_path),
        mime_type=file.content_type or "application/octet-stream",
        file_size=file_size,
        width=width,
        height=height,
        user_id=current_user.user_id,
        session_id=session_id or "default",
    )
    store.create_image_asset(image_asset)
    return ApiResponse(data=_image_payload(image_asset))

@router.get("/list", response_model=ApiResponse)
def list_images(

```

```

task_type: ImageTaskType | None = Query(default=None, alias="taskType"),
page: int = Query(default=1, ge=1),
page_size: int = Query(default=50, ge=1, le=200, alias="pageSize"),
session_id: str | None = Query(default=None, alias="sessionId"),
store: PostgresStore = Depends(get_store),
current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    records, total = store.list_image_assets(
        task_type=task_type,
        session_id=session_id,
        user_id=None if current_user.role in {"reviewer", "admin"} else current_user.user_id,
        page=page,
        page_size=page_size,
    )
    data = ImageHistoryResponse(records=records, total=total)
    return ApiResponse(data={"records": [_image_payload(record) for record in data.records], "total": data.total})

def _image_payload(image: ImageAsset) -> dict:
    return {
        "imageId": image.image_id,
        "imageUrl": image.file_url,
        "originalName": image.original_name,
        "fileName": image.file_name,
        "fileUrl": image.file_url,
        "filePath": image.file_path,
        "mimeType": image.mime_type,
        "fileSize": image.file_size,
        "width": image.width,
        "height": image.height,
        "sessionId": image.session_id,
        "createdAt": image.created_at.isoformat(),
        "updatedAt": image.updated_at.isoformat(),
        "hasCurrentTaskResult": image.has_current_task_result,
        "taskStatus": image.task_status,
    }

```

backend/app/api/labels.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.api.auth import require_admin
from app.core.dependencies import get_store
from app.models.schemas import ApiResponse, AuthUser, LabelConfigCreateRequest, LabelConfigUpdateRequest
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/labels", tags=["labels"])

@router.get("")
def list_labels(store: PostgresStore = Depends(get_store)) -> ApiResponse:
    labels = store.list_label_configs()
    return ApiResponse(data=[label.model_dump(mode="json", by_alias=True) for label in labels])

@router.post("")
def create_label(request: LabelConfigCreateRequest, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) -> ApiResponse:
    try:
        label = store.create_label_config(
            english_name=request.english_name,
            chinese_name=request.chinese_name,
            description=request.description,
            copy_from_label_id=request.copy_from_label_id,
        )
    except Exception as exc:
        raise HTTPException(status_code=409, detail="标签数据不合法") from exc
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

```

```

@router.post("/{label_id}/copy")
def copy_label(label_id: int, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) ->
ApiResponse:
    label = store.copy_label_config(label_id)
    if label is None:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

@router.put("/{label_id}")
def update_label(
    label_id: int,
    request: LabelConfigUpdateRequest,
    store: PostgresStore = Depends(get_store),
    _: AuthUser = Depends(require_admin),
) -> ApiResponse:
    try:
        label = store.update_label_config(
            label_id=label_id,
            english_name=request.english_name,
            chinese_name=request.chinese_name,
            description=request.description,
        )
    except Exception as exc:
        raise HTTPException(status_code=409, detail="标签数据不合法") from exc
    if label is None:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

@router.delete("/{label_id}")
def delete_label(label_id: int, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) ->
ApiResponse:
    deleted = store.delete_label_config(label_id)
    if not deleted:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data={"deleted": True})

```

backend/app/api/tasks.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.core.config import Settings, get_settings
from app.core.dependencies import get_store, get_task_service
from app.models.schemas import DetectTask, DetectionRequest
from app.repositories.postgres_store import PostgresStore
from app.services.task_service import TaskService

router = APIRouter(prefix="/api/tasks", tags=["tasks"])

@router.post("/detect", response_model=DetectTask, status_code=202)
def create_detection_task(
    request: DetectionRequest,
    task_service: TaskService = Depends(get_task_service),
    settings: Settings = Depends(get_settings),
    store: PostgresStore = Depends(get_store),
) -> DetectTask:
    if store.get_asset(request.asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    task = task_service.create_detection_task(request)
    if settings.run_tasks_inline:
        task_service.run_detection(task.id)
    return store.get_task(task.id) or task
    return task

```

```

@router.get("", response_model=list[DetectTask])
def list_tasks(store: PostgresStore = Depends(get_store)) -> list[DetectTask]:
    return store.list_tasks()

@router.get("/{task_id}", response_model=DetectTask)
def get_task(task_id: str, store: PostgresStore = Depends(get_store)) -> DetectTask:
    task = store.get_task(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="task not found")
    return task

```

backend/app/api/video_captions.py

```

from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException, Query
from fastapi.responses import FileResponse

from app.core.dependencies import get_store
from app.models.schemas import ApiResponse, VideoCaptionBatch, VideoCaptionFrame, VideoCaptionVideo
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/video-captions", tags=["video-captions"])

@router.get("/batches", response_model=ApiResponse)
def list_batches(store: PostgresStore = Depends(get_store)) -> ApiResponse:
    return ApiResponse(data=[_batch_payload(batch) for batch in store.list_video_caption_batches()])

@router.get("/batches/{batch_id}", response_model=ApiResponse)
def get_batch(batch_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    batch = store.get_video_caption_batch(batch_id)
    if batch is None:
        raise HTTPException(status_code=404, detail="video caption batch not found")
    return ApiResponse(data=_batch_payload(batch))

@router.get("/batches/{batch_id}/videos", response_model=ApiResponse)
def list_videos(batch_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    if store.get_video_caption_batch(batch_id) is None:
        raise HTTPException(status_code=404, detail="video caption batch not found")
    return ApiResponse(data=[_video_payload(video) for video in store.list_video_caption_videos(batch_id)])

@router.get("/videos/{video_id}/frames", response_model=ApiResponse)
def list_frames(
    video_id: str,
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=100, ge=1, le=500, alias="pageSize"),
    query: str = Query(default=""),
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    if store.get_video_caption_video(video_id) is None:
        raise HTTPException(status_code=404, detail="video caption video not found")
    frames, total = store.list_video_caption_frames(video_id, page=page, page_size=page_size, query=query.strip())
    return ApiResponse(data={"records": [_frame_payload(frame) for frame in frames], "total": total})

@router.get("/frames/{frame_id}", response_model=ApiResponse)
def get_frame(frame_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    frame = store.get_video_caption_frame(frame_id)
    if frame is None:
        raise HTTPException(status_code=404, detail="video caption frame not found")
    return ApiResponse(data=_frame_payload(frame))

```

```

@router.get("/frames/{frame_id}/image")
def get_frame_image(frame_id: str, store: PostgresStore = Depends(get_store)) -> FileResponse:
    frame = store.get_video_caption_frame(frame_id)
    if frame is None:
        raise HTTPException(status_code=404, detail="video caption frame not found")
    image_path = Path(frame.image_path)
    if not image_path.exists() or not image_path.is_file():
        raise HTTPException(status_code=404, detail="video caption frame image not found")
    return FileResponse(image_path, media_type="image/jpeg")

def _batch_payload(batch: VideoCaptionBatch) -> dict:
    return {
        "batchId": batch.batch_id,
        "name": batch.name,
        "sourceDir": batch.source_dir,
        "outputDir": batch.output_dir,
        "modelId": batch.model_id,
        "prompt": batch.prompt,
        "frameIntervalSeconds": batch.frame_interval_seconds,
        "status": batch.status,
        "totalVideos": batch.total_videos,
        "totalFrames": batch.total_frames,
        "metadata": batch.metadata,
        "createdAt": batch.created_at.isoformat(),
        "updatedAt": batch.updated_at.isoformat(),
    }

def _video_payload(video: VideoCaptionVideo) -> dict:
    return {
        "videoId": video.video_id,
        "batchId": video.batch_id,
        "filename": video.filename,
        "displayName": video.display_name or video.filename,
        "keywords": video.keywords,
        "sourcePath": video.source_path,
        "frameDir": video.frame_dir,
        "fps": video.fps,
        "width": video.width,
        "height": video.height,
        "frameCount": video.frame_count,
        "durationMs": video.duration_ms,
        "status": video.status,
        "error": video.error,
        "createdAt": video.created_at.isoformat(),
        "updatedAt": video.updated_at.isoformat(),
    }

def _frame_payload(frame: VideoCaptionFrame) -> dict:
    return {
        "frameId": frame.frame_id,
        "batchId": frame.batch_id,
        "videoId": frame.video_id,
        "frameIndex": frame.frame_index,
        "timestampMs": frame.timestamp_ms,
        "imagePath": frame.image_path,
        "imageUrl": frame.image_url or f"/api/video-captions/frames/{frame.frame_id}/image",
        "width": frame.width,
        "height": frame.height,
        "descriptionText": frame.description_text,
        "modelId": frame.model_id,
        "status": frame.status,
        "error": frame.error,
        "resultJson": frame.result_json,
        "createdAt": frame.created_at.isoformat(),
        "updatedAt": frame.updated_at.isoformat(),
    }

```

backend/app/api/vision.py

```
from typing import Any

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile

from app.core.dependencies import get_vision_service
from app.services.vision_service import VisionFeature, VisionService

router = APIRouter(prefix="/api/vision", tags=["vision"])

@router.post("/analyze")
def analyze_image(
    feature: VisionFeature,
    model_name: str | None = None,
    file: UploadFile = File(...),
    service: VisionService = Depends(get_vision_service),
) -> dict[str, Any]:
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="only image uploads supported")
    try:
        return service.analyze_upload(file, feature, model_name)
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc
```

backend/app/core/__init__.py

```
"""Core backend settings and dependencies."""
```

backend/app/core/config.py

```
from functools import lru_cache
from pathlib import Path

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Railway Multi-Target Recognition API"
    data_dir: Path = Path("./data")
    database_url: str = "postgresql+psycopg://deipss:CHANGE_ME@***"
    inference_backend: str = Field(default="mock", pattern="^(mock|ultralitics|tensorrt)$")
    model_path: str = ""
    deepseek_vl2_model_path: str = "./models/deepseek-vl2-tiny"
    deepseek_vl2_prompt: str = "请用中文简要描述这张图片的主要内容，并列出可见目标。"
    device: str = "0"
    run_tasks_inline: bool = False
    task_worker_enabled: bool = True
    task_poll_interval_seconds: float = Field(default=1.0, gt=0)
    cors_origins: str = ""

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    @property
    def upload_dir(self) -> Path:
        return self.data_dir / "uploads"

    @property
    def allowed_origins(self) -> list[str]:
        return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]

    @lru_cache
    def get_settings() -> Settings:
        settings = Settings()
```

```
settings.upload_dir.mkdir(parents=True, exist_ok=True)
settings.data_dir.mkdir(parents=True, exist_ok=True)
return settings
```

backend/app/core/dependencies.py

```
from functools import lru_cache

from app.core.config import get_settings
from app.repositories.postgres_store import PostgresStore
from app.services.export_service import ExportService
from app.services.inference_service import InferenceService
from app.services.storage_service import StorageService
from app.services.task_service import TaskService
from app.services.vision_service import VisionService

@lru_cache
def get_store() -> PostgresStore:
    return PostgresStore(get_settings().database_url)

def get_storage_service() -> StorageService:
    settings = get_settings()
    return StorageService(settings.upload_dir)

def get_inference_service() -> InferenceService:
    return InferenceService(get_settings(), get_storage_service())

def get_task_service() -> TaskService:
    return TaskService(get_store(), get_inference_service())

def get_export_service() -> ExportService:
    return ExportService(get_store())

@lru_cache
def get_vision_service() -> VisionService:
    return VisionService(get_settings())
```

backend/app/core/security.py

```
import hashlib
import hmac
import secrets

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return f"scrypt$16384$8$1${salt.hex()}${digest.hex()}"

def verify_password(password: str, encoded: str) -> bool:
    try:
        algorithm, n, r, p, salt_hex, digest_hex = encoded.split("$", 5)
        if algorithm != "scrypt":
            return False
        actual = hashlib.scrypt(
            password.encode("utf-8"),
            salt=bytes.fromhex(salt_hex),
            n=int(n),
            r=int(r),
            p=int(p),
            dklen=len(bytes.fromhex(digest_hex)),
        )
        return hmac.compare_digest(actual, digest_hex)
```

```

    )
    return hmac.compare_digest(actual, bytes.fromhex(digest_hex))
except (TypeError, ValueError):
    return False

```

```

def new_session_token() -> str:
    return secrets.token_urlsafe(48)

```

```

def hash_session_token(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()

```

backend/app/main.py

```

from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from fastapi.staticfiles import StaticFiles

from app.api import ai, annotations, assets, auth, export, image_tasks, images, labels, tasks, video_captions, vision
from app.core.config import get_settings
from app.core.dependencies import get_store, get_task_service
from app.services.task_worker import DatabaseTaskWorker

def create_app() -> FastAPI:
    settings = get_settings()

    @asynccontextmanager
    async def lifespan(_: FastAPI):
        worker = None
        if settings.task_worker_enabled and not settings.run_tasks_inline:
            worker = DatabaseTaskWorker(
                get_store(),
                get_task_service(),
                settings.task_poll_interval_seconds,
            )
        worker.start()
        try:
            yield
        finally:
            if worker is not None:
                worker.stop()
            get_store().close()

    app = FastAPI(title=settings.app_name, lifespan=lifespan)
    app.include_router(auth.router)

    @app.middleware("http")
    async def require_authenticated_session(request: Request, call_next):
        protected_path = request.url.path.startswith("/api/") or request.url.path.startswith("/uploads/")
        public_auth_path = request.url.path.startswith("/api/auth/")
        if protected_path and not public_auth_path and request.method != "OPTIONS":
            user = auth.resolve_user(request, get_store())
            if user is None:
                return JSONResponse(status_code=401, content={"detail": "登录状态已失效, 请重新登录"})
            request.state.current_user = user
        return await call_next(request)

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.allowed_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

```

```

)

app.include_router/assets.router)
app.include_router(tasks.router)
app.include_router(annotations.router)
app.include_router(export.router)
app.include_router(vision.router)
app.include_router(images.router)
app.include_router(image_tasks.router)
app.include_router(ai.router)
app.include_router(labels.router)
app.include_router(video_captions.router)
app.mount("/uploads", StaticFiles(directory=settings.upload_dir), name="uploads")

@app.get("/health")
def health() -> dict[str, str]:
    database_status = "ok" if get_store().healthcheck() else "unavailable"
    return {
        "status": "ok",
        "database": database_status,
        "inference_backend": settings.inference_backend,
    }

return app

app = create_app()

```

backend/app/models/__init__.py

```

"""Pydantic schemas and persistence models."""

```

backend/app/models/schemas.py

```

from datetime import datetime, timezone
from typing import Any, Literal
from uuid import uuid4

from pydantic import BaseModel, ConfigDict, Field

AssetType = Literal["image", "video", "unknown"]
TaskStatus = Literal["queued", "running", "completed", "failed"]
ObjectStatus = Literal["auto", "confirmed", "edited", "rejected"]
ObjectSource = Literal["auto", "manual"]
ReviewStatus = Literal["pending_review", "in_review", "approved", "rejected"]
ImageTaskType = Literal["detection", "segmentation", "pose", "classification", "caption"]
ImageTaskStatus = Literal["idle", "pending", "processing", "success", "failed"]
ResultSource = Literal["ai", "manual", "edited"]
ImageReviewStatus = Literal["draft", "pending_review", "approved", "rejected"]
VideoCaptionStatus = Literal["pending", "processing", "success", "failed"]

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid4().hex[:12]}"

def now_utc() -> datetime:
    return datetime.now(timezone.utc)

class BBox(BaseModel):
    x: float = Field(ge=0)
    y: float = Field(ge=0)
    width: float = Field(gt=0)
    height: float = Field(gt=0)

```

```

class DetectionObject(BaseModel):
    id: str = Field(default_factory=lambda: new_id("obj"))
    label: str
    confidence: float = Field(ge=0, le=1)
    bbox: BBox
    track_id: int | None = None
    source: ObjectSource = "auto"
    status: ObjectStatus = "auto"

class AnnotationFrame(BaseModel):
    frame_index: int = Field(ge=0)
    timestamp_ms: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    image_url: str | None = None
    review_status: ReviewStatus = "pending_review"
    objects: list[DetectionObject] = Field(default_factory=list)

class AnnotationsDocument(BaseModel):
    asset_id: str
    type: AssetType
    model: str
    review_status: ReviewStatus = "pending_review"
    frames: list[AnnotationFrame] = Field(default_factory=list)
    updated_at: datetime = Field(default_factory=now_utc)
    reviewed_at: datetime | None = None

class Asset(BaseModel):
    id: str = Field(default_factory=lambda: new_id("asset"))
    filename: str
    content_type: str
    type: AssetType = "unknown"
    path: str
    size_bytes: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    frame_count: int | None = Field(default=None, ge=1)
    duration_ms: int | None = Field(default=None, ge=0)
    fps: float | None = Field(default=None, gt=0)
    created_at: datetime = Field(default_factory=now_utc)

class DetectionRequest(BaseModel):
    asset_id: str
    confidence: float = Field(default=0.25, ge=0, le=1)
    iou: float = Field(default=0.7, ge=0, le=1)
    frame_stride: int = Field(default=1, ge=1)
    max_frames: int | None = Field(default=None, ge=1)
    model_name: str | None = None

class ReviewRequest(BaseModel):
    status: ReviewStatus

class DetectTask(BaseModel):
    id: str = Field(default_factory=lambda: new_id("task"))
    asset_id: str
    status: TaskStatus = "queued"
    progress: float = Field(default=0, ge=0, le=1)
    model_name: str = "mock"
    params: DetectionRequest
    error: str | None = None
    created_at: datetime = Field(default_factory=now_utc)
    started_at: datetime | None = None
    finished_at: datetime | None = None

```

```

class ImageAsset(BaseModel):
    image_id: str = Field(default_factory=lambda: new_id("img"))
    original_name: str
    file_name: str
    file_url: str
    file_path: str
    mime_type: str | None = None
    file_size: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    user_id: str | None = None
    session_id: str = "default"
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)
    has_current_task_result: bool = False
    task_status: ImageTaskStatus = "idle"

class ImageHistoryResponse(BaseModel):
    records: List[ImageAsset]
    total: int

class ImageTask(BaseModel):
    task_id: str = Field(default_factory=lambda: new_id("itask"))
    image_id: str
    task_type: ImageTaskType
    status: ImageTaskStatus = "pending"
    user_id: str | None = None
    session_id: str = "default"
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

class ImageTaskCreateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    session_id: str | None = Field(default=None, alias="sessionId")
    user_id: str | None = Field(default=None, alias="userId")

class ImageTaskResult(BaseModel):
    result_id: str = Field(default_factory=lambda: new_id("ires"))
    task_id: str
    image_id: str
    task_type: ImageTaskType
    status: ImageTaskStatus = "success"
    result_image_url: str = ""
    result_image_path: str = ""
    result_json: dict[str, Any] | None = None
    annotation_json: dict[str, Any] | None = None
    description_text: str = ""
    source: ResultSource = "edited"
    model_id: str = ""
    latest_version_id: str | None = None
    latest_version_no: int | None = None
    review_status: ImageReviewStatus = "draft"
    submitted_by: str | None = None
    submitted_at: datetime | None = None
    reviewed_by: str | None = None
    reviewed_at: datetime | None = None
    review_comment: str = ""
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

class ImageTaskResultSaveRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

```

```
task_id: str = Field(alias="taskId")
image_id: str = Field(alias="imageId")
task_type: ImageTaskType = Field(alias="taskType")
result_image_url: str = Field(default="", alias="resultImageUrl")
result_image_path: str = Field(default="", alias="resultImagePath")
result_json: dict[str, Any] | None = Field(default=None, alias="resultJson")
annotation_json: dict[str, Any] | None = Field(default=None, alias="annotationJson")
description_text: str = Field(default="", alias="descriptionText")
source: ResultSource = "edited"
model_id: str = Field(default="", alias="modelId")
```

```
class ImageTaskResultVersion(BaseModel):
    version_id: str = Field(default_factory=lambda: new_id("iver"))
    result_id: str
    task_id: str
    image_id: str
    task_type: ImageTaskType
    version_no: int = Field(ge=1)
    source: ResultSource = "edited"
    model_id: str = ""
    result_image_url: str = ""
    result_image_path: str = ""
    result_json: dict[str, Any] | None = None
    annotation_json: dict[str, Any] | None = None
    description_text: str = ""
    created_at: datetime = Field(default_factory=now_utc)
```

```
class ImageTaskAnnotationUpdateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    task_id: str = Field(alias="taskId")
    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    annotation_json: dict[str, Any] = Field(alias="annotationJson")
```

```
class ImageTaskReviewRequest(BaseModel):
    status: Literal["approved", "rejected"]
    comment: str = Field(default="", max_length=1000)
```

```
class AuthUser(BaseModel):
    user_id: str = Field(default_factory=lambda: new_id("user"), alias="userId")
    username: str
    display_name: str = Field(alias="displayName")
    role: str = "student"
    is_active: bool = Field(default=True, alias="isActive")
    created_at: datetime = Field(default_factory=now_utc, alias="createdAt")
    updated_at: datetime = Field(default_factory=now_utc, alias="updatedAt")

    model_config = ConfigDict(populate_by_name=True)
```

```
class AuthRegisterRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    username: str = Field(min_length=3, max_length=50, pattern=r"^[A-Za-z0-9_.-]+$")
    display_name: str = Field(min_length=1, max_length=80, alias="displayName")
    password: str = Field(min_length=6, max_length=128)
```

```
class AuthLoginRequest(BaseModel):
    username: str = Field(min_length=1, max_length=50)
    password: str = Field(min_length=1, max_length=128)
```

```
class AuthUserRoleUpdateRequest(BaseModel):
    role: Literal["student", "reviewer", "admin"]
```

```

class AiInferRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    model_name: str | None = Field(default=None, alias="modelName")
    session_id: str | None = Field(default=None, alias="sessionId")

class LabelConfig(BaseModel):
    label_id: int = Field(ge=0, alias="labelId")
    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""
    created_at: datetime = Field(default_factory=now_utc, alias="createdAt")
    updated_at: datetime = Field(default_factory=now_utc, alias="updatedAt")

    model_config = ConfigDict(populate_by_name=True)

class LabelConfigCreateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""
    copy_from_label_id: int | None = Field(default=None, ge=0, alias="copyFromLabelId")

class LabelConfigUpdateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""

class ApiResponse(BaseModel):
    code: int = 200
    message: str = "success"
    data: Any = None

class ErrorResponse(BaseModel):
    detail: str

class VideoCaptionBatch(BaseModel):
    batch_id: str = Field(default_factory=lambda: new_id("vcbatch"))
    name: str = Field(min_length=1, max_length=160)
    source_dir: str = ""
    output_dir: str = ""
    model_id: str = "deepseek-ai/deepseek-vl2-tiny"
    prompt: str = ""
    frame_interval_seconds: float = Field(default=1.0, gt=0)
    status: VideoCaptionStatus = "pending"
    total_videos: int = Field(default=0, ge=0)
    total_frames: int = Field(default=0, ge=0)
    metadata: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

class VideoCaptionVideo(BaseModel):
    video_id: str = Field(default_factory=lambda: new_id("vcvideo"))
    batch_id: str
    filename: str
    display_name: str = ""
    keywords: list[str] = Field(default_factory=list)
    source_path: str

```

```

frame_dir: str = ""
fps: float | None = Field(default=None, gt=0)
width: int | None = Field(default=None, ge=1)
height: int | None = Field(default=None, ge=1)
frame_count: int | None = Field(default=None, ge=1)
duration_ms: int | None = Field(default=None, ge=0)
status: VideoCaptionStatus = "pending"
error: str = ""
created_at: datetime = Field(default_factory=now_utc)
updated_at: datetime = Field(default_factory=now_utc)

```

```

class VideoCaptionFrame(BaseModel):
    frame_id: str = Field(default_factory=lambda: new_id("vcframe"))
    batch_id: str
    video_id: str
    frame_index: int = Field(ge=0)
    timestamp_ms: int = Field(ge=0)
    image_path: str
    image_url: str = ""
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    description_text: str = ""
    model_id: str = "deepseek-ai/deepseek-vl2-tiny"
    status: VideoCaptionStatus = "pending"
    error: str = ""
    result_json: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

```

backend/app/repositories/__init__.py

```

"""Persistence adapters."""

```

backend/app/repositories/postgres_store.py

```

from datetime import datetime

from sqlalchemy import JSON, Boolean, DateTime, Float, Integer, String, Text, UniqueConstraint, create_engine,
func, or_, select, text
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, sessionmaker

from app.models.schemas import (
    AnnotationsDocument,
    AuthUser,
    Asset,
    DetectTask,
    ImageAsset,
    ImageTask,
    ImageTaskResult,
    ImageTaskResultVersion,
    ImageTaskStatus,
    ImageTaskType,
    LabelConfig,
    VideoCaptionBatch,
    VideoCaptionFrame,
    VideoCaptionVideo,
    now_utc,
)

class Base(DeclarativeBase):
    pass

class AssetRecord(Base):
    __tablename__ = "assets"

```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class TaskRecord(Base):
    __tablename__ = "detect_tasks"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
asset_id: Mapped[str] = mapped_column(String(64), index=True)
status: Mapped[str] = mapped_column(String(32), index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class AnnotationRecord(Base):
    __tablename__ = "annotations"
```

```
asset_id: Mapped[str] = mapped_column(String(64), primary_key=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class UserRecord(Base):
    __tablename__ = "app_user"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
username: Mapped[str] = mapped_column(String(50), unique=True, index=True)
display_name: Mapped[str] = mapped_column(String(80))
password_hash: Mapped[str] = mapped_column(String(255))
role: Mapped[str] = mapped_column(String(32), default="student", index=True)
is_active: Mapped[bool] = mapped_column(Boolean, default=True, index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class AuthSessionRecord(Base):
    __tablename__ = "auth_session"
```

```
token_hash: Mapped[str] = mapped_column(String(64), primary_key=True)
user_id: Mapped[str] = mapped_column(String(64), index=True)
expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
last_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class ImageAssetRecord(Base):
    __tablename__ = "image_asset"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
original_name: Mapped[str] = mapped_column(String(255))
file_name: Mapped[str] = mapped_column(String(255))
file_url: Mapped[str] = mapped_column(String(500))
file_path: Mapped[str] = mapped_column(String(500))
mime_type: Mapped[str | None] = mapped_column(String(100), nullable=True)
file_size: Mapped[int] = mapped_column(Integer)
width: Mapped[int | None] = mapped_column(Integer, nullable=True)
height: Mapped[int | None] = mapped_column(Integer, nullable=True)
user_id: Mapped[str | None] = mapped_column(String(100), nullable=True, index=True)
session_id: Mapped[str] = mapped_column(String(100), default="default", index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class ImageTaskRecord(Base):
    __tablename__ = "image_task"
    __table_args__ = (
        UniqueConstraint("image_id", "task_type", "user_key", "session_id", name="uk_image_task_scope"),
    )
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
```

```

image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
status: Mapped[str] = mapped_column(String(50), default="pending", index=True)
user_id: Mapped[str | None] = mapped_column(String(100), nullable=True, index=True)
user_key: Mapped[str] = mapped_column(String(100), default="", index=True)
session_id: Mapped[str] = mapped_column(String(100), default="default", index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))

```

```

class ImageTaskResultRecord(Base):
    __tablename__ = "image_task_result"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
task_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
status: Mapped[str] = mapped_column(String(50), default="success", index=True)
result_image_url: Mapped[str] = mapped_column(String(500), default="")
result_image_path: Mapped[str] = mapped_column(String(500), default="")
result_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
annotation_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
description_text: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
has_result: Mapped[bool] = mapped_column(Boolean, default=True)
review_status: Mapped[str] = mapped_column(String(32), default="draft", index=True)
submitted_by: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
submitted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
reviewed_by: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
reviewed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
review_comment: Mapped[str] = mapped_column(Text, default="")

```

```

class ImageTaskResultVersionRecord(Base):
    __tablename__ = "image_task_result_version"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
result_id: Mapped[str] = mapped_column(String(64), index=True)
task_id: Mapped[str] = mapped_column(String(64), index=True)
image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
version_no: Mapped[int] = mapped_column(Integer, index=True)
source: Mapped[str] = mapped_column(String(32), default="edited", index=True)
model_id: Mapped[str] = mapped_column(String(100), default="")
result_image_url: Mapped[str] = mapped_column(String(500), default="")
result_image_path: Mapped[str] = mapped_column(String(500), default="")
result_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
annotation_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
description_text: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```

class LabelConfigRecord(Base):
    __tablename__ = "label_config"

```

```

label_id: Mapped[int] = mapped_column(Integer, primary_key=True)
english_name: Mapped[str] = mapped_column(String(120), index=True)
chinese_name: Mapped[str] = mapped_column(String(120), index=True)
description: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```

class VideoCaptionBatchRecord(Base):
    __tablename__ = "video_caption_batch"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
name: Mapped[str] = mapped_column(String(160), unique=True, index=True)
source_dir: Mapped[str] = mapped_column(String(1000), default="")
output_dir: Mapped[str] = mapped_column(String(1000), default="")

```

```

model_id: Mapped[str] = mapped_column(String(160), default="")
prompt: Mapped[str] = mapped_column(Text, default="")
frame_interval_seconds: Mapped[float] = mapped_column(Float, default=1.0)
status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
total_videos: Mapped[int] = mapped_column(Integer, default=0)
total_frames: Mapped[int] = mapped_column(Integer, default=0)
metadata_json: Mapped[dict] = mapped_column(JSON, default=dict)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class VideoCaptionVideoRecord(Base):
```

```

    __tablename__ = "video_caption_video"
    __table_args__ = (
        UniqueConstraint("batch_id", "source_path", name="uk_video_caption_video_source"),
    )

```

```

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    batch_id: Mapped[str] = mapped_column(String(64), index=True)
    filename: Mapped[str] = mapped_column(String(255), index=True)
    display_name: Mapped[str] = mapped_column(String(500), default="")
    keywords_json: Mapped[list] = mapped_column(JSON, default=list)
    source_path: Mapped[str] = mapped_column(String(1000))
    frame_dir: Mapped[str] = mapped_column(String(1000), default="")
    fps: Mapped[float | None] = mapped_column(Float, nullable=True)
    width: Mapped[int | None] = mapped_column(Integer, nullable=True)
    height: Mapped[int | None] = mapped_column(Integer, nullable=True)
    frame_count: Mapped[int | None] = mapped_column(Integer, nullable=True)
    duration_ms: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
    error: Mapped[str] = mapped_column(Text, default="")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class VideoCaptionFrameRecord(Base):
```

```

    __tablename__ = "video_caption_frame"
    __table_args__ = (
        UniqueConstraint("video_id", "frame_index", name="uk_video_caption_frame_index"),
    )

```

```

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    batch_id: Mapped[str] = mapped_column(String(64), index=True)
    video_id: Mapped[str] = mapped_column(String(64), index=True)
    frame_index: Mapped[int] = mapped_column(Integer, index=True)
    timestamp_ms: Mapped[int] = mapped_column(Integer, index=True)
    image_path: Mapped[str] = mapped_column(String(1000))
    image_url: Mapped[str] = mapped_column(String(500), default="")
    width: Mapped[int | None] = mapped_column(Integer, nullable=True)
    height: Mapped[int | None] = mapped_column(Integer, nullable=True)
    description_text: Mapped[str] = mapped_column(Text, default="")
    model_id: Mapped[str] = mapped_column(String(160), default="")
    status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
    error: Mapped[str] = mapped_column(Text, default="")
    result_json: Mapped[dict] = mapped_column(JSON, default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class PostgresStore:
```

```

    def __init__(self, database_url: str) -> None:
        connect_args = {"check_same_thread": False} if database_url.startswith("sqlite") else {}
        self.engine = create_engine(database_url, pool_pre_ping=True, connect_args=connect_args)
        self.session_factory = sessionmaker(self.engine, expire_on_commit=False)
        Base.metadata.create_all(self.engine)
        self._ensure_label_config_schema(database_url)
        self._ensure_video_caption_schema(database_url)
        self._ensure_image_task_review_schema(database_url)
        self._ensure_admin_user()

```

```

    def create_asset(self, asset: Asset) -> Asset:

```

```

with self.session_factory.begin() as session:
    record = AssetRecord(
        id=asset.id,
        created_at=asset.created_at,
        payload=asset.model_dump(mode="json"),
    )
    session.merge(record)
return asset

def create_user(self, username: str, display_name: str, password_hash: str) -> AuthUser | None:
    normalized_username = username.strip().lower()
    with self.session_factory.begin() as session:
        if session.scalar(select(UserRecord).where(UserRecord.username == normalized_username)):
            return None
        role = "admin" if (session.scalar(select(func.count()).select_from(UserRecord)) or 0) == 0 else
"student"
        user = AuthUser(username=normalized_username, displayName=display_name.strip(), role=role)
        session.add(
            UserRecord(
                id=user.user_id,
                username=user.username,
                display_name=user.display_name,
                password_hash=password_hash,
                role=user.role,
                is_active=user.is_active,
                created_at=user.created_at,
                updated_at=user.updated_at,
            )
        )
    return user

def get_user_credentials(self, username: str) -> tuple[AuthUser, str] | None:
    with self.session_factory() as session:
        record = session.scalar(select(UserRecord).where(UserRecord.username == username.strip().lower()))
        if record is None:
            return None
        return self._auth_user_from_record(record), record.password_hash

def create_auth_session(self, token_hash: str, user_id: str, expires_at: datetime) -> None:
    current = now_utc()
    with self.session_factory.begin() as session:
        session.add(
            AuthSessionRecord(
                token_hash=token_hash,
                user_id=user_id,
                expires_at=expires_at,
                created_at=current,
                last_seen_at=current,
            )
        )

def get_user_by_auth_session(self, token_hash: str) -> AuthUser | None:
    current = now_utc()
    with self.session_factory.begin() as session:
        auth_session = session.scalar(
            select(AuthSessionRecord).where(
                AuthSessionRecord.token_hash == token_hash,
                AuthSessionRecord.expires_at > current,
            )
        )
    if auth_session is None:
        expired_session = session.get(AuthSessionRecord, token_hash)
        if expired_session is not None:
            session.delete(expired_session)
        return None
    user = session.get(UserRecord, auth_session.user_id)
    if user is None or not user.is_active:
        session.delete(auth_session)
        return None
    auth_session.last_seen_at = current
    return self._auth_user_from_record(user)

```

```

def delete_auth_session(self, token_hash: str) -> None:
    with self.session_factory.begin() as session:
        record = session.get(AuthSessionRecord, token_hash)
        if record is not None:
            session.delete(record)

def list_users(self) -> List[AuthUser]:
    with self.session_factory() as session:
        records = session.scalars(select(UserRecord).order_by(UserRecord.created_at.asc())).all()
        return [self._auth_user_from_record(record) for record in records]

def update_user_role(self, user_id: str, role: str) -> AuthUser | None:
    with self.session_factory.begin() as session:
        record = session.get(UserRecord, user_id)
        if record is None:
            return None
        record.role = role
        record.updated_at = now_utc()
        return self._auth_user_from_record(record)

def list_assets(self) -> List[Asset]:
    with self.session_factory() as session:
        records = session.scalars(select(AssetRecord).order_by(AssetRecord.created_at.desc())).all()
        return [Asset.model_validate(record.payload) for record in records]

def get_asset(self, asset_id: str) -> Asset | None:
    with self.session_factory() as session:
        record = session.get(AssetRecord, asset_id)
        return Asset.model_validate(record.payload) if record else None

def create_task(self, task: DetectTask) -> DetectTask:
    with self.session_factory.begin() as session:
        session.add(self._task_record(task))
    return task

def update_task(self, task: DetectTask) -> DetectTask:
    with self.session_factory.begin() as session:
        record = session.get(TaskRecord, task.id)
        if record is None:
            session.add(self._task_record(task))
        else:
            record.status = task.status
            record.payload = task.model_dump(mode="json")
    return task

def claim_next_task(self) -> DetectTask | None:
    with self.session_factory.begin() as session:
        statement = (
            select(TaskRecord)
            .where(TaskRecord.status == "queued")
            .order_by(TaskRecord.created_at.asc())
            .with_for_update(skip_locked=True)
            .limit(1)
        )
        record = session.scalar(statement)
        if record is None:
            return None
        task = DetectTask.model_validate(record.payload)
        task.status = "running"
        task.progress = 0.05
        task.started_at = now_utc()
        record.status = task.status
        record.payload = task.model_dump(mode="json")
        return task

def list_tasks(self) -> List[DetectTask]:
    with self.session_factory() as session:
        records = session.scalars(select(TaskRecord).order_by(TaskRecord.created_at.desc())).all()
        return [DetectTask.model_validate(record.payload) for record in records]

```

```

def get_task(self, task_id: str) -> DetectTask | None:
    with self.session_factory() as session:
        record = session.get(TaskRecord, task_id)
        return DetectTask.model_validate(record.payload) if record else None

def save_annotations(self, annotations: AnnotationsDocument) -> AnnotationsDocument:
    with self.session_factory.begin() as session:
        record = session.get(AnnotationRecord, annotations.asset_id)
        payload = annotations.model_dump(mode="json")
        if record is None:
            session.add(
                AnnotationRecord(
                    asset_id=annotations.asset_id,
                    updated_at=annotations.updated_at,
                    payload=payload,
                )
            )
        else:
            record.updated_at = annotations.updated_at
            record.payload = payload
    return annotations

def get_annotations(self, asset_id: str) -> AnnotationsDocument | None:
    with self.session_factory() as session:
        record = session.get(AnnotationRecord, asset_id)
        return AnnotationsDocument.model_validate(record.payload) if record else None

def create_image_asset(self, image: ImageAsset) -> ImageAsset:
    with self.session_factory.begin() as session:
        session.add(self._image_asset_record(image))
    return image

def get_image_asset(self, image_id: str) -> ImageAsset | None:
    with self.session_factory() as session:
        record = session.get(ImageAssetRecord, image_id)
        return self._image_asset_from_record(record) if record else None

def get_accessible_image_asset(self, image_id: str, user_id: str) -> ImageAsset | None:
    with self.session_factory() as session:
        record = session.scalar(
            select(ImageAssetRecord).where(
                ImageAssetRecord.id == image_id,
                or_(ImageAssetRecord.user_id == user_id, ImageAssetRecord.user_id.is_(None)),
            )
        )
        return self._image_asset_from_record(record) if record else None

def list_image_assets(
    self,
    task_type: ImageTaskType | None = None,
    session_id: str | None = None,
    user_id: str | None = None,
    page: int = 1,
    page_size: int = 50,
) -> tuple[list[ImageAsset], int]:
    offset = max(page - 1, 0) * page_size
    with self.session_factory() as session:
        base = select(ImageAssetRecord)
        count_query = select(func.count()).select_from(ImageAssetRecord)
        if session_id:
            base = base.where(ImageAssetRecord.session_id == session_id)
            count_query = count_query.where(ImageAssetRecord.session_id == session_id)
        if user_id:
            ownership = or_(ImageAssetRecord.user_id == user_id, ImageAssetRecord.user_id.is_(None))
            base = base.where(ownership)
            count_query = count_query.where(ownership)
        base = base.order_by(ImageAssetRecord.created_at.desc())
        records = session.scalars(base.offset(offset).limit(page_size)).all()
        total = session.scalar(count_query)
        images = [self._image_asset_from_record(record) for record in records]
        if task_type:

```

```

        for image in images:
            task = self.get_image_task(image.image_id, task_type, image.session_id)
            result = self.get_image_task_result(image.image_id, task_type, image.session_id)
            image.task_status = task.status if task else "idle"
            image.has_current_task_result = result is not None
        return images, int(total or 0)

def create_or_get_image_task(
    self,
    image_id: str,
    task_type: ImageTaskType,
    session_id: str = "default",
    user_id: str | None = None,
) -> ImageTask:
    user_key = user_id or ""
    with self.session_factory.begin() as session:
        existing = session.scalar(
            select(ImageTaskRecord).where(
                ImageTaskRecord.image_id == image_id,
                ImageTaskRecord.task_type == task_type,
                ImageTaskRecord.session_id == session_id,
                ImageTaskRecord.user_key == user_key,
            )
        )
        if existing:
            return self._image_task_from_record(existing)
        task = ImageTask(image_id=image_id, task_type=task_type, session_id=session_id, user_id=user_id)
        session.add(self._image_task_record(task))
        return task

def update_image_task_status(self, task_id: str, status: ImageTaskStatus) -> None:
    with self.session_factory.begin() as session:
        record = session.get(ImageTaskRecord, task_id)
        if record:
            record.status = status
            record.updated_at = now_utc()

def get_image_task(
    self,
    image_id: str,
    task_type: ImageTaskType,
    session_id: str = "default",
) -> ImageTask | None:
    with self.session_factory() as session:
        record = session.scalar(
            select(ImageTaskRecord).where(
                ImageTaskRecord.image_id == image_id,
                ImageTaskRecord.task_type == task_type,
                ImageTaskRecord.session_id == session_id,
            )
        )
        return self._image_task_from_record(record) if record else None

def save_image_task_result(self, result: ImageTaskResult) -> tuple[ImageTaskResult, ImageTaskResultVersion]:
    with self.session_factory.begin() as session:
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id ==
result.task_id))
        if record is None:
            record = self._image_task_result_record(result)
            session.add(record)
        else:
            record.status = result.status
            record.result_image_url = result.result_image_url
            record.result_image_path = result.result_image_path
            record.result_json = result.result_json
            record.annotation_json = result.annotation_json
            record.description_text = result.description_text
            record.review_status = "draft"
            record.reviewed_by = None
            record.reviewed_at = None
            record.review_comment = ""

```

```

        record.updated_at = now_utc()
        result.result_id = record.id
    task = session.get(ImageTaskRecord, result.task_id)
    if task:
        task.status = result.status
        task.updated_at = now_utc()
    version_no = self._next_result_version_no(session, result.task_id)
    version = ImageTaskResultVersion(
        result_id=record.id,
        task_id=result.task_id,
        image_id=result.image_id,
        task_type=result.task_type,
        version_no=version_no,
        source=result.source,
        model_id=result.model_id,
        result_image_url=result.result_image_url,
        result_image_path=result.result_image_path,
        result_json=result.result_json,
        annotation_json=result.annotation_json,
        description_text=result.description_text,
    )
    session.add(self._image_task_result_version_record(version))
    result.latest_version_id = version.version_id
    result.latest_version_no = version.version_no
    return result, version

def get_image_task_result(
    self,
    image_id: str,
    task_type: ImageTaskType,
    session_id: str = "default",
) -> ImageTaskResult | None:
    with self.session_factory() as session:
        task = session.scalar(
            select(ImageTaskRecord).where(
                ImageTaskRecord.image_id == image_id,
                ImageTaskRecord.task_type == task_type,
                ImageTaskRecord.session_id == session_id,
            )
        )
        if not task:
            return None
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id == task.id))
        return self._image_task_result_from_record(record) if record else None

def get_image_task_result_by_task_id(self, task_id: str) -> ImageTaskResult | None:
    with self.session_factory() as session:
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id == task_id))
        return self._image_task_result_from_record(record) if record else None

def submit_image_task_result(self, task_id: str, user_id: str) -> ImageTaskResult | None:
    with self.session_factory.begin() as session:
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id == task_id))
        task = session.get(ImageTaskRecord, task_id)
        if (
            record is None
            or task is None
            or record.review_status not in {"draft", "rejected"}
            or (task.user_id is not None and task.user_id != user_id)
        ):
            return None
        record.review_status = "pending_review"
        record.submitted_by = user_id
        record.submitted_at = now_utc()
        record.reviewed_by = None
        record.reviewed_at = None
        record.review_comment = ""
        record.updated_at = now_utc()
        return self._image_task_result_from_record(record)

def review_image_task_result(self, task_id: str, status: str, comment: str, reviewer_id: str) ->

```

```

ImageTaskResult | None:
    with self.session_factory.begin() as session:
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id == task_id))
        if record is None or record.review_status != "pending_review" or record.submitted_by == reviewer_id:
            return None
        record.review_status = status
        record.reviewed_by = reviewer_id
        record.reviewed_at = now_utc()
        record.review_comment = comment.strip()
        record.updated_at = now_utc()
        return self._image_task_result_from_record(record)

def update_image_task_annotation(
    self,
    task_id: str,
    image_id: str,
    task_type: ImageTaskType,
    annotation_json: dict,
) -> ImageTaskResult | None:
    with self.session_factory.begin() as session:
        record = session.scalar(select(ImageTaskResultRecord).where(ImageTaskResultRecord.task_id == task_id))
        if record is None:
            return None
        record.image_id = image_id
        record.task_type = task_type
        record.annotation_json = annotation_json
        record.review_status = "draft"
        record.reviewed_by = None
        record.reviewed_at = None
        record.review_comment = ""
        record.updated_at = now_utc()
        return self._image_task_result_from_record(record)

def list_image_task_result_versions(
    self,
    image_id: str,
    task_type: ImageTaskType,
    session_id: str = "default",
) -> List[ImageTaskResultVersion]:
    with self.session_factory() as session:
        task = session.scalar(
            select(ImageTaskRecord).where(
                ImageTaskRecord.image_id == image_id,
                ImageTaskRecord.task_type == task_type,
                ImageTaskRecord.session_id == session_id,
            )
        )
        if task is None:
            return []
        records = session.scalars(
            select(ImageTaskResultVersionRecord)
            .where(ImageTaskResultVersionRecord.task_id == task.id)
            .order_by(ImageTaskResultVersionRecord.version_no.desc())
        ).all()
        return [self._image_task_result_version_from_record(record) for record in records]

def get_image_task_result_version(self, version_id: str) -> ImageTaskResultVersion | None:
    with self.session_factory() as session:
        record = session.get(ImageTaskResultVersionRecord, version_id)
        return self._image_task_result_version_from_record(record) if record else None

def restore_image_task_result_version(self, version_id: str) -> tuple[ImageTaskResult, ImageTaskResultVersion]
| None:
    version = self.get_image_task_result_version(version_id)
    if version is None:
        return None
    result = ImageTaskResult(
        task_id=version.task_id,
        image_id=version.image_id,
        task_type=version.task_type,
        result_image_url=version.result_image_url,

```

```

        result_image_path=version.result_image_path,
        result_json=version.result_json,
        annotation_json=version.annotation_json,
        description_text=version.description_text,
        source="edited",
        model_id=version.model_id,
    )
    return self.save_image_task_result(result)

def list_label_configs(self) -> list[LabelConfig]:
    with self.session_factory() as session:
        records = session.scalars(select(LabelConfigRecord).order_by(LabelConfigRecord.label_id.asc())).all()
        return [self._label_config_from_record(record) for record in records]

def create_label_config(
    self,
    english_name: str,
    chinese_name: str,
    description: str = "",
    copy_from_label_id: int | None = None,
) -> LabelConfig:
    with self.session_factory.begin() as session:
        if copy_from_label_id is not None:
            template = session.get(LabelConfigRecord, copy_from_label_id)
            if template:
                english_name = english_name or f"{template.english_name}_copy"
                chinese_name = chinese_name or f"{template.chinese_name}副本"
                description = description or template.description
        next_id = session.scalar(select(func.coalesce(func.max(LabelConfigRecord.label_id), -1) + 1))
        now = now_utc()
        record = LabelConfigRecord(
            label_id=int(next_id or 0),
            english_name=english_name.strip(),
            chinese_name=chinese_name.strip(),
            description=description.strip(),
            created_at=now,
            updated_at=now,
        )
        session.add(record)
        session.flush()
        return self._label_config_from_record(record)

def copy_label_config(self, label_id: int) -> LabelConfig | None:
    with self.session_factory.begin() as session:
        template = session.get(LabelConfigRecord, label_id)
        if template is None:
            return None
        next_id = session.scalar(select(func.coalesce(func.max(LabelConfigRecord.label_id), -1) + 1))
        now = now_utc()
        record = LabelConfigRecord(
            label_id=int(next_id or 0),
            english_name=template.english_name,
            chinese_name=template.chinese_name,
            description=template.description,
            created_at=now,
            updated_at=now,
        )
        session.add(record)
        session.flush()
        return self._label_config_from_record(record)

def update_label_config(
    self,
    label_id: int,
    english_name: str,
    chinese_name: str,
    description: str = "",
) -> LabelConfig | None:
    with self.session_factory.begin() as session:
        record = session.get(LabelConfigRecord, label_id)
        if record is None:

```

```

        return None
    record.english_name = english_name.strip()
    record.chinese_name = chinese_name.strip()
    record.description = description.strip()
    record.updated_at = now_utc()
    session.flush()
    return self._label_config_from_record(record)

def delete_label_config(self, label_id: int) -> bool:
    with self.session_factory.begin() as session:
        record = session.get(LabelConfigRecord, label_id)
        if record is None:
            return False
        session.delete(record)
    return True

def create_or_update_video_caption_batch(self, batch: VideoCaptionBatch) -> VideoCaptionBatch:
    with self.session_factory.begin() as session:
        record = session.scalar(select(VideoCaptionBatchRecord).where(VideoCaptionBatchRecord.name ==
batch.name))
        if record is None:
            record = self._video_caption_batch_record(batch)
            session.add(record)
        else:
            record.source_dir = batch.source_dir
            record.output_dir = batch.output_dir
            record.model_id = batch.model_id
            record.prompt = batch.prompt
            record.frame_interval_seconds = batch.frame_interval_seconds
            record.status = batch.status
            record.total_videos = batch.total_videos
            record.total_frames = batch.total_frames
            record.metadata_json = batch.metadata
            record.updated_at = now_utc()
            batch.batch_id = record.id
        session.flush()
    return self._video_caption_batch_from_record(record)

def update_video_caption_batch_counts(self, batch_id: str, status: str | None = None) -> VideoCaptionBatch |
None:
    with self.session_factory.begin() as session:
        record = session.get(VideoCaptionBatchRecord, batch_id)
        if record is None:
            return None
        record.total_videos = int(
session.scalar(select(func.count()).select_from(VideoCaptionVideoRecord).where(VideoCaptionVideoRecord.batch_id ==
batch_id))
            or 0
        )
        record.total_frames = int(
session.scalar(select(func.count()).select_from(VideoCaptionFrameRecord).where(VideoCaptionFrameRecord.batch_id ==
batch_id))
            or 0
        )
        if status:
            record.status = status
            record.updated_at = now_utc()
            session.flush()
    return self._video_caption_batch_from_record(record)

def list_video_caption_batches(self) -> List[VideoCaptionBatch]:
    with self.session_factory() as session:
        records =
session.scalars(select(VideoCaptionBatchRecord).order_by(VideoCaptionBatchRecord.updated_at.desc())).all()
    return [self._video_caption_batch_from_record(record) for record in records]

def get_video_caption_batch(self, batch_id: str) -> VideoCaptionBatch | None:
    with self.session_factory() as session:
        record = session.get(VideoCaptionBatchRecord, batch_id)

```

```

        return self._video_caption_batch_from_record(record) if record else None

def create_or_update_video_caption_video(self, video: VideoCaptionVideo) -> VideoCaptionVideo:
    with self.session_factory.begin() as session:
        record = session.scalar(
            select(VideoCaptionVideoRecord).where(
                VideoCaptionVideoRecord.batch_id == video.batch_id,
                VideoCaptionVideoRecord.source_path == video.source_path,
            )
        )
        if record is None:
            record = self._video_caption_video_record(video)
            session.add(record)
        else:
            record.filename = video.filename
            if video.display_name:
                record.display_name = video.display_name
            if video.keywords:
                record.keywords_json = video.keywords
            record.frame_dir = video.frame_dir
            record.fps = video.fps
            record.width = video.width
            record.height = video.height
            record.frame_count = video.frame_count
            record.duration_ms = video.duration_ms
            record.status = video.status
            record.error = video.error
            record.updated_at = now_utc()
            video.video_id = record.id
        session.flush()
        return self._video_caption_video_from_record(record)

def update_video_caption_video_summary(
    self,
    video_id: str,
    display_name: str,
    keywords: list[str],
) -> VideoCaptionVideo | None:
    with self.session_factory.begin() as session:
        record = session.get(VideoCaptionVideoRecord, video_id)
        if record is None:
            return None
        record.display_name = display_name.strip()
        record.keywords_json = keywords
        record.updated_at = now_utc()
        session.flush()
        return self._video_caption_video_from_record(record)

def list_video_caption_videos(self, batch_id: str) -> list[VideoCaptionVideo]:
    with self.session_factory() as session:
        records = session.scalars(
            select(VideoCaptionVideoRecord)
            .where(VideoCaptionVideoRecord.batch_id == batch_id)
            .order_by(VideoCaptionVideoRecord.filename.asc())
        ).all()
        return [self._video_caption_video_from_record(record) for record in records]

def get_video_caption_video(self, video_id: str) -> VideoCaptionVideo | None:
    with self.session_factory() as session:
        record = session.get(VideoCaptionVideoRecord, video_id)
        return self._video_caption_video_from_record(record) if record else None

def save_video_caption_frame(self, frame: VideoCaptionFrame) -> VideoCaptionFrame:
    with self.session_factory.begin() as session:
        record = session.scalar(
            select(VideoCaptionFrameRecord).where(
                VideoCaptionFrameRecord.video_id == frame.video_id,
                VideoCaptionFrameRecord.frame_index == frame.frame_index,
            )
        )
        if record is None:

```

```

        record = self._video_caption_frame_record(frame)
        session.add(record)
    else:
        record.batch_id = frame.batch_id
        record.timestamp_ms = frame.timestamp_ms
        record.image_path = frame.image_path
        record.image_url = frame.image_url
        record.width = frame.width
        record.height = frame.height
        record.description_text = frame.description_text
        record.model_id = frame.model_id
        record.status = frame.status
        record.error = frame.error
        record.result_json = frame.result_json
        record.updated_at = now_utc()
        frame.frame_id = record.id
    session.flush()
    return self._video_caption_frame_from_record(record)

def get_video_caption_frame(self, frame_id: str) -> VideoCaptionFrame | None:
    with self.session_factory() as session:
        record = session.get(VideoCaptionFrameRecord, frame_id)
        return self._video_caption_frame_from_record(record) if record else None

def get_video_caption_frame_by_index(self, video_id: str, frame_index: int) -> VideoCaptionFrame | None:
    with self.session_factory() as session:
        record = session.scalar(
            select(VideoCaptionFrameRecord).where(
                VideoCaptionFrameRecord.video_id == video_id,
                VideoCaptionFrameRecord.frame_index == frame_index,
            )
        )
        return self._video_caption_frame_from_record(record) if record else None

def list_video_caption_frames(
    self,
    video_id: str,
    page: int = 1,
    page_size: int = 100,
    query: str = "",
) -> tuple[list[VideoCaptionFrame], int]:
    offset = max(page - 1, 0) * page_size
    with self.session_factory() as session:
        statement = select(VideoCaptionFrameRecord).where(VideoCaptionFrameRecord.video_id == video_id)
        count_statement =
select(func.count()).select_from(VideoCaptionFrameRecord).where(VideoCaptionFrameRecord.video_id == video_id)
        if query:
            pattern = f"%{query}%"
            condition = or_(
                VideoCaptionFrameRecord.description_text.ilike(pattern),
                VideoCaptionFrameRecord.error.ilike(pattern),
            )
            statement = statement.where(condition)
            count_statement = count_statement.where(condition)
        records = session.scalars(
            statement.order_by(VideoCaptionFrameRecord.frame_index.asc()).offset(offset).limit(page_size)
        ).all()
        total = session.scalar(count_statement)
        return [self._video_caption_frame_from_record(record) for record in records], int(total or 0)

def close(self) -> None:
    self.engine.dispose()

def healthcheck(self) -> bool:
    with self.engine.connect() as connection:
        return connection.execute(text("SELECT 1")).scalar_one() == 1

def _ensure_label_config_schema(self, database_url: str) -> None:
    if not database_url.startswith("postgresql"):
        return
    with self.engine.begin() as connection:

```

```

        connection.execute(text("ALTER TABLE label_config DROP CONSTRAINT IF EXISTS
label_config_english_name_key"))
        connection.execute(text("DROP INDEX IF EXISTS ix_label_config_english_name"))

    def _ensure_video_caption_schema(self, database_url: str) -> None:
        if not database_url.startswith("postgresql"):
            return
        with self.engine.begin() as connection:
            connection.execute(text("ALTER TABLE video_caption_video ADD COLUMN IF NOT EXISTS display_name
VARCHAR(500) DEFAULT ''"))
            connection.execute(text("ALTER TABLE video_caption_video ADD COLUMN IF NOT EXISTS keywords_json JSON
DEFAULT '[]'::json"))

    def _ensure_image_task_review_schema(self, database_url: str) -> None:
        if not database_url.startswith("postgresql"):
            return
        with self.engine.begin() as connection:
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS review_status
VARCHAR(32) DEFAULT 'draft'"))
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS submitted_by
VARCHAR(64)"))
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS submitted_at
TIMESTAMPZ"))
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS reviewed_by
VARCHAR(64)"))
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS reviewed_at
TIMESTAMPZ"))
            connection.execute(text("ALTER TABLE image_task_result ADD COLUMN IF NOT EXISTS review_comment TEXT
DEFAULT ''"))

    def _ensure_admin_user(self) -> None:
        with self.session_factory.begin() as session:
            has_admin = session.scalar(select(func.count()).select_from(UserRecord).where(UserRecord.role ==
"admin")) or 0
            if has_admin:
                return
            first_user = session.scalar(select(UserRecord).order_by(UserRecord.created_at.asc()).limit(1))
            if first_user is not None:
                first_user.role = "admin"
                first_user.updated_at = now_utc()

    def _task_record(self, task: DetectTask) -> TaskRecord:
        return TaskRecord(
            id=task.id,
            asset_id=task.asset_id,
            status=task.status,
            created_at=task.created_at,
            payload=task.model_dump(mode="json"),
        )

    def _auth_user_from_record(self, record: UserRecord) -> AuthUser:
        return AuthUser(
            userId=record.id,
            username=record.username,
            displayName=record.display_name,
            role=record.role,
            isActive=record.is_active,
            createdAt=record.created_at,
            updatedAt=record.updated_at,
        )

    def _image_asset_record(self, image: ImageAsset) -> ImageAssetRecord:
        return ImageAssetRecord(
            id=image.image_id,
            original_name=image.original_name,
            file_name=image.file_name,
            file_url=image.file_url,
            file_path=image.file_path,
            mime_type=image.mime_type,
            file_size=image.file_size,
            width=image.width,

```

```

        height=image.height,
        user_id=image.user_id,
        session_id=image.session_id,
        created_at=image.created_at,
        updated_at=image.updated_at,
    )

def _image_asset_from_record(self, record: ImageAssetRecord) -> ImageAsset:
    return ImageAsset(
        image_id=record.id,
        original_name=record.original_name,
        file_name=record.file_name,
        file_url=record.file_url,
        file_path=record.file_path,
        mime_type=record.mime_type,
        file_size=record.file_size,
        width=record.width,
        height=record.height,
        user_id=record.user_id,
        session_id=record.session_id,
        created_at=record.created_at,
        updated_at=record.updated_at,
    )

def _image_task_record(self, task: ImageTask) -> ImageTaskRecord:
    return ImageTaskRecord(
        id=task.task_id,
        image_id=task.image_id,
        task_type=task.task_type,
        status=task.status,
        user_id=task.user_id,
        user_key=task.user_id or "",
        session_id=task.session_id,
        created_at=task.created_at,
        updated_at=task.updated_at,
    )

def _image_task_from_record(self, record: ImageTaskRecord) -> ImageTask:
    return ImageTask(
        task_id=record.id,
        image_id=record.image_id,
        task_type=record.task_type,
        status=record.status,
        user_id=record.user_id,
        session_id=record.session_id,
        created_at=record.created_at,
        updated_at=record.updated_at,
    )

def _image_task_result_record(self, result: ImageTaskResult) -> ImageTaskResultRecord:
    return ImageTaskResultRecord(
        id=result.result_id,
        task_id=result.task_id,
        image_id=result.image_id,
        task_type=result.task_type,
        status=result.status,
        result_image_url=result.result_image_url,
        result_image_path=result.result_image_path,
        result_json=result.result_json,
        annotation_json=result.annotation_json,
        description_text=result.description_text,
        created_at=result.created_at,
        updated_at=result.updated_at,
        review_status=result.review_status,
        submitted_by=result.submitted_by,
        submitted_at=result.submitted_at,
        reviewed_by=result.reviewed_by,
        reviewed_at=result.reviewed_at,
        review_comment=result.review_comment,
    )

```

```

def _image_task_result_from_record(self, record: ImageTaskResultRecord) -> ImageTaskResult:
    latest = self._latest_version_for_task(record.task_id)
    return ImageTaskResult(
        result_id=record.id,
        task_id=record.task_id,
        image_id=record.image_id,
        task_type=record.task_type,
        status=record.status,
        result_image_url=record.result_image_url,
        result_image_path=record.result_image_path,
        result_json=record.result_json,
        annotation_json=record.annotation_json,
        description_text=record.description_text,
        created_at=record.created_at,
        updated_at=record.updated_at,
        latest_version_id=latest.version_id if latest else None,
        latest_version_no=latest.version_no if latest else None,
        review_status=record.review_status or "draft",
        submitted_by=record.submitted_by,
        submitted_at=record.submitted_at,
        reviewed_by=record.reviewed_by,
        reviewed_at=record.reviewed_at,
        review_comment=record.review_comment or "",
    )

def _image_task_result_version_record(self, version: ImageTaskResultVersion) -> ImageTaskResultVersionRecord:
    return ImageTaskResultVersionRecord(
        id=version.version_id,
        result_id=version.result_id,
        task_id=version.task_id,
        image_id=version.image_id,
        task_type=version.task_type,
        version_no=version.version_no,
        source=version.source,
        model_id=version.model_id,
        result_image_url=version.result_image_url,
        result_image_path=version.result_image_path,
        result_json=version.result_json,
        annotation_json=version.annotation_json,
        description_text=version.description_text,
        created_at=version.created_at,
    )

def _image_task_result_version_from_record(self, record: ImageTaskResultVersionRecord) ->
ImageTaskResultVersion:
    return ImageTaskResultVersion(
        version_id=record.id,
        result_id=record.result_id,
        task_id=record.task_id,
        image_id=record.image_id,
    )

```

backend/app/services/__init__.py

```

"""Backend service layer."""

```

backend/app/services/export_service.py

```

from app.models.schemas import AnnotationsDocument
from app.repositories.postgres_store import PostgresStore

class ExportService:
    def __init__(self, store: PostgresStore) -> None:
        self.store = store

    def get_annotations(self, asset_id: str) -> AnnotationsDocument | None:
        return self.store.get_annotations(asset_id)

```

```

def to_coco(self, annotations: AnnotationsDocument) -> dict:
    categories: dict[str, int] = {}
    images = []
    coco_annotations = []
    annotation_id = 1

    for frame in annotations.frames:
        image_id = frame.frame_index + 1
        images.append(
            {
                "id": image_id,
                "file_name": f"{annotations.asset_id}_{frame.frame_index:06d}.jpg",
                "width": frame.width,
                "height": frame.height,
            }
        )
        for obj in frame.objects:
            if obj.label not in categories:
                categories[obj.label] = len(categories) + 1
            category_id = categories[obj.label]
            coco_annotations.append(
                {
                    "id": annotation_id,
                    "image_id": image_id,
                    "category_id": category_id,
                    "bbox": [obj.bbox.x, obj.bbox.y, obj.bbox.width, obj.bbox.height],
                    "area": obj.bbox.width * obj.bbox.height,
                    "iscrowd": 0,
                    "score": obj.confidence,
                }
            )
            annotation_id += 1

    return {
        "images": images,
        "annotations": coco_annotations,
        "categories": [{"id": idx, "name": label} for label, idx in categories.items()],
    }

def to_yolo_text(self, annotations: AnnotationsDocument) -> str:
    labels = sorted({obj.label for frame in annotations.frames for obj in frame.objects})
    label_to_id = {label: idx for idx, label in enumerate(labels)}
    lines = ["# frame_index label_id cx cy w h label confidence"]
    for frame in annotations.frames:
        width = frame.width or 1
        height = frame.height or 1
        for obj in frame.objects:
            cx = (obj.bbox.x + obj.bbox.width / 2) / width
            cy = (obj.bbox.y + obj.bbox.height / 2) / height
            bw = obj.bbox.width / width
            bh = obj.bbox.height / height
            lines.append(
                f"{frame.frame_index} {label_to_id[obj.label]} "
                f"{cx:.6f} {cy:.6f} {bw:.6f} {bh:.6f} {obj.label} {obj.confidence:.4f}"
            )
    return "\n".join(lines) + "\n"

```

backend/app/services/inference_service.py

```

from pathlib import Path
from typing import Any

from PIL import Image

from app.core.config import Settings
from app.models.schemas import (
    AnnotationFrame,
    AnnotationsDocument,

```

```

Asset,
BoundingBox,
DetectionObject,
DetectionRequest,
)
from app.services.storage_service import FrameSource, StorageService

MODEL_ALIASES = {
    "railway-yolo-default": "yolo11n.pt",
    "custom-railway-detector": "yolo11n.pt",
    "yolov8n": "yolov8n.pt",
    "yolo11n": "yolo11n.pt",
    "yolo11n-seg": "yolo11n-seg.pt",
    "railway-seg-light": "yolo11n-seg.pt",
    "custom-segmentation": "yolo11n-seg.pt",
    "yolo11n-pose": "yolo11n-pose.pt",
    "railway-worker-pose": "yolo11n-pose.pt",
    "custom-pose": "yolo11n-pose.pt",
    "yolo11n-cls": "yolo11n-cls.pt",
    "railway-scene-cls": "yolo11n-cls.pt",
}

class InferenceService:
    def __init__(self, settings: Settings, storage: StorageService) -> None:
        self.settings = settings
        self.storage = storage
        self._models: dict[str, Any] = {}

    @property
    def model_name(self) -> str:
        if self.settings.inference_backend in {"ultralalytics", "tensorrt"}:
            return Path(self.settings.model_path).name if self.settings.model_path else "yolo11n.pt"
        return "mock"

    def predict(self, asset: Asset, request: DetectionRequest) -> AnnotationsDocument:
        frames = self.storage.extract_frames(
            asset,
            frame_stride=request.frame_stride,
            max_frames=request.max_frames,
        )
        if self.settings.inference_backend in {"ultralalytics", "tensorrt"}:
            return self._predict_ultralalytics(asset, request, frames)
        return self._predict_mock(asset, frames)

    def _predict_mock(self, asset: Asset, frames: list[FrameSource]) -> AnnotationsDocument:
        annotation_frames: list[AnnotationFrame] = []
        for frame_source in frames:
            width = frame_source.width
            height = frame_source.height
            bbox = BoundingBox(
                x=round(width * 0.35, 2),
                y=round(height * 0.28, 2),
                width=round(width * 0.3, 2),
                height=round(height * 0.42, 2),
            )
            annotation_frames.append(
                AnnotationFrame(
                    frame_index=frame_source.frame_index,
                    timestamp_ms=frame_source.timestamp_ms,
                    width=width,
                    height=height,
                    image_url=frame_source.image_url,
                    objects=[
                        DetectionObject(
                            label="railway_target",
                            confidence=0.88,
                            bbox=bbox,
                        )
                    ],
                )
            )
        )

```

```

    )
    return AnnotationsDocument(asset_id=asset.id, type=asset.type, model="mock", frames=annotation_frames)

def _predict_ultralytics(
    self,
    asset: Asset,
    request: DetectionRequest,
    frames: list[FrameSource],
) -> AnnotationsDocument:
    model_path = self._resolve_model_path(request.model_name)
    model = self._load_ultralytics_model(model_path)
    annotation_frames: list[AnnotationFrame] = []

    for frame_source in frames:
        results = model(
            str(frame_source.path),
            conf=request.confidence,
            iou=request.iou,
            device=self._device(),
            verbose=False,
        )
        result = results[0]
        height, width = self._result_shape(result, asset, frame_source)
        names = getattr(result, "names", {}) or {}
        objects: list[DetectionObject] = []

        for box in result.bboxes or []:
            xyxy = box.xyxy[0].tolist()
            class_id = int(box.cls[0].item()) if box.cls is not None else -1
            confidence = float(box.conf[0].item()) if box.conf is not None else 0.0
            track_id = int(box.id[0].item()) if getattr(box, "id", None) is not None else None
            objects.append(
                DetectionObject(
                    label=str(names.get(class_id, class_id)),
                    confidence=confidence,
                    bbox=BBox(
                        x=max(float(xyxy[0]), 0.0),
                        y=max(float(xyxy[1]), 0.0),
                        width=max(float(xyxy[2] - xyxy[0]), 1.0),
                        height=max(float(xyxy[3] - xyxy[1]), 1.0),
                    ),
                    track_id=track_id,
                )
            )

        annotation_frames.append(
            AnnotationFrame(
                frame_index=frame_source.frame_index,
                timestamp_ms=frame_source.timestamp_ms,
                width=width,
                height=height,
                image_url=frame_source.image_url,
                objects=objects,
            )
        )

    return AnnotationsDocument(
        asset_id=asset.id,
        type=asset.type,
        model=request.model_name or Path(model_path).name,
        frames=annotation_frames,
    )

def _load_ultralytics_model(self, model_path: str) -> Any:
    if model_path in self._models:
        return self._models[model_path]
    try:
        from ultralytics import YOLO
    except Exception as exc:
        raise RuntimeError("ultralytics not installed in this Python environment") from exc

```

```

        if self.settings.inference_backend == "tensorrt":
            if Path(model_path).suffix != ".engine":
                raise RuntimeError("TensorRT inference requires a .engine MODEL_PATH")
        self._models[model_path] = YOLO(model_path)
        return self._models[model_path]

def _resolve_model_path(self, model_name: str | None) -> str:
    candidate = MODEL_ALIASES.get(model_name or "", model_name or self.settings.model_path or "yolo11n.pt")
    path = Path(candidate)
    if path.is_absolute():
        return str(path)

    local_path = Path(self.settings.data_dir).parent / candidate
    if local_path.exists():
        return str(local_path)

    backend_path = Path(__file__).resolve().parents[2] / candidate
    if backend_path.exists():
        return str(backend_path)

    return candidate

def _result_shape(self, result: Any, asset: Asset, frame_source: FrameSource) -> tuple[int, int]:
    if getattr(result, "orig_shape", None):
        height, width = result.orig_shape[:2]
        return int(height), int(width)
    if frame_source.height and frame_source.width:
        return frame_source.height, frame_source.width
    with Image.open(frame_source.path) as image:
        width, height = image.size
    return height, width

def _device(self) -> str | None:
    if not self.settings.device:
        return None
    return self.settings.device

```

backend/app/services/storage_service.py

```

import shutil
from dataclasses import dataclass
from pathlib import Path

from fastapi import UploadFile
from PIL import Image

from app.models.schemas import Asset

IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
VIDEO_EXTENSIONS = {".mp4", ".mov", ".avi", ".mkv", ".webm"}

@dataclass(frozen=True)
class FrameSource:
    frame_index: int
    timestamp_ms: int
    path: Path
    width: int
    height: int
    image_url: str

class StorageService:
    def __init__(self, upload_dir: Path) -> None:
        self.upload_dir = upload_dir
        self.upload_dir.mkdir(parents=True, exist_ok=True)

    def save_upload(self, upload: UploadFile) -> Asset:

```

```

filename = Path(upload.filename or "upload.bin").name
extension = Path(filename).suffix.lower()
asset_type = self._guess_asset_type(upload.content_type or "", extension)

asset = Asset(
    filename=filename,
    content_type=upload.content_type or "application/octet-stream",
    type=asset_type,
    path="",
    size_bytes=0,
)
asset_dir = self.upload_dir / asset.id
asset_dir.mkdir(parents=True, exist_ok=True)
target_path = asset_dir / filename

with target_path.open("wb") as handle:
    shutil.copyfileobj(upload.file, handle)

asset.path = str(target_path)
asset.size_bytes = target_path.stat().st_size
if asset.type == "image":
    self._populate_image_metadata(asset, target_path)
elif asset.type == "video":
    self._populate_video_metadata(asset, target_path)
return asset

def extract_frames(
    self,
    asset: Asset,
    frame_stride: int = 1,
    max_frames: int | None = None,
) -> List[FrameSource]:
    if asset.type == "image":
        return [self._extract_image_frame(asset)]
    if asset.type == "video":
        return self._extract_video_frames(asset, frame_stride=frame_stride, max_frames=max_frames)
    raise ValueError(f"unsupported asset type: {asset.type}")

def frame_path(self, asset_id: str, frame_index: int) -> Path:
    return self.upload_dir / asset_id / "frames" / f"frame_{frame_index:06d}.jpg"

def _guess_asset_type(self, content_type: str, extension: str) -> str:
    if content_type.startswith("image/") or extension in IMAGE_EXTENSIONS:
        return "image"
    if content_type.startswith("video/") or extension in VIDEO_EXTENSIONS:
        return "video"
    return "unknown"

def _populate_image_metadata(self, asset: Asset, path: Path) -> None:
    with Image.open(path) as image:
        asset.width, asset.height = image.size

def _populate_video_metadata(self, asset: Asset, path: Path) -> None:
    try:
        import cv2
    except Exception:
        return

    capture = cv2.VideoCapture(str(path))
    try:
        if not capture.isOpened():
            return
        fps = capture.get(cv2.CAP_PROP_FPS) or 0
        frame_count = capture.get(cv2.CAP_PROP_FRAME_COUNT) or 0
        asset.width = int(capture.get(cv2.CAP_PROP_FRAME_WIDTH) or 0) or None
        asset.height = int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0) or None
        asset.frame_count = int(frame_count) if frame_count else None
        asset.fps = float(fps) if fps else None
        asset.duration_ms = int(frame_count / fps * 1000) if fps and frame_count else None
    finally:
        capture.release()

```

```

def _frame_dir(self, asset: Asset) -> Path:
    frame_dir = Path(asset.path).parent / "frames"
    frame_dir.mkdir(parents=True, exist_ok=True)
    return frame_dir

def _extract_image_frame(self, asset: Asset) -> FrameSource:
    source_path = Path(asset.path)
    frame_path = self._frame_dir(asset) / "frame_000000.jpg"
    with Image.open(source_path) as image:
        rgb_image = image.convert("RGB")
        rgb_image.save(frame_path, format="JPEG", quality=92)
        width, height = rgb_image.size
    return FrameSource(
        frame_index=0,
        timestamp_ms=0,
        path=frame_path,
        width=width,
        height=height,
        image_url=f"/api/assets/{asset.id}/frames/0/image",
    )

def _extract_video_frames(
    self,
    asset: Asset,
    frame_stride: int,
    max_frames: int | None,
) -> List[FrameSource]:
    try:
        import cv2
    except Exception as exc:
        raise RuntimeError("opencv-python-headless is required for video frame extraction") from exc

    capture = cv2.VideoCapture(asset.path)
    if not capture.isOpened():
        raise RuntimeError(f"could not open video: {asset.filename}")

    frames: List[FrameSource] = []
    frame_dir = self._frame_dir(asset)
    fps = capture.get(cv2.CAP_PROP_FPS) or asset.fps or 0
    raw_index = 0
    try:
        while True:
            ok, frame = capture.read()
            if not ok:
                break
            if raw_index % frame_stride != 0:
                raw_index += 1
                continue
            frame_path = frame_dir / f"frame_{raw_index:06d}.jpg"
            cv2.imwrite(str(frame_path), frame)
            height, width = frame.shape[:2]
            timestamp_ms = int(raw_index / fps * 1000) if fps else 0
            frames.append(
                FrameSource(
                    frame_index=raw_index,
                    timestamp_ms=timestamp_ms,
                    path=frame_path,
                    width=int(width),
                    height=int(height),
                    image_url=f"/api/assets/{asset.id}/frames/{raw_index}/image",
                )
            )
            if max_frames is not None and len(frames) >= max_frames:
                break
            raw_index += 1
    finally:
        capture.release()

    if not frames:

```

```
        raise RuntimeError(f"no frames extracted from video: {asset.filename}")
    return frames
```

backend/app/services/task_service.py

```
from app.models.schemas import DetectTask, DetectionRequest, now_utc
from app.repositories.postgres_store import PostgresStore
from app.services.inference_service import InferenceService

class TaskService:
    def __init__(self, store: PostgresStore, inference: InferenceService) -> None:
        self.store = store
        self.inference = inference

    def create_detection_task(self, request: DetectionRequest) -> DetectTask:
        task = DetectTask(
            asset_id=request.asset_id,
            params=request,
            model_name=request.model_name or self.inference.model_name,
        )
        return self.store.create_task(task)

    def run_detection(self, task_id: str, claimed_task: DetectTask | None = None) -> None:
        task = claimed_task or self.store.get_task(task_id)
        if task is None:
            return

        asset = self.store.get_asset(task.asset_id)
        if asset is None:
            task.status = "failed"
            task.error = f"asset not found: {task.asset_id}"
            task.finished_at = now_utc()
            self.store.update_task(task)
            return

        try:
            if claimed_task is None:
                task.status = "running"
                task.progress = 0.1
                task.started_at = now_utc()
                self.store.update_task(task)

            annotations = self.inference.predict(asset, task.params)
            self.store.save_annotations(annotations)

            task.status = "completed"
            task.progress = 1
            task.finished_at = now_utc()
            self.store.update_task(task)
        except Exception as exc:
            task.status = "failed"
            task.error = str(exc)
            task.finished_at = now_utc()
            self.store.update_task(task)
```

backend/app/services/task_worker.py

```
import logging
from threading import Event, Thread

from app.repositories.postgres_store import PostgresStore
from app.services.task_service import TaskService

logger = logging.getLogger(__name__)
```

```

class DatabaseTaskWorker:
    def __init__(
        self,
        store: PostgresStore,
        task_service: TaskService,
        poll_interval_seconds: float,
    ) -> None:
        self.store = store
        self.task_service = task_service
        self.poll_interval_seconds = poll_interval_seconds
        self._stop_event = Event()
        self._thread = Thread(target=self._run, name="postgres-task-worker", daemon=True)

    def start(self) -> None:
        if not self._thread.is_alive():
            self._thread.start()

    def stop(self) -> None:
        self._stop_event.set()
        if self._thread.is_alive():
            self._thread.join(timeout=max(self.poll_interval_seconds * 2, 2))

    def _run(self) -> None:
        logger.info("PostgreSQL task worker started")
        while not self._stop_event.is_set():
            try:
                task = self.store.claim_next_task()
                if task is None:
                    self._stop_event.wait(self.poll_interval_seconds)
                    continue
                self.task_service.run_detection(task.id, claimed_task=task)
            except Exception:
                logger.exception("PostgreSQL task worker iteration failed")
                self._stop_event.wait(self.poll_interval_seconds)
        logger.info("PostgreSQL task worker stopped")

```

backend/app/services/vision_service.py

```

import tempfile
from pathlib import Path
from typing import Any, Literal

from fastapi import UploadFile
from PIL import Image

from app.core.config import Settings

VisionFeature = Literal["detection", "pose", "segmentation", "classification", "caption"]

DEFAULT_MODELS: dict[VisionFeature, str] = {
    "detection": "yolo11n.pt",
    "pose": "yolo11n-pose.pt",
    "segmentation": "yolo11n-seg.pt",
    "classification": "yolo11n-cls.pt",
    "caption": "Salesforce/blip-image-captioning-base",
}

MODEL_ALIASES: dict[str, str] = {
    "railway-yolo-default": "yolo11n.pt",
    "custom-railway-detector": "yolo11n.pt",
    "yolov8n": "yolov8n.pt",
    "yolo11n": "yolo11n.pt",
    "yolo11n-seg": "yolo11n-seg.pt",
    "railway-seg-light": "yolo11n-seg.pt",
    "custom-segmentation": "yolo11n-seg.pt",
    "yolo11n-pose": "yolo11n-pose.pt",
    "railway-worker-pose": "yolo11n-pose.pt",
    "custom-pose": "yolo11n-pose.pt",
    "yolo11n-cls": "yolo11n-cls.pt",
}

```

```

"railway-scene-cls": "yolo11n-cls.pt",
"resnet50": "yolo11n-cls.pt",
"efficientnet-b0": "yolo11n-cls.pt",
"deepseek-vl2-tiny": "deepseek-ai/deepseek-vl2-tiny",
"deepseek-ai/deepseek-vl2-tiny": "deepseek-ai/deepseek-vl2-tiny",
"blip-image-captioning-base": "Salesforce/blip-image-captioning-base",
"yolo-cls-fallback": "yolo11n-cls.pt",
"custom-caption": "Salesforce/blip-image-captioning-base",
}

```

```

class VisionService:

```

```

    def __init__(self, settings: Settings) -> None:
        self.settings = settings
        self._yolo_models: dict[str, Any] = {}
        self._caption_model: Any | None = None
        self._caption_processor: Any | None = None
        self._deepseek_vl2_model: Any | None = None
        self._deepseek_vl2_processor: Any | None = None
        self._deepseek_vl2_tokenizer: Any | None = None

```

```

    def analyze_upload(
        self,
        file: UploadFile,
        feature: VisionFeature,
        model_name: str | None = None,
    ) -> dict[str, Any]:
        suffix = Path(file.filename or "image.jpg").suffix or ".jpg"
        with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as target:
            target.write(file.file.read())
            image_path = Path(target.name)
        try:
            return self.analyze_path(image_path, feature, model_name)
        finally:
            image_path.unlink(missing_ok=True)

```

```

    def analyze_path(
        self,
        image_path: Path,
        feature: VisionFeature,
        model_name: str | None = None,
    ) -> dict[str, Any]:
        if not image_path.exists():
            raise RuntimeError(f"图片文件不存在: {image_path}")
        if feature == "caption":
            return self._caption(image_path, model_name)
        return self._yolo(feature, image_path, model_name)

```

```

    def _yolo(
        self,
        feature: VisionFeature,
        image_path: Path,
        model_name: str | None = None,
    ) -> dict[str, Any]:
        resolved_model = self._resolve_model(feature, model_name)
        model = self._load_yolo(resolved_model)
        results = model(str(image_path), device=self._device(), verbose=False)
        result = results[0]
        height, width = result.orig_shape[:2]

        if feature == "classification":
            items: list[dict[str, Any]] = []
            probs = result.probs
            names = result.names
            if probs is not None:
                for index in probs.top5:
                    score = float(probs.data[index].item())
                    items.append({"label": str(names.get(int(index), index)), "confidence": score})
            return {
                "feature": feature,
                "model": model_name or resolved_model,

```

```

        "image": {"width": int(width), "height": int(height)},
        "results": items,
    }

boxes: list[dict[str, Any]] = []
names = result.names
for box in result.boxes or []:
    xyxy = box.xyxy[0].tolist()
    class_id = int(box.cls[0].item()) if box.cls is not None else -1
    boxes.append(
        {
            "label": str(names.get(class_id, class_id)),
            "confidence": float(box.conf[0].item()) if box.conf is not None else 0.0,
            "bbox": {
                "x": max(float(xyxy[0]), 0.0),
                "y": max(float(xyxy[1]), 0.0),
                "width": max(float(xyxy[2] - xyxy[0]), 1.0),
                "height": max(float(xyxy[3] - xyxy[1]), 1.0),
            },
        }
    )

payload: dict[str, Any] = {
    "feature": feature,
    "model": model_name or resolved_model,
    "image": {"width": int(width), "height": int(height)},
    "objects": boxes,
}

if feature == "pose" and result.keypoints is not None:
    payload["poses"] = [
        {
            "keypoints": keypoints.xy[0].tolist(),
            "confidence": keypoints.conf[0].tolist() if keypoints.conf is not None else [],
        }
        for keypoints in result.keypoints
    ]

if feature == "segmentation" and result.masks is not None:
    segments = []
    for index, polygon in enumerate(result.masks.xy):
        box = boxes[index] if index < len(boxes) else {}
        segments.append(
            {
                "label": box.get("label", "object"),
                "confidence": box.get("confidence", 0.0),
                "polygon": polygon.tolist(),
            }
        )
    payload["segments"] = segments

return payload

def _caption(self, image_path: Path, model_name: str | None = None) -> dict[str, Any]:
    with Image.open(image_path) as image:
        rgb_image = image.convert("RGB")
        width, height = rgb_image.size
        try:
            resolved_model = MODEL_ALIASES.get(model_name or "", model_name or DEFAULT_MODELS["caption"])
            if resolved_model == "deepseek-ai/deepseek-vl2-tiny":
                text = self._deepseek_vl2_caption(image_path)
                used_model = "deepseek-ai/deepseek-vl2-tiny"
            else:
                model, processor = self._load_caption_model()
                inputs = processor(rgb_image, return_tensors="pt").to(model.device)
                outputs = model.generate(**inputs, max_new_tokens=40)
                text = processor.decode(outputs[0], skip_special_tokens=True)
                used_model = model_name or DEFAULT_MODELS["caption"]
        except Exception:
            classified = self._yolo("classification", image_path, "yolo11n-cls").get("results", [])[:3]
            labels = [item["label"] for item in classified]

```

```

        text = (
            "This image contains " + " , ".join(labels) + "."
            if labels
            else "No obvious visual category identified."
        )
        used_model = model_name or "yolo11n-cls fallback description"
    return {
        "feature": "caption",
        "model": used_model,
        "image": {"width": width, "height": height},
        "caption": text,
    }

def _load_yolo(self, model_name: str) -> Any:
    if model_name in self._yolo_models:
        return self._yolo_models[model_name]
    try:
        from ultralytics import YOLO
    except Exception as exc:
        raise RuntimeError("当前 Python 环境未安装 ultralytics, 无法执行 YOLO 视觉功能") from exc

    resolved = model_name
    if resolved.startswith("/") and not Path(resolved).exists():
        resolved = "yolo11n.pt"
    self._yolo_models[model_name] = YOLO(resolved)
    return self._yolo_models[model_name]

def _load_caption_model(self) -> tuple[Any, Any]:
    if self._caption_model is not None and self._caption_processor is not None:
        return self._caption_model, self._caption_processor
    try:
        from transformers import BlipForConditionalGeneration, BlipProcessor
    except Exception as exc:
        raise RuntimeError("当前 Python 环境未安装 transformers, 无法执行图文描述功能") from exc

    model_name = DEFAULT_MODELS["caption"]
    self._caption_processor = BlipProcessor.from_pretrained(model_name)
    self._caption_model = BlipForConditionalGeneration.from_pretrained(model_name)
    return self._caption_model, self._caption_processor

def _deepseek_vl2_caption(self, image_path: Path) -> str:
    model, processor, tokenizer = self._load_deepseek_vl2_model()
    conversation = [
        {
            "role": "<|User|>",
            "content": f"<image>\n{self.settings.deepseek_vl2_prompt}",
            "images": [str(image_path)],
        },
        {"role": "<|Assistant|>", "content": ""},
    ]
    try:
        import torch
        from deepseek_vl2.utils.io import load_pil_images
    except Exception as exc:
        raise RuntimeError("当前 Python 环境未安装 DeepSeek-VL2 代码包, 无法执行 deepseek-vl2-tiny 图片理解")
    from exc

    self._patch_xformers_attention()
    pil_images = load_pil_images(conversation)
    prepare_inputs = processor(
        conversations=conversation,
        images=pil_images,
        force_batchify=True,
        system_prompt="",
    ).to(model.device)
    with torch.inference_mode():
        inputs_embeds = model.prepare_inputs_embeds(**prepare_inputs)
        language_model = getattr(model, "language_model", None) or getattr(model, "language")
        outputs = language_model.generate(
            inputs_embeds=inputs_embeds,
            attention_mask=prepare_inputs.attention_mask,

```

```

        pad_token_id=tokenizer.eos_token_id,
        bos_token_id=tokenizer.bos_token_id,
        eos_token_id=tokenizer.eos_token_id,
        max_new_tokens=256,
        do_sample=False,
        use_cache=True,
    )
    return tokenizer.decode(outputs[0].detach().cpu().tolist(), skip_special_tokens=True).strip()

def _patch_xformers_attention(self) -> None:
    try:
        import torch
        import xformers.ops as xformers_ops
    except Exception as exc:
        raise RuntimeError("当前 Python 环境缺少 xformers, 无法执行 deepseek-vl2-tiny 视觉注意力") from exc

    if getattr(xformers_ops.memory_efficient_attention, "_railway_torch_fallback", False):
        return

    def torch_attention(
        query: Any,
        key: Any,
        value: Any,
        attn_bias: Any = None,
        p: float = 0.0,
        scale: float | None = None,
        **_: Any,
    ) -> Any:
        if attn_bias is not None:
            raise RuntimeError("deepseek-vl2-tiny 当前仅支持无 attn_bias 的 PyTorch 注意力回退")
        query_t = query.transpose(1, 2)
        key_t = key.transpose(1, 2)
        value_t = value.transpose(1, 2)
        output = torch.nn.functional.scaled_dot_product_attention(
            query_t,
            key_t,
            value_t,
            dropout_p=p,
            scale=scale,
        )
        return output.transpose(1, 2).contiguous()

    torch_attention._railway_torch_fallback = True # type: ignore[attr-defined]
    xformers_ops.memory_efficient_attention = torch_attention

def _load_deepseek_vl2_model(self) -> tuple[Any, Any, Any]:
    if (
        self._deepseek_vl2_model is not None
        and self._deepseek_vl2_processor is not None
        and self._deepseek_vl2_tokenizer is not None
    ):
        return self._deepseek_vl2_model, self._deepseek_vl2_processor, self._deepseek_vl2_tokenizer
    try:
        import torch
        from transformers import AutoModelForCausalLM
        from deepseek_vl2.models import DeepseekVLV2Processor
    except Exception as exc:
        raise RuntimeError("当前 Python 环境缺少 transformers 或 DeepSeek-VL2 依赖, 无法加载 deepseek-vl2-tiny") from exc

    model_path = self._deepseek_vl2_model_path()
    processor = DeepseekVLV2Processor.from_pretrained(model_path)
    model = AutoModelForCausalLM.from_pretrained(model_path, trust_remote_code=True)
    if torch.cuda.is_available() and self._device() is not None:
        device = f"cuda:{self._device()}" if self._device().isdigit() else self._device()
        model = model.to(torch.bfloat16).to(device)
    else:
        model = model.to(torch.float32)
    model = model.eval()
    self._deepseek_vl2_processor = processor
    self._deepseek_vl2_tokenizer = processor.tokenizer

```

```

self._deepseek_vl2_model = model
return self._deepseek_vl2_model, self._deepseek_vl2_processor, self._deepseek_vl2_tokenizer

def _deepseek_vl2_model_path(self) -> str:
    configured = Path(self.settings.deepseek_vl2_model_path)
    if configured.exists():
        return str(configured)
    return "deepseek-ai/deepseek-vl2-tiny"

def _resolve_model(self, feature: VisionFeature, model_name: str | None) -> str:
    if model_name and model_name in MODEL_ALIASES:
        return MODEL_ALIASES[model_name]
    if model_name and (model_name.endswith(".pt") or model_name.startswith("/")):
        return model_name
    if feature == "detection" and self.settings.model_path:
        return self.settings.model_path
    return DEFAULT_MODELS[feature]

def _device(self) -> str | None:
    return self.settings.device or None

```

backend/models/deepseek-vl2-tiny/config.json

```

{
  "candidate_resolutions": [
    [
      384,
      384
    ],
    [
      384,
      768
    ],
    [
      768,
      384
    ],
    [
      384,
      1152
    ],
    [
      1152,
      384
    ],
    [
      384,
      1536
    ],
    [
      1536,
      384
    ],
    [
      768,
      768
    ],
    [
      384,
      1920
    ],
    [
      1920,
      384
    ],
    [
      384,
      2304
    ],
  ],

```

```

[
  2304,
  384
],
[
  768,
  1152
],
[
  1152,
  768
],
[
  384,
  2688
],
[
  2688,
  384
],
[
  384,
  3072
],
[
  3072,
  384
],
[
  768,
  1536
],
[
  1536,
  768
],
[
  384,
  3456
],
[
  3456,
  384
],
[
  1152,
  1152
]
],
"global_view_pos": "head",
"language_config": {
  "architectures": [
    "DeepseekV2ForCausalLM"
  ],
  "auto_map": {
    "AutoConfig": "configuration_deepseek.DeepseekV2Config",
    "AutoModel": "modeling_deepseek.DeepseekV2Model",
    "AutoModelForCausalLM": "modeling_deepseek.DeepseekV2ForCausalLM"
  },
  "bos_token_id": 0,
  "eos_token_id": 1,
  "first_k_dense_replace": 1,
  "hidden_size": 1280,
  "intermediate_size": 6848,
  "kv_lora_rank": null,
  "lm_head": true,
  "max_position_embeddings": 4096,
  "model_type": "deepseek_v2",
  "moe_intermediate_size": 896,
  "n_group": 1,
  "n_routed_experts": 64,

```

```

    "n_shared_experts": 2,
    "num_attention_heads": 10,
    "num_experts_per_tok": 6,
    "num_hidden_layers": 12,
    "num_key_value_heads": 10,
    "q_lora_rank": null,
    "qk_nope_head_dim": 0,
    "qk_rope_head_dim": 0,
    "rm_head": false,
    "topk_group": 1,
    "topk_method": "greedy",
    "torch_dtype": "bfloat16",
    "use_mla": false,
    "v_head_dim": 0,
    "vocab_size": 129280
  },
  "model_type": "deepseek_vl_v2",
  "projector_config": {
    "model_type": "mlp_projector",
    "n_embed": 1280
  },
  "tile_tag": "2D",
  "torch_dtype": "bfloat16",
  "transformers_version": "4.38.2",
  "vision_config": {
    "layers": 27,
    "mlp_ratio": 3.7362,
    "model_name": "siglip_so400m_patch14_384",
    "model_type": "vision",
    "patch_size": 14,
    "width": 1152
  }
}

```

backend/models/deepseek-vl2-tiny/processor_config.json

```

{
  "add_special_token": false,
  "candidate_resolutions": [
    [
      384,
      384
    ],
    [
      384,
      768
    ],
    [
      768,
      384
    ],
    [
      384,
      1152
    ],
    [
      1152,
      384
    ],
    [
      384,
      1536
    ],
    [
      1536,
      384
    ],
    [
      768,

```

```
768
],
[
  384,
  1920
],
[
  1920,
  384
],
[
  384,
  2304
],
[
  2304,
  384
],
[
  768,
  1152
],
[
  1152,
  768
],
[
  384,
  2688
],
[
  2688,
  384
],
[
  384,
  3072
],
[
  3072,
  384
],
[
  768,
  1536
],
[
  1536,
  768
],
[
  384,
  3456
],
[
  3456,
  384
],
[
  1152,
  1152
]
],
"downsample_ratio": 2,
"ignore_id": -100,
"image_mean": [
  0.5,
  0.5,
  0.5
],
"image_std": [
```

```

    0.5,
    0.5,
    0.5
],
"image_token": "<image>",
"mask_prompt": false,
"normalize": true,
"pad_token": "<\uff5c\u2581pad\u2581\uff5c>",
"patch_size": 14,
"processor_class": "DeepseekVLV2Processor",
"sft_format": "deepseek"
}

```

backend/pyproject.toml

```

[project]
name = "railway-recognition-backend"
version = "0.1.0"
description = "FastAPI backend for railway multi-target recognition and annotation review."
requires-python = ">=3.11"
dependencies = [
    "fastapi>=0.115.0",
    "huggingface-hub>=0.26.0",
    "pydantic-settings>=2.6.0",
    "python-multipart>=0.0.12",
    "pillow>=10.4.0",
    "opencv-python-headless>=4.10.0",
    "psycopy[binary]>=3.2.0",
    "sqlalchemy>=2.0.36",
    "socksio>=1.0.0",
    "transformers==4.38.2",
    "uvicorn[standard]>=0.32.0",
    "ultralytics>=8.4.70",
]

[dependency-groups]
dev = [
    "httpx>=0.27.0",
    "pytest>=8.3.0",
]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]

```

backend/requirements-gpu.txt

```

# Optional GPU inference stack. Install in the backend environment when you want
# to run real YOLO inference instead of the default mock backend.
ultralytics>=8.3.0
opencv-python-headless>=4.10.0
transformers==4.38.2
accelerate>=0.34.0
huggingface-hub>=0.26.0
socksio>=1.0.0
sentencepiece>=0.2.0
attrdict>=2.0.1
einops>=0.8.0
timm>=1.0.0
xformers>=0.0.35

```

backend/scripts/export_tensorrt.py

```

import argparse
from pathlib import Path

```

```

from ultralytics import YOLO

def main() -> None:
    parser = argparse.ArgumentParser(description="Export an Ultralytics YOLO model to TensorRT.")
    parser.add_argument("model", type=Path, help="Path to the source .pt model")
    parser.add_argument("--device", default="0", help="CUDA device, for example 0")
    parser.add_argument("--imgsz", type=int, default=640, help="Inference image size")
    parser.add_argument("--workspace", type=float, default=4.0, help="TensorRT workspace size in GiB")
    parser.add_argument("--batch", type=int, default=1, help="Maximum engine batch size")
    parser.add_argument("--fp32", action="store_true", help="Disable FP16 export")
    args = parser.parse_args()

    if args.model.suffix != ".pt":
        raise SystemExit("The source model must be an Ultralytics .pt file")
    if not args.model.exists():
        raise SystemExit(f"Model not found: {args.model}")

    model = YOLO(str(args.model))
    output = model.export(
        format="engine",
        device=args.device,
        imgsz=args.imgsz,
        workspace=args.workspace,
        batch=args.batch,
        half=not args.fp32,
    )
    print(output)

if __name__ == "__main__":
    main()

```

backend/scripts/process_video_captions.py

```

import argparse
import re
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from app.core.config import get_settings
from app.core.dependencies import get_store, get_vision_service
from app.models.schemas import VideoCaptionBatch, VideoCaptionFrame, VideoCaptionVideo, new_id, now_utc

DEFAULT_SOURCE_DIR = Path("<project-path>")
DEFAULT_MODEL_ID = "deepseek-ai/deepseek-vl2-tiny"
DEFAULT_PROMPT = "请用中文描述画面，列出可见的铁路目标、人员、车辆、轨道、设备、施工场景或安全风险。"
VIDEO_EXTENSIONS = {".mp4", ".mov", ".avi", ".mkv", ".m4v"}

def main() -> None:
    parser = argparse.ArgumentParser(description="Extract video frames, caption them with local DeepSeek-VL2, and save results to DB.")
    parser.add_argument("--source-dir", type=Path, default=DEFAULT_SOURCE_DIR)
    parser.add_argument("--batch-name", default="")
    parser.add_argument("--frame-interval-seconds", type=float, default=1.0)
    parser.add_argument("--model-id", default=DEFAULT_MODEL_ID)
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--limit-videos", type=int, default=None)
    parser.add_argument("--max-frames-per-video", type=int, default=None)
    parser.add_argument("--force", action="store_true", help="Re-caption frames that already have successful results.")
    args = parser.parse_args()

    if args.frame_interval_seconds <= 0:
        raise SystemExit("--frame-interval-seconds must be greater than 0")

```

```

if not args.source_dir.exists():
    raise SystemExit(f"source directory not found: {args.source_dir}")

settings = get_settings()
store = get_store()
service = get_vision_service()
service.settings.deepseek_vl2_prompt = args.prompt

source_dir = args.source_dir.resolve()
batch_name = args.batch_name.strip() or source_dir.name
output_dir = settings.data_dir / "video_captions" / _slug(batch_name)
output_dir.mkdir(parents=True, exist_ok=True)

batch = store.create_or_update_video_caption_batch(
    VideoCaptionBatch(
        name=batch_name,
        source_dir=str(source_dir),
        output_dir=str(output_dir),
        model_id=args.model_id,
        prompt=args.prompt,
        frame_interval_seconds=args.frame_interval_seconds,
        status="processing",
    )
)

video_paths = sorted(path for path in source_dir.iterdir() if path.is_file() and path.suffix.lower() in
VIDEO_EXTENSIONS)
if args.limit_videos is not None:
    video_paths = video_paths[: args.limit_videos]

print(f"Batch {batch.name}: {len(video_paths)} video(s), interval={args.frame_interval_seconds}s", flush=True)
for position, video_path in enumerate(video_paths, start=1):
    print(f"[{position}/{len(video_paths)}] {video_path.name}", flush=True)
    _process_video(
        video_path=video_path,
        batch=batch,
        output_dir=output_dir,
        frame_interval_seconds=args.frame_interval_seconds,
        model_id=args.model_id,
        store=store,
        service=service,
        max_frames=args.max_frames_per_video,
        force=args.force,
    )

store.update_video_caption_batch_counts(batch.batch_id, status="success")

def _process_video(
    video_path: Path,
    batch: VideoCaptionBatch,
    output_dir: Path,
    frame_interval_seconds: float,
    model_id: str,
    store,
    service,
    max_frames: int | None,
    force: bool,
) -> None:
    try:
        import cv2
    except Exception as exc:
        raise RuntimeError("opencv-python-headless is required for video frame extraction") from exc

    capture = cv2.VideoCapture(str(video_path))
    if not capture.isOpened():
        video = VideoCaptionVideo(
            batch_id=batch.batch_id,
            filename=video_path.name,
            source_path=str(video_path),
            status="failed",

```

```

        error="could not open video",
    )
    store.create_or_update_video_caption_video(video)
    return

fps = float(capture.get(cv2.CAP_PROP_FPS) or 0) or None
frame_count = int(capture.get(cv2.CAP_PROP_FRAME_COUNT) or 0) or None
width = int(capture.get(cv2.CAP_PROP_FRAME_WIDTH) or 0) or None
height = int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0) or None
duration_ms = int(frame_count / fps * 1000) if fps and frame_count else None
frame_dir = output_dir / video_path.stem / "frames"
frame_dir.mkdir(parents=True, exist_ok=True)

video = store.create_or_update_video_caption_video(
    VideoCaptionVideo(
        batch_id=batch.batch_id,
        filename=video_path.name,
        source_path=str(video_path),
        frame_dir=str(frame_dir),
        fps=fps,
        width=width,
        height=height,
        frame_count=frame_count,
        duration_ms=duration_ms,
        status="processing",
    )
)

step = max(1, int(round((fps or 1.0) * frame_interval_seconds)))
target_indexes = range(0, frame_count or 0, step) if frame_count else _unknown_frame_indexes(capture, step)
processed = 0
succeeded = 0

try:
    for frame_index in target_indexes:
        if max_frames is not None and processed >= max_frames:
            break

        existing = store.get_video_caption_frame_by_index(video.video_id, frame_index)
        if existing and existing.status == "success" and not force:
            processed += 1
            succeeded += 1
            continue

        capture.set(cv2.CAP_PROP_POS_FRAMES, frame_index)
        ok, frame = capture.read()
        if not ok:
            continue

        timestamp_ms = int(frame_index / fps * 1000) if fps else 0
        image_path = frame_dir / f"frame_{frame_index:06d}_{timestamp_ms:010d}.jpg"
        cv2.imwrite(str(image_path), frame)
        frame_height, frame_width = frame.shape[:2]
        record = VideoCaptionFrame(
            frame_id=existing.frame_id if existing else new_id("vcframe"),
            batch_id=batch.batch_id,
            video_id=video.video_id,
            frame_index=frame_index,
            timestamp_ms=timestamp_ms,
            image_path=str(image_path),
            width=int(frame_width),
            height=int(frame_height),
            model_id=model_id,
            status="processing",
        )
        record.image_url = f"/api/video-captions/frames/{record.frame_id}/image"

    try:
        description = service._deepseek_vl2_caption(image_path)
        record.description_text = description
        record.status = "success"

```

```

        succeeded += 1
        record.result_json = {
            "description": description,
            "model": model_id,
            "video": video_path.name,
            "frameIndex": frame_index,
            "timestampMs": timestamp_ms,
        }
    except Exception as exc:
        record.status = "failed"
        record.error = str(exc)
    store.save_video_caption_frame(record)
    processed += 1
    print(
        f" frame={frame_index} status={record.status} timestamp_ms={timestamp_ms}",
        flush=True,
    )
finally:
    capture.release()

saved_status = "success" if succeeded > 0 else "failed"
saved_error = "" if succeeded > 0 else "no successful frame captions"
store.create_or_update_video_caption_video(
    VideoCaptionVideo(
        video_id=video.video_id,
        batch_id=batch.batch_id,
        filename=video.filename,
        source_path=video.source_path,
        frame_dir=video.frame_dir,
        fps=video.fps,
        width=video.width,
        height=video.height,
        frame_count=video.frame_count,
        duration_ms=video.duration_ms,
        status=saved_status,
        error=saved_error,
        created_at=video.created_at,
        updated_at=now_utc(),
    )
)
store.update_video_caption_batch_counts(batch.batch_id, status="processing")

def _unknown_frame_indexes(capture, step: int):
    frame_index = 0
    while True:
        ok = capture.grab()
        if not ok:
            break
        if frame_index % step == 0:
            yield frame_index
        frame_index += 1

def _slug(value: str) -> str:
    slug = re.sub(r"^[A-Za-z0-9_-]+", "-", value.strip()).strip("-")
    return slug or "video-captions"

if __name__ == "__main__":
    main()

```

backend/scripts/update_video_caption_titles.py

```

import re
import sys
from collections import Counter
from pathlib import Path

```

```

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from app.core.dependencies import get_store

KEYWORD_RULES: list[tuple[str, tuple[str, ...]]] = [
    ("接触网", ("接触网", "供电线路", "电力线路", "架空线路", "电线", "电线杆", "支柱", "横梁")),
    ("人员", ("人员", "行人", "工人", "工作人员", "作业人员", "人体")),
    ("轨道", ("轨道", "铁轨", "钢轨", "铁路轨道", "道岔", "枕木", "碎石")),
    ("标识牌", ("标识牌", "标志牌", "警示牌", "牌", "文字", "标有")),
    ("设备", ("设备", "箱体", "控制箱", "信号设备", "仪器", "探伤")),
    ("车辆", ("车辆", "列车", "火车", "车厢", "汽车", "工程车")),
    ("桥梁", ("桥梁", "桥", "高架", "涵洞")),
    ("站场", ("站台", "站场", "车站", "建筑物", "站房")),
    ("植被", ("树木", "绿树", "植被", "草地", "灌木")),
    ("雨天地面", ("湿滑", "积水", "雨", "水洼", "潮湿")),
    ("施工", ("施工", "作业", "实训", "检修", "维护")),
    ("安全风险", ("风险", "危险", "警示", "安全", "障碍", "侵限")),
]

def main() -> None:
    store = get_store()
    updated = 0
    for batch in store.list_video_caption_batches():
        for video in store.list_video_caption_videos(batch.batch_id):
            frames, _ = store.list_video_caption_frames(video.video_id, page=1, page_size=5000)
            keywords = _extract_keywords(" ".join(frame.description_text for frame in frames))
            display_name = _display_name(video.filename, keywords)
            store.update_video_caption_video_summary(video.video_id, display_name, keywords)
            updated += 1
            print(f"{video.filename} -> {display_name}")
    print(f"updated {updated} video title(s)")

def _extract_keywords(text: str) -> list[str]:
    counts: Counter[str] = Counter()
    for keyword, aliases in KEYWORD_RULES:
        counts[keyword] = sum(text.count(alias) for alias in aliases)
    selected = [keyword for keyword, count in counts.most_common() if count > 0]
    return selected[:4] or ["铁路场景"]

def _display_name(filename: str, keywords: list[str]) -> str:
    timestamp = _timestamp_label(filename)
    stem = Path(filename).stem
    suffix = " • ".join(keywords[:3])
    return f"{timestamp} • {suffix}" if timestamp else f"{stem} • {suffix}"

def _timestamp_label(filename: str) -> str:
    match = re.search(r"DJI_(\d{8})(\d{6})_(\d+)", filename)
    if not match:
        return ""
    date, time, index = match.groups()
    return f"{date[:4]}-{date[4:6]}-{date[6:8]} {time[:2]}:{time[2:4]}:{time[4:6]} #{index}"

if __name__ == "__main__":
    main()

```

backend/tests/test_smoke.py

```

import io

import pytest
from fastapi.testclient import TestClient
from PIL import Image

```

```

from app.core.config import get_settings
from app.core.dependencies import get_store
from app.main import create_app
from app.api.ai import _result_from_analysis
from app.models.schemas import VideoCaptionBatch, VideoCaptionFrame, VideoCaptionVideo

def _register_student(client: TestClient, username: str = "student001") -> dict:
    response = client.post(
        "/api/auth/register",
        json={"username": username, "displayName": f"学生 {username}", "password": "railway123"},
    )
    assert response.status_code == 200
    return response.json()["data"]

def test_upload_detect_edit_and_export(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path} / 'test.db'")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("RUN_TASKS_INLINE", "true")
    get_settings.cache_clear()
    get_store.cache_clear()

    client = TestClient(create_app())
    _register_student(client)

    health = client.get("/health")
    assert health.status_code == 200
    assert health.json()["status"] == "ok"

    image = Image.new("RGB", (640, 360), color="white")
    buffer = io.BytesIO()
    image.save(buffer, format="PNG")
    buffer.seek(0)

    upload = client.post(
        "/api/assets/upload",
        files={"file": ("rail.png", buffer.getvalue(), "image/png")},
    )
    assert upload.status_code == 200
    asset = upload.json()
    assert asset["type"] == "image"
    assert asset["width"] == 640

    task_response = client.post(
        "/api/tasks/detect",
        json={"asset_id": asset["id"], "confidence": 0.25, "iou": 0.7, "frame_stride": 1},
    )
    assert task_response.status_code == 202
    task = task_response.json()
    assert task["status"] == "completed"

    annotations_response = client.get(f"/api/assets/{asset['id']}/annotations")
    assert annotations_response.status_code == 200
    annotations = annotations_response.json()
    assert annotations["frames"][0]["objects"][0]["label"] == "railway_target"
    assert annotations["frames"][0]["image_url"] == f"/api/assets/{asset['id']}/frames/0/image"

    frame_response = client.get(annotations["frames"][0]["image_url"])
    assert frame_response.status_code == 200
    assert frame_response.headers["content-type"] == "image/jpeg"

    annotations["frames"][0]["objects"][0]["label"] = "confirmed_target"
    annotations["frames"][0]["objects"][0]["status"] = "edited"
    save_response = client.put(f"/api/assets/{asset['id']}/annotations", json=annotations)
    assert save_response.status_code == 200
    assert save_response.json()["frames"][0]["objects"][0]["label"] == "confirmed_target"

    export_response = client.get(f"/api/assets/{asset['id']}/export?format=json")
    assert export_response.status_code == 200

```

```

assert export_response.json()["frames"][0]["objects"][0]["label"] == "confirmed_target"

yolo_response = client.get(f"/api/assets/{asset['id']}/export?format=yolo")
assert yolo_response.status_code == 200
assert "confirmed_target" in yolo_response.text

review_response = client.post(
    f"/api/assets/{asset['id']}/annotations/review",
    json={"status": "approved"},
)
assert review_response.status_code == 200
reviewed = review_response.json()
assert reviewed["review_status"] == "approved"
assert reviewed["frames"][0]["review_status"] == "approved"

```

```

def test_video_upload_extracts_frames_and_detects(monkeypatch, tmp_path):
    cv2 = pytest.importorskip("cv2")
    import numpy as np

    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:///{'tmp_path' / 'test.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("RUN_TASKS_INLINE", "true")
    get_settings.cache_clear()
    get_store.cache_clear()

    video_path = tmp_path / "rail.mp4"
    writer = cv2.VideoWriter(
        str(video_path),
        cv2.VideoWriter_fourcc(*"mp4v"),
        5,
        (160, 90),
    )
    for value in (40, 90, 140):
        writer.write(np.full((90, 160, 3), value, dtype=np.uint8))
    writer.release()

    client = TestClient(create_app())
    _register_student(client)
    upload = client.post(
        "/api/assets/upload",
        files={"file": ("rail.mp4", video_path.read_bytes(), "video/mp4")},
    )
    assert upload.status_code == 200
    asset = upload.json()
    assert asset["type"] == "video"
    assert asset["frame_count"] == 3

    task_response = client.post(
        "/api/tasks/detect",
        json={"asset_id": asset["id"], "confidence": 0.25, "iou": 0.7, "frame_stride": 1},
    )
    assert task_response.status_code == 202
    assert task_response.json()["status"] == "completed"

    annotations_response = client.get(f"/api/assets/{asset['id']}/annotations")
    assert annotations_response.status_code == 200
    annotations = annotations_response.json()
    assert [frame["frame_index"] for frame in annotations["frames"]] == [0, 1, 2]
    assert all(frame["objects"] for frame in annotations["frames"])
    assert client.get(annotations["frames"][1]["image_url"]).status_code == 200

def test_database_queue_claims_queued_task(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:///{'tmp_path' / 'queue.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("RUN_TASKS_INLINE", "false")
    monkeypatch.setenv("TASK_WORKER_ENABLED", "false")
    get_settings.cache_clear()

```

```

get_store.cache_clear()

image = Image.new("RGB", (160, 90), color="white")
buffer = io.BytesIO()
image.save(buffer, format="PNG")

client = TestClient(create_app())
_register_student(client)
upload = client.post(
    "/api/assets/upload",
    files={"file": ("queued.png", buffer.getvalue(), "image/png")},
)
asset = upload.json()
response = client.post("/api/tasks/detect", json={"asset_id": asset["id"]})
assert response.status_code == 202
assert response.json()["status"] == "queued"

claimed = get_store().claim_next_task()
assert claimed is not None
assert claimed.id == response.json()["id"]
assert claimed.status == "running"

def test_label_config_crud_uses_zero_based_ids(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path / 'labels.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("RUN_TASKS_INLINE", "true")
    get_settings.cache_clear()
    get_store.cache_clear()

    client = TestClient(create_app())
    _register_student(client)

    created = client.post(
        "/api/labels",
        json={"englishName": "person", "chineseName": "行人", "description": "可见人体目标"},
    )
    assert created.status_code == 200
    assert created.json()["data"]["labelId"] == 0

    copied = client.post("/api/labels/0/copy")
    assert copied.status_code == 200
    assert copied.json()["data"]["labelId"] == 1
    assert copied.json()["data"]["englishName"] == "person"
    assert copied.json()["data"]["chineseName"] == "行人"

    updated = client.put(
        "/api/labels/1",
        json={"englishName": "worker", "chineseName": "工作人员", "description": "现场作业人员"},
    )
    assert updated.status_code == 200
    assert updated.json()["data"]["description"] == "现场作业人员"

    listed = client.get("/api/labels")
    assert listed.status_code == 200
    assert [label["labelId"] for label in listed.json()["data"]] == [0, 1]

    deleted = client.delete("/api/labels/0")
    assert deleted.status_code == 200
    assert deleted.json()["data"]["deleted"] is True

def test_image_history_defaults_to_all_backend_sessions(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path / 'image_history.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("TASK_WORKER_ENABLED", "false")
    get_settings.cache_clear()
    get_store.cache_clear()

```

```

client = TestClient(create_app())
_register_student(client)
image = Image.new("RGB", (160, 90), color="white")
buffer = io.BytesIO()
image.save(buffer, format="PNG")
payload = buffer.getvalue()

for session_id in ("browser-a", "browser-b"):
    response = client.post(
        "/api/images/upload",
        data={"taskType": "detection", "sessionId": session_id},
        files={"file": (f"{session_id}.png", payload, "image/png")},
    )
    assert response.status_code == 200

all_history = client.get("/api/images/list?taskType=detection&pageSize=200")
assert all_history.status_code == 200
assert all_history.json()["data"]["total"] == 2
assert {item["sessionId"] for item in all_history.json()["data"]["records"]} == {"browser-a", "browser-b"}

scoped_history = client.get("/api/images/list?taskType=detection&pageSize=200&sessionId=browser-a")
assert scoped_history.status_code == 200
assert scoped_history.json()["data"]["total"] == 1
assert scoped_history.json()["data"]["records"][0]["sessionId"] == "browser-a"

def test_login_session_persists_and_student_images_are_isolated(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path / 'auth.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("TASK_WORKER_ENABLED", "false")
    get_settings.cache_clear()
    get_store.cache_clear()

    app = create_app()
    first_client = TestClient(app)
    first_user = _register_student(first_client, "student-a")
    session_cookie = first_client.cookies.get("railway_session")
    assert session_cookie

    image = Image.new("RGB", (160, 90), color="white")
    buffer = io.BytesIO()
    image.save(buffer, format="PNG")
    upload = first_client.post(
        "/api/images/upload",
        data={"taskType": "detection", "sessionId": "default"},
        files={"file": ("owned.png", buffer.getvalue(), "image/png")},
    )
    assert upload.status_code == 200

    restored_client = TestClient(app)
    restored_client.cookies.set("railway_session", session_cookie)
    me = restored_client.get("/api/auth/me")
    assert me.status_code == 200
    assert me.json()["data"]["userId"] == first_user["userId"]

    assert restored_client.post("/api/auth/logout").status_code == 200
    assert first_client.get("/api/images/list").status_code == 401

    second_user = _register_student(first_client, "student-b")
    assert second_user["userId"] != first_user["userId"]
    second_history = first_client.get("/api/images/list?taskType=detection&pageSize=200")
    assert second_history.status_code == 200
    assert second_history.json()["data"]["total"] == 0

def test_student_submission_requires_reviewer_and_approved_result_is_locked(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path / 'review.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("TASK_WORKER_ENABLED", "false")

```

```

get_settings.cache_clear()
get_store.cache_clear()

app = create_app()
admin_client = TestClient(app)
admin = _register_student(admin_client, "admin-user")
assert admin["role"] == "admin"

student_client = TestClient(app)
student = _register_student(student_client, "student-user")
assert student["role"] == "student"

reviewer_client = TestClient(app)
reviewer = _register_student(reviewer_client, "reviewer-user")
promoted = admin_client.put(f"/api/auth/users/{reviewer['userId']}/role", json={"role": "reviewer"})
assert promoted.status_code == 200
assert promoted.json()["data"]["role"] == "reviewer"

image = Image.new("RGB", (160, 90), color="white")
buffer = io.BytesIO()
image.save(buffer, format="PNG")
uploaded = student_client.post(
    "/api/images/upload",
    data={"taskType": "detection", "sessionId": "default"},
    files={"file": ("review.png", buffer.getvalue(), "image/png")},
).json()["data"]
task = student_client.post(
    "/api/image-tasks/create",
    json={"imageId": uploaded["imageId"], "taskType": "detection", "sessionId": "default"},
).json()["data"]
saved = student_client.post(
    "/api/image-tasks/result/save",
    json={
        "taskId": task["taskId"],
        "imageId": uploaded["imageId"],
        "taskType": "detection",
        "annotationJson": {"shapes": []},
    },
)
assert saved.status_code == 200

submitted = student_client.post(f"/api/image-tasks/result/{task['taskId']}/submit")
assert submitted.status_code == 200
assert submitted.json()["data"]["reviewStatus"] == "pending_review"

reviewer_history = reviewer_client.get("/api/images/list?taskType=detection&pageSize=200")
assert reviewer_history.status_code == 200
assert reviewer_history.json()["data"]["total"] == 1
reviewer_result = reviewer_client.get(
    f"/api/image-tasks/result?imageId={uploaded['imageId']}&taskType=detection&sessionId=default"
)
assert reviewer_result.status_code == 200
assert reviewer_result.json()["data"]["taskId"] == task["taskId"]
reviewer_edit = reviewer_client.put(
    "/api/image-tasks/annotation",
    json={
        "taskId": task["taskId"],
        "imageId": uploaded["imageId"],
        "taskType": "detection",
        "annotationJson": {"shapes": []},
    },
)
assert reviewer_edit.status_code == 403
assert student_client.post(
    f"/api/image-tasks/result/{task['taskId']}/review",
    json={"status": "approved", "comment": "student cannot review"},
).status_code == 403

approved = admin_client.post(
    f"/api/image-tasks/result/{task['taskId']}/review",
    json={"status": "approved", "comment": "标注符合要求"},
)

```

```
)
assert approved.status_code == 200
assert approved.json()["data"]["reviewStatus"] == "approved"
assert approved.json()["data"]["reviewComment"] == "标注符合要求"
```

```
locked = student_client.post(
    "/api/image-tasks/result/save",
    json={
        "taskId": task["taskId"],
        "imageId": uploaded["imageId"],
        "taskType": "detection",
        "annotationJson": {"shapes": []},
    },
)
assert locked.status_code == 409
```

```
def test_real_vision_result_keeps_general_model_labels():
    result = _result_from_analysis(
        "detection",
        {
            "objects": [
                {
                    "label": "person",
                    "confidence": 0.91,
                    "bbox": {"x": 10, "y": 20, "width": 30, "height": 40},
                },
                {
                    "label": "car",
                    "confidence": 0.82,
                    "bbox": {"x": 50, "y": 60, "width": 70, "height": 80},
                },
            ]
        },
    ),
)

assert [box["label"] for box in result["boxes"]] == ["person", "car"]
assert all(box["label"] != "railway_target" for box in result["boxes"])
```

```
def test_video_caption_results_are_queryable(monkeypatch, tmp_path):
    monkeypatch.setenv("DATA_DIR", str(tmp_path / "data"))
    monkeypatch.setenv("DATABASE_URL", f"sqlite+pysqlite:/// {tmp_path / 'video_captions.db'}")
    monkeypatch.setenv("INFERENCE_BACKEND", "mock")
    monkeypatch.setenv("RUN_TASKS_INLINE", "true")
    get_settings.cache_clear()
    get_store.cache_clear()

    client = TestClient(create_app())
    _register_student(client)
    store = get_store()
    frame_dir = tmp_path / "frames"
    frame_dir.mkdir()
    frame_path = frame_dir / "frame_000000.jpg"
    Image.new("RGB", (120, 80), color="white").save(frame_path, format="JPEG")

    batch = store.create_or_update_video_caption_batch(
        VideoCaptionBatch(
            name="railway-multi-target-recognition",
            source_dir="/videos",
            output_dir=str(frame_dir),
            status="success",
        )
    )

    video = store.create_or_update_video_caption_video(
        VideoCaptionVideo(
            batch_id=batch.batch_id,
            filename="DJI_0001.MP4",
            source_path="/videos/DJI_0001.MP4",
            frame_dir=str(frame_dir),
            status="success",
        )
    )
```

```

    )
  )
  frame = store.save_video_caption_frame(
    VideoCaptionFrame(
      batch_id=batch.batch_id,
      video_id=video.video_id,
      frame_index=0,
      timestamp_ms=0,
      image_path=str(frame_path),
      image_url="/api/video-captions/frames/test/image",
      width=120,
      height=80,
      description_text="画面中可见铁路轨道和作业人员",
      status="success",
    )
  )
  store.update_video_caption_batch_counts(batch.batch_id, status="success")

  batches = client.get("/api/video-captions/batches")
  assert batches.status_code == 200
  assert batches.json()["data"][0]["totalVideos"] == 1
  assert batches.json()["data"][0]["totalFrames"] == 1

  videos = client.get(f"/api/video-captions/batches/{batch.batch_id}/videos")
  assert videos.status_code == 200
  assert videos.json()["data"][0]["filename"] == "DJI_0001.MP4"

  frames = client.get(f"/api/video-captions/videos/{video.video_id}/frames?query=人员")
  assert frames.status_code == 200
  assert frames.json()["data"]["records"][0]["frameId"] == frame.frame_id

  image_response = client.get(f"/api/video-captions/frames/{frame.frame_id}/image")
  assert image_response.status_code == 200
  assert image_response.headers["content-type"] == "image/jpeg"

```

frontend/index.html

```

<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <title>铁路多目标识别</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>

```

frontend/package-lock.json

```

{
  "name": "railway-recognition-frontend",
  "version": "0.1.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "railway-recognition-frontend",
      "version": "0.1.0",
      "dependencies": {
        "@lucide/vue": "^1.17.0",
        "@mediapipe/tasks-vision": "^0.10.22-rc.20250304",
        "@vitejs/plugin-vue": "^6.0.0",
        "vite": "^7.2.0",
        "vue": "^3.5.0"
      }
    }
  }

```

```

    },
    "devDependencies": {
      "typescript": "^5.9.0"
    }
  },
  "node_modules/@babel/helper-string-parser": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-Pb5ijPrZ89GDH8223L4UP8i6QApWxs04RbPQJTEwDV0/keR2E36MeKnyr6LYmUUvqRRI+Iv87SuF1W6ErINzYw==",
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-validator-identifier": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-qehxGkRj55h/ff8EMaJ+cYhyaKLHixqYDn682wQD7RNP9UuJ0QsHog2uS0r2vzr4pw+sXf90NeeayjcNaX3fFg==",
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/parser": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-hn0RnJP/1P/zFEndoeX+n+t1RwWRJiJpM/j07FW32Kn9r5+sJB2JW0dYo4L6k78j15eCwY3Gm/7364B1EMwtNg==",
    "license": "MIT",
    "dependencies": {
      "@babel/types": "^7.29.7"
    },
    "bin": {
      "parser": "bin/babel-parser.js"
    },
    "engines": {
      "node": ">=6.0.0"
    }
  },
  "node_modules/@babel/types": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-4zBIxpPzowiZpusoFkyGVwakdRJUyuH5PxQ/PrqghfdFWWasvncdPfQXHrenDai+gyLARuLZjZowC0j6fjT4pA==",
    "license": "MIT",
    "dependencies": {
      "@babel/helper-string-parser": "^7.29.7",
      "@babel/helper-validator-identifier": "^7.29.7"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@esbuild/aix-ppc64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-EKX3QvmhziEMdEJokhALr0YiD0LhQNwDqkPYyPhiSwKrh7/4KRjQc04sZ8db+5DVVnZ1LmbNDI1uAMPEUBnQPg==",
    "cpu": [
      "ppc64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "aix"
    ],
    "engines": {
      "node": ">=18"
    }
  }
}

```

```

    },
    "node_modules/@esbuild/android-arm": {
      "version": "0.27.7",
      "resolved": "****",
      "integrity": "sha512-
jbPXvB4Yj2yBV7HUfE2KHe4GJX51QpLCN1pGbYjvsyCZbQmies29EoJbkEc+vYuU5o45AfQn37vZLyXy4YJ8RQ==",
      "cpu": [
        "arm"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "android"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/android-arm64": {
      "version": "0.27.7",
      "resolved": "****",
      "integrity": "sha512-
62dPZHpIXzvChfvfLJow3q5dDtINMkwiRzPylSCfriLvZeq0a1bWChrGx/BbUbPw0rsWKMn8idSLlkLzBy+dgQ==",
      "cpu": [
        "arm64"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "android"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/android-x64": {
      "version": "0.27.7",
      "resolved": "****",
      "integrity": "sha512-
x5VpMODneVDb70PYV2VQOmIUUiBtY3D3mPBG8NxVk5CogneYhkR7MmM3yR/uMdITLrC1mL/NV1rj4bMJuy9MCg==",
      "cpu": [
        "x64"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "android"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/darwin-arm64": {
      "version": "0.27.7",
      "resolved": "****",
      "integrity": "sha512-
5LckdqeUBPLKUwvoCXIgI2D9/ABmPq3Rdp7IfL70393YgaASt7tbju3Ac+ePVi3KDH6N2RqePfHnXkaDtY9fkW==",
      "cpu": [
        "arm64"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "darwin"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/darwin-x64": {
      "version": "0.27.7",

```

```

    "resolved": "****",
    "integrity": "sha512-
rYnXrKcXuT7Z+WL5K980jVFdvVKhCHhUwid+dDYQpH+qu+TefcomiMAJpIiC2EM3Rjtq0s03StMV/+3w3MyyqQ==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "darwin"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/freebsd-arm64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
B48PqeCsEg0tzmE2GbnM2roU29AMTu0IN91dsM030t+Ydis3z/3Ngoj5hhns0SSwNzS+6JppqWsuHtp6E82L2w==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "freebsd"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/freebsd-x64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
j0BDK5XEjA4m5IJK3bpAQF9/LeLu/Z9ZcdhTRLf4cajLB+8VEhFFRjWgfy3M104r02GQ/b2dLwCUGpiF/eATNQ==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "freebsd"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-arm": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
RkT/YXYBTSULo3+af8Ib0ykH8u2MBh57o7q/DAs3lTJlyVQkgQvLrPTnjIzzRPQyavxtPtfg0EopvDyIt0j1rA==",
    "cpu": [
      "arm"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-arm64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
RZPHBoxXuNnPQ09rvjh5jdkRmVizktkT7TCDkDmQ0W2SwHInKCAV95GRuvdSvA7w4VMwfCjUiPwDi0Z06Nfe9A==",

```

```

    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-ia32": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
GA48aKNkyQDbd3KtkpLYWT102C5sn/EZTY4XR0kx0NgruHPU72L+gW+FfF8tf2cFjeHaRbWp0Ya/uRBz/Xq1Pg==",
    "cpu": [
      "ia32"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-loong64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
a4P0ruNM2oWsD4WKvBSEKGIiWQF8fZ0A5ych0t6JBpZ+JN2n2JH9WAv56S0yu9X5IqAjqSIPTaJkqN8F7X0Q5Q==",
    "cpu": [
      "loong64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-mips64el": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
KabT5I6StirGfIz0FMgl1I+R1H73Gp0ofL9A3nG3i/cYFJzKHhouBV5VwK1CSgKvVaG4q1RNpCTR2LuTVB3fIw==",
    "cpu": [
      "mips64el"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-ppc64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
gRsL4x6wsGHGRqhtI+i fpN/vp0FTQtnbsupUF5R5YTAgy/lKeLYR1hXbnBdzDjGbMYjVJLJTd20FmMewAgwIQ==",
    "cpu": [
      "ppc64"
    ],

```

```
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-riscv64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-hL25Lbx01Q0ngGzu2U5xeXtxCw+/GvMN3ejANqXkxZ/opySAZMrc+9LY/WyjAan41unrR3YrmtTsUpwT66InQ==",
    "cpu": [
      "riscv64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-s390x": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-2k8go8Ycu1Kb46vEelhu1vqEP+UeRVj2zY1pSuPdgvbd5ykAw82Lrro28vXURmzEsUV0NzCf54yARIK8r0fdw==",
    "cpu": [
      "s390x"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-x64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-hzznmADPt+0msYzw1EE33ccA+HPdIqiCRq7cQeL1JLq2gb1+0yWBkMcrYGBJ+sxVzve2ZJEVeePbLM2iEIZSxA==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/netbsd-arm64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-b6pqtrQdigZBwZxAn1UpazEi svwaIDvdbMbmrl y7cDTMFnw/+3LVxxCTG0rkPVnsYIosJJXAsILG9XcQS+Yu6w==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
```

```
    "netbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/netbsd-x64": {
  "version": "0.27.7",
  "resolved": "****",
  "integrity": "sha512-0fatKLojr6U+WN5EDYuoQhtM+1xco+/6FSzJJnuWiUw5eVcicybK3dq5EeV/QHT1uy6GoDhGbFpprUiHUYggrw==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "netbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openbsd-arm64": {
  "version": "0.27.7",
  "resolved": "****",
  "integrity": "sha512-AFuojMQTxAz75Fo8idVcqoQWEHIXFRb0c1TrVcFSgCZtQfSdc1RXgB3tj0n/krRHENUB4j00bfGjyl2mJrU37A==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openbsd-x64": {
  "version": "0.27.7",
  "resolved": "****",
  "integrity": "sha512+A1NjmfM8WNDv5CLVQYJ5PshuRm/4cI6WMZRg1by1GwPIQPCTs1GLEUHwi iQGT5zDdyLiRM/L1G0Pv54gvtKIg==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openharmony-arm64": {
  "version": "0.27.7",
  "resolved": "****",
  "integrity": "sha512+KrvYb/C8zA9CU/g0sR6w2RBw7IGc5J2BPnc3dYc5VJxHCSF1yNMxTV5LQ7GuKteQXZtspjFbiuW5/d0j7H4Yw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openharmony"
  ],
  "engines": {
```

```
    "node": ">=18"
  },
  "node_modules/@esbuild/sunos-x64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-ikktIhFBzQNT/QDyOL580ti9+5mL/YZeUPKU2ivGtGjdTYoqz6j0bj6n0MfhASpS4GU4Q/CLh1QtXWAvCYKamA==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "sunos"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-arm64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-7yRhbHvPqSpRUV7Q20VuDwbjW5kIMwTHpptuUzV+AA46kiPze5Z7qgt6CLCK3pWFrHeNfDd1VKgyP40+ng17CA==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-ia32": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-SmwKXe6VHIyZYbBLJrh0oCJRB/Z1tckzmgtLFFY0fpMAx63BJEaL9ExI8x7v0oA03Zh6D/Oi1gVxYEr5oUCFhw==",
    "cpu": [
      "ia32"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-x64": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-56hiAJPhwQ1R4i+21FVF7V8kSD5zZTdHcVuRFMW0hn753vVfQN8xLx4uOPT4xoGH0Z/oVATuR82AiQSTDipaHg==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">=18"
    }
  },
}
```

```

"node_modules/@jridgewell/sourcemap-codec": {
  "version": "1.5.5",
  "resolved": "****",
  "integrity": "sha512-
cYQ9310grqxueWbl+uWuIUaiUaDcj7W0Q5fVhEljNVgRf0Uhy9fy2zTvfqWsnbh8SL70VScFbICvJnLKB00g==",
  "license": "MIT"
},
"node_modules/@lucide/vue": {
  "version": "1.17.0",
  "resolved": "****",
  "integrity": "sha512-
6Q1ZHgr5FbmJzKWe5BxLNdJlJ2LbmuH1zwDtVzUJofX0w9UREwKgq4F4jwKqFYyIS4Rj3FiJvDi2k6djukmmw==",
  "license": "ISC",
  "peerDependencies": {
    "vue": ">=3.0.1"
  }
},
"node_modules/@mediapipe/tasks-vision": {
  "version": "0.10.22-rc.20250304",
  "resolved": "****",
  "integrity": "sha512-
dElxVXMFgthshfIj+qAVm8KE2jmNo2p8oXFib8WzEjb7GNax/CLWbC8UJfoSZwjEMVrdHJ4YUfa7P3ifl6MIWw==",
  "license": "Apache-2.0"
},
"node_modules/@rollup/rollup-android-arm-eabi": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
JnBB8MdXj45cajvTu05FmPlvFVJRQgvrz1uSEL3NwqFnReAPGwb8EanbGi4z2nRaqlzJJSv5/JmycoTKLRZxHA==",
  "cpu": [
    "arm"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "android"
  ]
},
"node_modules/@rollup/rollup-android-arm64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
Jx2g7iSjw4A0T0HDPHM9RV3GNjRXwybWtSFZiZAYUTjUwjVrYIwq3kBf+LnhqJLzXFAqTAh2F7IGI+0568exPw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "android"
  ]
},
"node_modules/@rollup/rollup-darwin-arm64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
0F1L/Z3Eqv8mT2n3dCpe08GcTvHvVqkP5/t6DMsn0KzhYVcg+s7NcL5DS8qjKYEei06Az0Gt6nyB0Ray5qILCA==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "darwin"
  ]
},
"node_modules/@rollup/rollup-darwin-x64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
qLttcH871ujY4YcVfUSSHhOw+CsoTatYz8gRbH07Bb92QH059/P0y5do1KM541fY0BpD2x4AJH/gID0zFiqVKQ==",

```

```
"cpu": [
  "x64"
],
"license": "MIT",
"optional": true,
"os": [
  "darwin"
]
},
"node_modules/@rollup/rollup-freebsd-arm64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
fUI4RapGE00h3mb8mgfvc102nU1RpDZUKnDQm3xB1Ipg7C2wTs5Kstz7G2uWK99a8S2yTMq8/P4uycwNa0nJyw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "freebsd"
  ]
},
"node_modules/@rollup/rollup-freebsd-x64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
H5YrdvJaDtI/U9/emrD4b++xkvp3y/Jv0e4rizHbxvkyMfRS/CiRYdji+PL8D0brEaNFwUh1drQxgAGIL6Xudw==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "freebsd"
  ]
},
"node_modules/@rollup/rollup-linux-arm-gnueabihf": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
Q8CBCCQtDFrYtXoeUXSrnfXK0nyUhx6bz+SkL6A0E7V8kaICJ5pamq1WtbfVpVgHr5TSpXY6ak3avmDc5fHTyJA==",
  "cpu": [
    "arm"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-arm-musleabihf": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
nwnhk1581l0FBVellGcVCAT00i06onEA3WB53sf01V03I0UPbKMh9sX0NYME2K0ovXcNayJfNtHfm6mpJELatQ==",
  "cpu": [
    "arm"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
},
```

```

"node_modules/@rollup/rollup-linux-arm64-gnu": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
x5Xr49hwt3hdW75U0Zm3395YwwzPyauktslv29KpWL/T+vVAzoT3azLcTwv0eMciBNrx+DYjH4paehHoLpPvpq==",
  "cpu": [
    "arm64"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-arm64-musl": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
unMS3H73DpaoPyjGKleM/s0mkmsauTENpw4INQY8y4+IuLjNjueQ5QCtC0D3N38Y38yhAU80oZ20S2Tm6w==",
  "cpu": [
    "arm64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-loong64-gnu": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
zNZzGRnAhwjFEYmvphJRv5XaQGs62cCmeYYHUT//NbVEnHauw+I85nGG+SiVg5ld4GX8D1IbKIX+ozITQnhMQ==",
  "cpu": [
    "loong64"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-loong64-musl": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
LdpWGL8X209B2SIvWjqlc8VZgM6PKfontSerGepulDQmHYrA0tnMCXeJkxXGbc+PPZV0uu5czJo7fNV6aew8rQ==",
  "cpu": [
    "loong64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-ppc64-gnu": {
  "version": "4.61.1",
  "resolved": "****"

```

```
    "integrity": "sha512-
EC5kTtNaNG0mbMGqar8dvJy6y/hg99GAwjfBz++pxZhQATXGcRjd6c5en5wcbru0vkRmiMGsQKdMJ00f6sza4g==",
    "cpu": [
      "ppc64"
    ],
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-ppc64-musl": {
    "version": "4.61.1",
    "resolved": "****",
    "integrity": "sha512-
8hiwp6D4acEcNK78I4rP0/XtS1sknWIAMJBpDR4L6zUtyTm5KiTDr5bXmWt4foY7nAN7AThDHgkLIEZOWKbzWw==",
    "cpu": [
      "ppc64"
    ],
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-riscv64-gnu": {
    "version": "4.61.1",
    "resolved": "****",
    "integrity": "sha512-
10dh/h/BqA7DuMPWSxkR8uks18FRwnw0Eqr5z0TEl+N0wP/OMzKX80FR/Of9xxDA7D5qef1Nzar5WDD2kCCr1g==",
    "cpu": [
      "riscv64"
    ],
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-riscv64-musl": {
    "version": "4.61.1",
    "resolved": "****",
    "integrity": "sha512-
YKJ5lg35DP17gcA0ggnihe+APw9HLyj1Xn7gsmGumBJAUDa6NGXNixJzmkwLhcK9T0uuyQjdamzvJefk07qHZQ==",
    "cpu": [
      "riscv64"
    ],
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-s390x-gnu": {
    "version": "4.61.1",
    "resolved": "****",
    "integrity": "sha512-
MLiL5G2Jj6a7B3LWGctg+XPL9vdXYuzCtNXfx0Q0nPjc2m6ueUktocPGH9bnAM0bNRKb/bAwTujUU7IJQdQA+g==",
    "cpu": [
```

```
    "s390x"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-x64-gnu": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
bVWI0Ik6pV01p4CdUbPP7CJ/434z+0ooYjDuFcR+44N35YvKUC66G8MGnvcWx5mWKW3g61J+t74L3Kj15Kwn2Q==",
  "cpu": [
    "x64"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-x64-musl": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
qy5pBvZbqMFheBz61R1rzsezjm0J702oNGoWtGoY89SZYLufxAJTBAqDChqAIdB4rCiIb i9nF7yZ83GnNiLwSw==",
  "cpu": [
    "x64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-openbsd-x64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
E83TXjI4zm0+5f2q0+U0udaCYIhYwpJ5jq6VCZNIZ+6CbfhKrKAGEze iASBL9ELxAXFsRS9ZhESv8mfnj6TKeg==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ]
},
"node_modules/@rollup/rollup-openharmony-arm64": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-
fbWnKqVkj rJN38vNe3ahkbk6 i e j S/3b0Nt7EEtPpE6RBacZcGXNKbzFHN3GUU LXOPghUg0j6XUGrtjX9z1sIvA==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openharmony"
  ]
}
```

```
]
},
"node_modules/@rollup/rollup-win32-arm64-msvc": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-ArMl38iVAbk0New1ogihQNY6iphLi4ZaRsa037gUzv5yeKPY8TD3Dmy4x2RNC1VztU/uqm+G+/RwFrSka30y2g==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ]
},
"node_modules/@rollup/rollup-win32-ia32-msvc": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-0mYtjHS9ucAbcATycCNK9IGbk/cCe/ma7EmSLGZdsxn0A8cjrIyU04wDpVAD9Ni0fLUR9KTxdi053u0kherqjQ==",
  "cpu": [
    "ia32"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ]
},
"node_modules/@rollup/rollup-win32-x64-gnu": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-gK1iCEPfpoSG9wfbBihXxvBmi8ZfcwffYkEsC/Eih+iFENTAewvNcrEQ69LI0WY05pePHKLHH07nq5AILG0/HQQ==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ]
},
"node_modules/@rollup/rollup-win32-x64-msvc": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-X+zaP2x+j4RXGfbp/seSoRHWnPxAplDszisZxbYH5C/jTxFhCtDNDPGZb9LJyYps24wGxruPF7Y+sIXt9Gzw==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ]
},
"node_modules/@types/estree": {
  "version": "1.0.9",
  "resolved": "****",
  "integrity": "sha512-GhdPgy1el4/ImP05X05Uw4cw2/M93BCUmnEvWZNstLCzEKME4Fkk+YpoA50iHNQmoS7Cafb8Xa3Pya8m1Qrzeg==",
  "license": "MIT"
},
"node_modules/@vitejs/plugin-vue": {
  "version": "6.0.7",
  "resolved": "****",
  "integrity": "sha512-km+p+XdSz9Sxm5rqUbqcSfZYaAniKxWBj1KURL+Jr7UaPvvX7BmaWMDp69I5rrFDeQGyxAG7NXdc57vz+snhWg==",
  "license": "MIT",
}
```

```

"dependencies": {
  "@rolldown/pluginutils": "^1.0.1"
},
"engines": {
  "node": "^20.19.0 || >=22.12.0"
},
"peerDependencies": {
  "vite": "^5.0.0 || ^6.0.0 || ^7.0.0 || ^8.0.0",
  "vue": "^3.2.25"
}
},
"node_modules/@vitejs/plugin-vue/node_modules/@rolldown/pluginutils": {
  "version": "1.0.1",
  "resolved": "****",
  "integrity": "sha512-2j9bGt5Jh8hj+vPtgzPtL72j0yRxHAYumoo6TNfAjsLB04UtpSvPbPcDcBMxz7n+9CYB0c1GxQFxyRg2jimqGw==",
  "license": "MIT"
},
"node_modules/@vue/compiler-core": {
  "version": "3.5.38",
  "resolved": "****",
  "integrity": "sha512-s99aGxwYig9ErHbct27KXEGhrBYLRI6c4MwAgXEr0AbX9xiW37/uMa+XUD069zLz83dng8UUZ70CT0JrLrYrEQ==",
  "license": "MIT",
  "dependencies": {
    "@babel/parser": "^7.29.7",
    "@vue/shared": "3.5.38",
    "entities": "^7.0.1",
    "estree-walker": "^2.0.2",
    "source-map-js": "^1.2.1"
  }
},
"node_modules/@vue/compiler-dom": {
  "version": "3.5.38",
  "resolved": "****",
  "integrity": "sha512-JTqp25l8aFfJYF7/KmsXZjAXJz7T+SjmTJLoXVjHtc2BrSgSiW2n9Aem/cWq10Pe68A8JL06B3eVdhLP0H4TVw==",
  "license": "MIT",
  "dependencies": {
    "@vue/compiler-core": "3.5.38",
    "@vue/shared": "3.5.38"
  }
},
"node_modules/@vue/compiler-sfc": {
  "version": "3.5.38",
  "resolved": "****",
  "integrity": "sha512-DuA2GiZawSEW442iw/9+Fkol8hTgb4Ke5KkHmSry65QA7YuyMbIdy8p0XZRMvNwJdGRz307W8g1CSzdvS4nuNg==",
  "license": "MIT",
  "dependencies": {
    "@babel/parser": "^7.29.7",
    "@vue/compiler-core": "3.5.38",
    "@vue/compiler-dom": "3.5.38",
    "@vue/compiler-ssr": "3.5.38",
    "@vue/shared": "3.5.38",
    "estree-walker": "^2.0.2",
    "magic-string": "^0.30.21",
    "postcss": "^8.5.15",
    "source-map-js": "^1.2.1"
  }
},
"node_modules/@vue/compiler-ssr": {
  "version": "3.5.38",
  "resolved": "****",
  "integrity": "sha512-7s+W5Gc42FGxZMcwL8H5B29T8BJPMdB7KHFE+BbAuZ/iTEdTtv7z2XiMjiaUUw4w3ZcCEdHs36RuYJ2VA7bA==",
  "license": "MIT",
  "dependencies": {
    "@vue/compiler-dom": "3.5.38",
    "@vue/shared": "3.5.38"
  }
}

```

```
    },
    "node_modules/@vue/reactivity": {
      "version": "3.5.38",
      "resolved": "****",
      "integrity": "sha512-
pG6LV/NDNRbKiZcUjFFLAfjaL8mcv4DmR9avNcUw2gDHBzZneuS2TwCmp633ynzxz9YYKNeEPK2I8Wraqy2HUQ==",
      "license": "MIT",
      "dependencies": {
        "@vue/shared": "3.5.38"
      }
    },
    "node_modules/@vue/runtime-core": {
      "version": "3.5.38",
      "resolved": "****",
      "integrity": "sha512-
iyw8WVFF1CpCXncZY5Ei6rSd6oZr5DgEom//fUjRBRL56AXPD+s9ATvukRt77ZFTuYlnVA1bxY+dJB94tWVYw==",
      "license": "MIT",
      "dependencies": {
        "@vue/reactivity": "3.5.38",
        "@vue/shared": "3.5.38"
      }
    },
    "node_modules/@vue/runtime-dom": {
      "version": "3.5.38",
      "resolved": "****",
      "integrity": "sha512-
apX2wt9sdfDshS+a2xueFZLVpt0GkRJZSoPmrW/SA4yzXTznhfcMVW59gr7h4YQeY0vJhdJkk2rsIDwgFFgC5A==",
      "license": "MIT",
      "dependencies": {
        "@vue/reactivity": "3.5.38",
        "@vue/runtime-core": "3.5.38",
        "@vue/shared": "3.5.38",
        "csstype": "^3.2.3"
      }
    },
    "node_modules/@vue/server-renderer": {
      "version": "3.5.38",
      "resolved": "****",
      "integrity": "sha512-
vue8vbf2QLV4quHqzwmJy6dwfmRhP1J8l4wtZg60CL6VoKqcPY2oe7may3+1d9qfpedjK5PRLFqd5k3Isj9mUw==",
      "license": "MIT",
      "dependencies": {
        "@vue/compiler-ssr": "3.5.38",
        "@vue/shared": "3.5.38"
      },
      "peerDependencies": {
        "vue": "3.5.38"
      }
    },
    "node_modules/@vue/shared": {
      "version": "3.5.38",
      "resolved": "****",
      "integrity": "sha512-
FTW0AFZNaK5/mQqvGBwVfUlnLU38TiQn4+DQgIFUnrBBJQ1crMJ82yeGQLV5jyKFs08yRukpbuP7x+nRbH6aug==",
      "license": "MIT"
    },
    "node_modules/csstype": {
      "version": "3.2.3",
      "resolved": "****",
      "integrity": "sha512-
z1HGKcYy2xA8AGQfwrn0PAY+PB7X/GSj3UVJW9qKyn43xWa+gL5nXmU4qqLMRzWVLF8KusUX8T/0kCi0YpAIQ==",
      "license": "MIT"
    },
    "node_modules/entities": {
      "version": "7.0.1",
      "resolved": "****",
      "integrity": "sha512-
TwrgLOFUQTH994YUyl1yT4uyavY5nNB5muff+RtWaqNVCAK408b5ZnnbNAUEWLTcpum9w6arT70i1XdQ4UeOPA==",
      "license": "BSD-2-Clause",
      "engines": {
        "node": ">=0.12"
      }
    }
  }
}
```

```

    },
    "funding": {
      "url": "****"
    }
  },
  "node_modules/esbuild": {
    "version": "0.27.7",
    "resolved": "****",
    "integrity": "sha512-
IxpibTjyVnmrIQo5aqNpCgoACA/dTKLTlhMHihVHhdkxKyP01uBBthumTOrdHmcsk9uMonIWS0m4FljWzILh3w==",
    "hasInstallScript": true,
    "license": "MIT",
    "bin": {
      "esbuild": "bin/esbuild"
    },
    "engines": {
      "node": ">=18"
    },
    "optionalDependencies": {
      "@esbuild/aix-ppc64": "0.27.7",
      "@esbuild/android-arm": "0.27.7",
      "@esbuild/android-arm64": "0.27.7",
      "@esbuild/android-x64": "0.27.7",
      "@esbuild/darwin-arm64": "0.27.7",
      "@esbuild/darwin-x64": "0.27.7",
      "@esbuild/freebsd-arm64": "0.27.7",
      "@esbuild/freebsd-x64": "0.27.7",
      "@esbuild/linux-arm": "0.27.7",
      "@esbuild/linux-arm64": "0.27.7",
      "@esbuild/linux-ia32": "0.27.7",
      "@esbuild/linux-loong64": "0.27.7",
      "@esbuild/linux-mips64el": "0.27.7",
      "@esbuild/linux-ppc64": "0.27.7",
      "@esbuild/linux-riscv64": "0.27.7",
      "@esbuild/linux-s390x": "0.27.7",
      "@esbuild/linux-x64": "0.27.7",
      "@esbuild/netbsd-arm64": "0.27.7",
      "@esbuild/netbsd-x64": "0.27.7",
      "@esbuild/openbsd-arm64": "0.27.7",
      "@esbuild/openbsd-x64": "0.27.7",
      "@esbuild/openharmony-arm64": "0.27.7",
      "@esbuild/sunos-x64": "0.27.7",
      "@esbuild/win32-arm64": "0.27.7",
      "@esbuild/win32-ia32": "0.27.7",
      "@esbuild/win32-x64": "0.27.7"
    }
  },
  "node_modules/estree-walker": {
    "version": "2.0.2",
    "resolved": "****",
    "integrity": "sha512-
Rfkk/Mp/DL7JVje3u18FxFuJQLTNR2q6QfMSMB7AvCBx91NGj/ba3kCfza0f6dVDbw7YLRf/nDrn7pQrCCyQ/w==",
    "license": "MIT"
  },
  "node_modules/fdir": {
    "version": "6.5.0",
    "resolved": "****",
    "integrity": "sha512-
tIbYtZbuc0s0BRGqPJkshJUYdL+SDH7dVM8gjy+ERP3WAUJLEFJE+02kanyHtwjW0nwrKYBiwAmM0p4kLJAnXg==",
    "license": "MIT",
    "engines": {
      "node": ">=12.0.0"
    },
    "peerDependencies": {
      "picomatch": "^3 || ^4"
    },
    "peerDependenciesMeta": {
      "picomatch": {
        "optional": true
      }
    }
  }
}

```

```
    },
    "node_modules/fsevents": {
      "version": "2.3.3",
      "resolved": "****",
      "integrity": "sha512-5xoDfX+fl7faATnagmWPpbFtwh/R77WmMMqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzLhffQw==",
      "hasInstallScript": true,
      "license": "MIT",
      "optional": true,
      "os": [
        "darwin"
      ],
      "engines": {
        "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
      }
    },
    "node_modules/magic-string": {
      "version": "0.30.21",
      "resolved": "****",
      "integrity": "sha512-vd2F4YUyEXKGcLHoq+TEyCjxueSeHnFxyjNp80yg0XV4vUhnDer/LvvLqM/arB5bXQN5K2/3oinyCRyx8T2CQ==",
      "license": "MIT",
      "dependencies": {
        "@jridgewell/sourcemap-codec": "^1.5.5"
      }
    },
    "node_modules/nanoid": {
      "version": "3.3.12",
      "resolved": "****",
      "integrity": "sha512-ZB9RH/39ppq5Vu6Y+NmUaFhQR6pp+M2Xt76XBnEwDaGcVAqhLvxl3B2bKS5D3NH3QR76v3aSrKaF/Kiy7LEtQ==",
      "funding": [
        {
          "type": "github",
          "url": "****"
        }
      ],
      "license": "MIT",
      "bin": {
        "nanoid": "bin/nanoid.cjs"
      },
      "engines": {
        "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
      }
    },
    "node_modules/picocolors": {
      "version": "1.1.1",
      "resolved": "****",
      "integrity": "sha512-xceH2snhtb5M9liqDsmEw56le376mTZkEX/jEb/RxNFyegNul7eNsLCXP9FDj/Lcu0X8KEyMceP2ntpaHrDEVA==",
      "license": "ISC"
    },
    "node_modules/picomatch": {
      "version": "4.0.4",
      "resolved": "****",
      "integrity": "sha512-8P221e0850734824732fd2ddcc191634a83be3b0db52c34e228957e57fb8b8c000",
      "license": "MIT",
      "engines": {
        "node": ">=12"
      },
      "funding": {
        "url": "****"
      }
    },
    "node_modules/postcss": {
      "version": "8.5.15",
      "resolved": "****",
      "integrity": "sha512-0F5bK4CvX5kN9T9uMAfIctN1SfYctwph1MZ26J7v4TlL8XGfLhLZb8H7yeb54u3hELWl7vXnZjY8IU7Q==",
      "funding": [

```

```

    {
      "type": "opencollective",
      "url": "****"
    },
    {
      "type": "tidelift",
      "url": "****"
    },
    {
      "type": "github",
      "url": "****"
    }
  ],
  "license": "MIT",
  "dependencies": {
    "nanoid": "^3.3.12",
    "picocolors": "^1.1.1",
    "source-map-js": "^1.2.1"
  },
  "engines": {
    "node": "^10 || ^12 || >=14"
  }
},
"node_modules/rollup": {
  "version": "4.61.1",
  "resolved": "****",
  "integrity": "sha512-I4KW6iuRpuu2uHBLraZ1wNZeODP7LnRha+VJ9tNaYVaVgKhW0aI3h4RYnoRPeql0fLHm/Co55b7snEDc0f0JrA==",
  "license": "MIT",
  "dependencies": {
    "@types/estree": "1.0.9"
  },
  "bin": {
    "rollup": "dist/bin/rollup"
  },
  "engines": {
    "node": ">=18.0.0",
    "npm": ">=8.0.0"
  },
  "optionalDependencies": {
    "@rollup/rollup-android-arm-eabi": "4.61.1",
    "@rollup/rollup-android-arm64": "4.61.1",
    "@rollup/rollup-darwin-arm64": "4.61.1",
    "@rollup/rollup-darwin-x64": "4.61.1",
    "@rollup/rollup-freebsd-arm64": "4.61.1",
    "@rollup/rollup-freebsd-x64": "4.61.1",
    "@rollup/rollup-linux-arm-gnueabi": "4.61.1",
    "@rollup/rollup-linux-arm-musleabi": "4.61.1",
    "@rollup/rollup-linux-arm64-gnu": "4.61.1",
    "@rollup/rollup-linux-arm64-musl": "4.61.1",
    "@rollup/rollup-linux-loong64-gnu": "4.61.1",
    "@rollup/rollup-linux-loong64-musl": "4.61.1",
    "@rollup/rollup-linux-ppc64-gnu": "4.61.1",
    "@rollup/rollup-linux-ppc64-musl": "4.61.1",
    "@rollup/rollup-linux-riscv64-gnu": "4.61.1",
    "@rollup/rollup-linux-riscv64-musl": "4.61.1",
    "@rollup/rollup-linux-s390x-gnu": "4.61.1",
    "@rollup/rollup-linux-x64-gnu": "4.61.1",
    "@rollup/rollup-linux-x64-musl": "4.61.1",
    "@rollup/rollup-openbsd-x64": "4.61.1",
    "@rollup/rollup-openharmony-arm64": "4.61.1",
    "@rollup/rollup-win32-arm64-msvc": "4.61.1",
    "@rollup/rollup-win32-ia32-msvc": "4.61.1",
    "@rollup/rollup-win32-x64-gnu": "4.61.1",
    "@rollup/rollup-win32-x64-msvc": "4.61.1",
    "fsevents": "~2.3.2"
  }
},
"node_modules/source-map-js": {
  "version": "1.2.1",
  "resolved": "****"
}

```

```

    "integrity": "sha512-
UXWMKhLowVkb728IUtQPXxfYU+usdybtUrK/8uGE8CQMvrh0pwvzDBWj0QhSL7MQc7vIsISBG8VQ8+IDQxpFQA==",
    "license": "BSD-3-Clause",
    "engines": {
      "node": ">=0.10.0"
    }
  },
  "node_modules/tinyglobby": {
    "version": "0.2.17",
    "resolved": "****",
    "integrity": "sha512-
wXR/dYpcqKmfWpEdZjiKJ0wCNfndD0DMnrW/cYjVGttEkBfVgcLFHoNrLj47mj0Vici9yyNu65aLsgF4NQyTa2g==",
    "license": "MIT",
    "dependencies": {
      "fdir": "^6.5.0",
      "picomatch": "^4.0.4"
    },
    "engines": {
      "node": ">=12.0.0"
    },
    "funding": {
      "url": "****"
    }
  },
  "node_modules/typescript": {
    "version": "5.9.3",
    "resolved": "****",
    "integrity": "sha512-
jlllvZzPDinLr9eUt3J/t7V6FgNEw9QjvBPdysz9KfQDD41fQrC2Y4vKQdiaUpFT4bXlb1RHhLpp8wtm6M5TgSw==",
    "devOptional": true,
    "license": "Apache-2.0",
    "bin": {
      "tsc": "bin/tsc",
      "tsserver": "bin/tsserver"
    },
    "engines": {
      "node": ">=14.17"
    }
  },
  "node_modules/vite": {
    "version": "7.3.5",
    "resolved": "****",
    "integrity": "sha512-
Ku0aNhcnGFN2zIPGA7wRmzF+LJA1sea7rHq17aiJ++9LzY1WWG6Jpwqwe1KNbRVPIqHmr8GLYx7jbrQcN/7/ww==",
    "license": "MIT",
    "dependencies": {
      "esbuild": "^0.27.0",
      "fdir": "^6.5.0",
      "picomatch": "^4.0.3",
      "postcss": "^8.5.6",
      "rollup": "^4.43.0",
      "tinyglobby": "^0.2.15"
    },
    "bin": {
      "vite": "bin/vite.js"
    },
    "engines": {
      "node": "^20.19.0 || >=22.12.0"
    },
    "funding": {
      "url": "****"
    },
    "optionalDependencies": {
      "fsevents": "~2.3.3"
    },
    "peerDependencies": {
      "@types/node": "^20.19.0 || >=22.12.0",
      "jiti": ">=1.21.0",
      "less": "^4.0.0",
      "lightningcss": "^1.21.0",
      "sass": "^1.70.0",

```

```

    "sass-embedded": "^1.70.0",
    "stylus": ">=0.54.8",
    "sugarss": "^5.0.0",
    "terser": "^5.16.0",
    "tsx": "^4.8.1",
    "yaml": "^2.4.2"
  },
  "peerDependenciesMeta": {
    "@types/node": {
      "optional": true
    },
    "jiti": {
      "optional": true
    },
    "less": {
      "optional": true
    },
    "lightningcss": {
      "optional": true
    },
    "sass": {
      "optional": true
    },
    "sass-embedded": {
      "optional": true
    },
    "stylus": {
      "optional": true
    },
    "sugarss": {
      "optional": true
    },
    "terser": {
      "optional": true
    },
    "tsx": {
      "optional": true
    },
    "yaml": {
      "optional": true
    }
  }
},
"node_modules/vue": {
  "version": "3.5.38",
  "resolved": "****",
  "integrity": "sha512-
vAMKHfImQLYSy0C+PBue4s3ERZ2xGKfgZg5GXAsLInq1dyh2H78ILVP5sK0KPFVW4kv+0GCIvBEondcjpZp7A==",
  "license": "MIT",
  "dependencies": {
    "@vue/compiler-dom": "3.5.38",
    "@vue/compiler-sfc": "3.5.38",
    "@vue/runtime-dom": "3.5.38",
    "@vue/server-renderer": "3.5.38",
    "@vue/shared": "3.5.38"
  },
  "peerDependencies": {
    "typescript": "*"
  },
  "peerDependenciesMeta": {
    "typescript": {
      "optional": true
    }
  }
}
}
}
}

```

frontend/package.json

```
{
  "name": "railway-recognition-frontend",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@lucide/vue": "^1.17.0",
    "@mediapipe/tasks-vision": "^0.10.22-rc.20250304",
    "@vitejs/plugin-vue": "^6.0.0",
    "vite": "^7.2.0",
    "vue": "^3.5.0"
  },
  "devDependencies": {
    "typescript": "^5.9.0"
  }
}
```

frontend/src/api.ts

```
export type ImageTaskType = "detection" | "segmentation" | "pose" | "classification" | "caption";
export type ImageTaskStatus = "idle" | "pending" | "processing" | "success" | "failed";
export type ImageReviewStatus = "draft" | "pending_review" | "approved" | "rejected";

export type ImageAsset = {
  imageId: string;
  imageUrl: string;
  originalName: string;
  fileName: string;
  fileUrl: string;
  filePath?: string;
  mimeType?: string | null;
  fileSize: number;
  width?: number | null;
  height?: number | null;
  sessionId?: string;
  createdAt: string;
  updatedAt?: string;
  reviewStatus: ImageReviewStatus;
  submittedBy?: string | null;
  submittedAt?: string | null;
  reviewedBy?: string | null;
  reviewedAt?: string | null;
  reviewComment?: string;
  hasCurrentTaskResult: boolean;
  taskStatus: ImageTaskStatus;
};

export type ImageHistoryData = {
  records: ImageAsset[];
  total: number;
};

export type ImageTask = {
  taskId: string;
  imageId: string;
  taskType: ImageTaskType;
  status: ImageTaskStatus;
  createdAt?: string;
  updatedAt?: string;
};

export type ImageTaskResult = {
```

```

resultId?: string;
taskId: string;
imageId: string;
taskType: ImageTaskType;
status: ImageTaskStatus;
resultImageUrl?: string;
resultImagePath?: string;
resultJson?: Record<string, unknown> | null;
annotationJson?: Record<string, unknown> | null;
descriptionText?: string;
source?: "ai" | "manual" | "edited";
modelId?: string;
latestVersionId?: string | null;
latestVersionNo?: number | null;
createdAt?: string;
updatedAt?: string;
};

export type ImageTaskResultVersion = {
  versionId: string;
  resultId: string;
  taskId: string;
  imageId: string;
  taskType: ImageTaskType;
  versionNo: number;
  source: "ai" | "manual" | "edited";
  modelId?: string;
  resultImageUrl?: string;
  resultImagePath?: string;
  resultJson?: Record<string, unknown> | null;
  annotationJson?: Record<string, unknown> | null;
  descriptionText?: string;
  createdAt: string;
};

export type LabelConfig = {
  labelId: number;
  englishName: string;
  chineseName: string;
  description: string;
  createdAt: string;
  updatedAt: string;
};

export type VideoCaptionBatch = {
  batchId: string;
  name: string;
  sourceDir: string;
  outputDir: string;
  modelId: string;
  prompt: string;
  frameIntervalSeconds: number;
  status: "pending" | "processing" | "success" | "failed";
  totalVideos: number;
  totalFrames: number;
  metadata: Record<string, unknown>;
  createdAt: string;
  updatedAt: string;
};

export type VideoCaptionVideo = {
  videoId: string;
  batchId: string;
  filename: string;
  displayName: string;
  keywords: string[];
  sourcePath: string;
  frameDir: string;
  fps?: number | null;
  width?: number | null;
  height?: number | null;
};

```

```

    frameCount?: number | null;
    durationMs?: number | null;
    status: "pending" | "processing" | "success" | "failed";
    error: string;
    createdAt: string;
    updatedAt: string;
  };

export type VideoCaptionFrame = {
  frameId: string;
  batchId: string;
  videoId: string;
  frameIndex: number;
  timestampMs: number;
  imagePath: string;
  imageUrl: string;
  width?: number | null;
  height?: number | null;
  descriptionText: string;
  modelId: string;
  status: "pending" | "processing" | "success" | "failed";
  error: string;
  resultJson: Record<string, unknown>;
  createdAt: string;
  updatedAt: string;
};

export type VideoCaptionFrameData = {
  records: VideoCaptionFrame[];
  total: number;
};

export type ApiEnvelope<T> = {
  code: number;
  message: string;
  data: T;
};

export type AuthUser = {
  userId: string;
  username: string;
  displayName: string;
  role: string;
};

export class ApiRequestError extends Error {
  constructor(
    message: string,
    readonly status: number,
  ) {
    super(message);
    this.name = "ApiRequestError";
  }
}

const API_BASE_URL = (import.meta.env.VITE_API_BASE_URL ?? "").replace(/\/$/, "");

async function request<T>(path: string, options?: RequestInit): Promise<T> {
  const url = `${API_BASE_URL}${path}`;
  let response: Response;
  try {
    response = await fetch(url, { credentials: "include", ...options });
  } catch (error) {
    console.error("[API] Network error", {
      url,
      method: options?.method ?? "GET",
      error,
    });
    throw new Error("网络异常, 请检查后端服务或网络连接");
  }
  if (!response.ok) {

```

```

    let detail = response.statusText;
    try {
        const payload = await response.json();
        const rawDetail = payload.detail ?? payload.message ?? payload;
        detail = typeof rawDetail === "string" ? rawDetail : JSON.stringify(rawDetail);
    } catch {
        detail = await response.text();
    }
    console.error("[API] Request failed", {
        url,
        method: options?.method ?? "GET",
        status: response.status,
        detail,
    });
    throw new ApiRequestError(detail || `HTTP ${response.status}`, response.status);
}
return response.json() as Promise<T>;
}

async function apiData<T>(path: string, options?: RequestInit): Promise<T> {
    const payload = await request<ApiEnvelope<T>>(path, options);
    return payload.data;
}

export function getCurrentUser(): Promise<AuthUser | null> {
    return apiData<AuthUser | null>("/api/auth/me");
}

export function login(data: { username: string; password: string }): Promise<AuthUser> {
    return apiData<AuthUser>("/api/auth/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data),
    });
}

export function register(data: { username: string; displayName: string; password: string }): Promise<AuthUser> {
    return apiData<AuthUser>("/api/auth/register", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data),
    });
}

export function logout(): Promise<{ loggedOut: boolean }> {
    return apiData<{ loggedOut: boolean }>("/api/auth/logout", { method: "POST" });
}

export function listUsers(): Promise<AuthUser[]> {
    return apiData<AuthUser[]>("/api/auth/users");
}

export function updateUserRole(userId: string, role: "student" | "reviewer" | "admin"): Promise<AuthUser> {
    return apiData<AuthUser>(`/api/auth/users/${userId}/role`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ role }),
    });
}

export function uploadImage(file: File, taskType: ImageTaskType, sessionId: string): Promise<ImageAsset> {
    const form = new FormData();
    form.append("file", file);
    form.append("taskType", taskType);
    form.append("sessionId", sessionId);
    return apiData<ImageAsset>("/api/images/upload", { method: "POST", body: form });
}

export function listImages(params: {
    taskType?: ImageTaskType;
    page?: number;

```

```

    pageSize?: number;
    sessionId?: string;
  }): Promise<ImageHistoryData> {
    const query = new URLSearchParams();
    if (params.taskType) query.set("taskType", params.taskType);
    query.set("page", String(params.page ?? 1));
    query.set("pageSize", String(params.pageSize ?? 50));
    if (params.sessionId) query.set("sessionId", params.sessionId);
    return apiData<ImageHistoryData>(`/api/images/list?${query.toString()}`);
  }

export function createImageTask(data: {
  imageId: string;
  taskType: ImageTaskType;
  sessionId: string;
}): Promise<ImageTask> {
  return apiData<ImageTask>("/api/image-tasks/create", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function saveTaskResult(data: {
  taskId: string;
  imageId: string;
  taskType: ImageTaskType;
  resultImageUrl?: string;
  resultJson?: Record<string, unknown> | null;
  annotationJson?: Record<string, unknown> | null;
  descriptionText?: string;
  source?: "ai" | "manual" | "edited";
  modelId?: string;
}): Promise<{ resultId: string; versionId: string; versionNo: number }> {
  return apiData<{ resultId: string; versionId: string; versionNo: number }>("/api/image-tasks/result/save", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function getTaskResult(params: {
  imageId: string;
  taskType: ImageTaskType;
  sessionId: string;
}): Promise<ImageTaskResult | null> {
  const query = new URLSearchParams({
    imageId: params.imageId,
    taskType: params.taskType,
    sessionId: params.sessionId,
  });
  return apiData<ImageTaskResult | null>(`/api/image-tasks/result?${query.toString()}`);
}

export function listTaskResultVersions(params: {
  imageId: string;
  taskType: ImageTaskType;
  sessionId: string;
}): Promise<ImageTaskResultVersion[]> {
  const query = new URLSearchParams({
    imageId: params.imageId,
    taskType: params.taskType,
    sessionId: params.sessionId,
  });
  return apiData<ImageTaskResultVersion[]>(`/api/image-tasks/result/versions?${query.toString()}`);
}

export function getTaskResultVersion(versionId: string): Promise<ImageTaskResultVersion> {
  return apiData<ImageTaskResultVersion>(`/api/image-tasks/result/version/${versionId}`);
}

```

```

export function restoreTaskResultVersion(versionId: string): Promise<ImageTaskResult> {
  return apiData<ImageTaskResult>(`/api/image-tasks/result/version/${versionId}/restore`, {
    method: "POST",
  });
}

export function updateAnnotation(data: {
  taskId: string;
  imageId: string;
  taskType: ImageTaskType;
  annotationJson: Record<string, unknown>;
}): Promise<ImageTaskResult> {
  return apiData<ImageTaskResult>("/api/image-tasks/annotation", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function submitTaskResult(taskId: string): Promise<ImageTaskResult> {
  return apiData<ImageTaskResult>(`/api/image-tasks/result/${taskId}/submit`, { method: "POST" });
}

export function reviewTaskResult(taskId: string, status: "approved" | "rejected", comment: string):
Promise<ImageTaskResult> {
  return apiData<ImageTaskResult>(`/api/image-tasks/result/${taskId}/review`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ status, comment }),
  });
}

export function inferImageTask(data: {
  imageId: string;
  taskType: ImageTaskType;
  sessionId: string;
  modelName?: string;
}): Promise<ImageTaskResult> {
  return apiData<ImageTaskResult>("/api/ai/infer", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function listLabels(): Promise<LabelConfig[]> {
  return apiData<LabelConfig[]>("/api/labels");
}

export function createLabel(data: {
  englishName: string;
  chineseName: string;
  description: string;
  copyFromLabelId?: number | null;
}): Promise<LabelConfig> {
  return apiData<LabelConfig>("/api/labels", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function updateLabel(
  labelId: number,
  data: {
    englishName: string;
    chineseName: string;
    description: string;
  },
): Promise<LabelConfig> {
  return apiData<LabelConfig>(`/api/labels/${labelId}`, {

```

```

    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
}

export function copyLabel(labelId: number): Promise<LabelConfig> {
  return apiData<LabelConfig>(`/api/labels/${labelId}/copy`, { method: "POST" });
}

export function deleteLabel(labelId: number): Promise<{ deleted: boolean }> {
  return apiData<{ deleted: boolean }>(`/api/labels/${labelId}`, { method: "DELETE" });
}

export function listVideoCaptionBatches(): Promise<VideoCaptionBatch[]> {
  return apiData<VideoCaptionBatch[]>("/api/video-captions/batches");
}

export function listVideoCaptionVideos(batchId: string): Promise<VideoCaptionVideo[]> {
  return apiData<VideoCaptionVideo[]>(`/api/video-captions/batches/${batchId}/videos`);
}

export function listVideoCaptionFrames(params: {
  videoId: string;
  page?: number;
  pageSize?: number;
  query?: string;
}): Promise<VideoCaptionFrameData> {
  const query = new URLSearchParams({
    page: String(params.page ?? 1),
    pageSize: String(params.pageSize ?? 200),
  });
  if (params.query) query.set("query", params.query);
  return apiData<VideoCaptionFrameData>(`/api/video-captions/videos/${params.videoId}/frames?${query.toString()}`);
}

export function assetUrl(path?: string | null): string {
  if (!path) return "";
  if (/^https?:\/\//.test(path)) return path;
  return `${API_BASE_URL}${path}`;
}

```

frontend/src/App.vue

```

<script setup lang="ts">
import { computed, onBeforeUnmount, onMounted, reactive, ref, watch } from "vue";
import {
  Boxes,
  Camera,
  Check,
  ChevronLeft,
  ChevronRight,
  Copy,
  Download,
  Edit3,
  Film,
  Image as ImageIcon,
  LogOut,
  Loader2,
  Maximize2,
  MousePointer2,
  Move,
  Pentagon,
  Plus,
  RefreshCw,
  RotateCcw,
  Save,
  Search,
  Settings,

```

```

    Sparkles,
    Tags,
    Trash2,
    Upload,
    UserRound,
    Volume2,
    Waypoints,
    ZoomIn,
  } from "@lucide/vue";
import LiveGesturePage from "../components/LiveGesturePage.vue";
import {
  ApiRequestError,
  type AuthUser,
  type ImageAsset,
  type ImageReviewStatus,
  type ImageTaskResult,
  type ImageTaskResultVersion,
  type ImageTaskStatus,
  type ImageTaskType,
  type LabelConfig,
  type VideoCaptionBatch,
  type VideoCaptionFrame,
  type VideoCaptionVideo,
  assetUrl,
  createImageTask,
  createLabel,
  copyLabel,
  deleteLabel,
  getTaskResult,
  getTaskResultVersion,
  inferImageTask,
  getCurrentUser,
  listImages,
  listLabels,
  listUsers,
  listTaskResultVersions,
  listVideoCaptionBatches,
  listVideoCaptionFrames,
  listVideoCaptionVideos,
  login,
  logout,
  register,
  restoreTaskResultVersion,
  reviewTaskResult,
  saveTaskResult,
  submitTaskResult,
  updateAnnotation,
  updateLabel,
  updateUserRole,
  uploadImage,
} from "../api";

type PageMode = "workspace" | "videoCaptions" | "liveGesture" | "settings";
type AnnotationTool = "select" | "pan" | "box" | "polygon" | "line";
type ShapeType = "box" | "polygon" | "line";
type ResizeSide = "history" | "thumbnail";
type AuthMode = "login" | "register";

type TaskConfig = {
  type: ImageTaskType;
  title: string;
  description: string;
  models: string[];
};

type TaskState = {
  currentImageId: string | null;
  imageUrl: string;
  imageInfo: ImageAsset | null;
  taskId: string | null;
  resultImageUrl: string;

```

```

resultJson: Record<string, unknown> | null;
annotationJson: Record<string, unknown> | null;
descriptionText: string;
status: ImageTaskStatus;
versions: ImageTaskResultVersion[];
activeVersionId: string | null;
latestVersionNo: number | null;
isViewingHistory: boolean;
reviewStatus: ImageReviewStatus;
reviewComment: string;
submittedAt: string | null;
reviewedAt: string | null;
};

type Point = { x: number; y: number };
type PoseKeypoint = Point & { name: string; score: number };
type PoseSkeletonLine = {
  name: string;
  x1: number;
  y1: number;
  x2: number;
  y2: number;
  score: number;
};

type AnnotationShape = {
  id: string;
  type: ShapeType;
  labelId: number | null;
  label: string;
  points: Point[];
  score?: number;
  source: "ai" | "manual" | "edited";
};

const taskConfigs: TaskConfig[] = [
  { type: "detection", title: "目标检测", description: "检测框、类别与置信度", models: ["yolo11n", "yolov8n"] },
  { type: "segmentation", title: "实例分割", description: "区域 mask、轮廓与类别", models: ["yolo11n-seg"] },
  { type: "pose", title: "姿态识别", description: "关键点与骨架连线", models: ["yolo11n-pose"] },
  { type: "classification", title: "图像分类", description: "分类标签与概率", models: ["yolo11n-cls"] },
  {
    type: "caption",
    title: "图像描述",
    description: "生成图片描述文本",
    models: ["deepseek-vl2-tiny", "blip-image-captioning-base", "yolo-cls-fallback"],
  },
];

const poseSkeletonPairs = [
  ["left_shoulder", "right_shoulder"],
  ["left_shoulder", "left_elbow"],
  ["left_elbow", "left_wrist"],
  ["right_shoulder", "right_elbow"],
  ["right_elbow", "right_wrist"],
  ["left_shoulder", "left_hip"],
  ["right_shoulder", "right_hip"],
  ["left_hip", "right_hip"],
  ["left_hip", "left_knee"],
  ["left_knee", "left_ankle"],
  ["right_hip", "right_knee"],
  ["right_knee", "right_ankle"],
  ["nose", "left_eye"],
  ["nose", "right_eye"],
  ["left_eye", "left_ear"],
  ["right_eye", "right_ear"],
];

const currentPage = ref<PageMode>("workspace");
const currentTaskType = ref<ImageTaskType>("detection");
const selectedModelByTask = reactive<Record<ImageTaskType, string>>({
  detection: "yolo11n",
  segmentation: "yolo11n-seg",
});

```

```

    pose: "yolo11n-pose",
    classification: "yolo11n-cls",
    caption: "deepseek-vl2-tiny",
  });

const imageHistoryList = ref<ImageAsset[]>([]);
const labels = ref<LabelConfig[]>([]);
const activeRequestCount = ref(0);
const loading = computed(() => activeRequestCount.value > 0);
const processingOverlayText = ref("");
const message = ref("");
const errorDialog = ref("");
const historyCollapsed = ref(false);
const thumbnailCollapsed = ref(false);
const workspaceBodyRef = ref<HTMLElement | null>(null);
const panelLayout = reactive({
  historyWidth: 280,
  thumbnailWidth: 300,
  resizing: null as ResizeSide | null,
});
const originalPreview = ref<ImageAsset | null>(null);
const activeTool = ref<AnnotationTool>("select");
const selectedShapeId = ref<string | null>(null);
const activePresetLabelId = ref<number | null>(null);
const draftShape = ref<AnnotationShape | null>(null);
const polygonDraft = ref<Point[]>([]);
const drawing = reactive({ active: false, mode: "" as "" | "draw" | "move" | "resize", shapeId: "", start: null as Point | null });

const labelForm = reactive({
  englishName: "",
  chineseName: "",
  description: "",
});
const editingLabelId = ref<number | null>(null);
const editingLabel = reactive({ englishName: "", chineseName: "", description: "" });
const currentUser = ref<AuthUser | null>(null);
const authReady = ref(false);
const authMode = ref<AuthMode>("login");
const authForm = reactive({ username: "", displayName: "", password: "****" });
const users = ref<AuthUser[]>([]);
const reviewCommentInput = ref("");
const videoCaptionBatches = ref<VideoCaptionBatch[]>([]);
const videoCaptionVideos = ref<VideoCaptionVideo[]>([]);
const videoCaptionFrames = ref<VideoCaptionFrame[]>([]);
const selectedVideoCaptionBatchId = ref("");
const selectedVideoCaptionVideoId = ref("");
const selectedVideoCaptionFrameId = ref("");
const videoCaptionQuery = ref("");
const videoCaptionTotal = ref(0);

const defaultSessionId = "default";
const taskStates = reactive<Record<ImageTaskType, TaskState>>({
  detection: createDefaultTaskState(),
  segmentation: createDefaultTaskState(),
  pose: createDefaultTaskState(),
  classification: createDefaultTaskState(),
  caption: createDefaultTaskState(),
});

const currentState = computed(() => taskStates[currentTaskType.value]);
const currentConfig = computed(() => taskConfigs.find((item) => item.type === currentTaskType.value) ?? taskConfigs[0]);
const currentImageUrl = computed(() => assetUrl(currentState.value.imageUrl));
const workspaceGridStyle = computed(() => ({
  gridTemplateColumns: `${historyCollapsed.value ? 48 : panelLayout.historyWidth}px 6px minmax(360px, 1fr) 6px ${
    thumbnailCollapsed.value ? 48 : panelLayout.thumbnailWidth
  }px`,
}));
const captionIsTransferring = computed(() => currentTaskType.value === "caption" && (loading.value || currentState.value.status === "processing"));

```

```

const view = reactive({ zoom: 1, x: 0, y: 0, dragging: false, startX: 0, startY: 0, originX: 0, originY: 0 });

const annotationShapes = computed(() => readShapes(currentState.value.annotationJson));
const selectedShape = computed(() => annotationShapes.value.find((shape) => shape.id === selectedShapeId.value) ?? null);
const activePresetLabel = computed(() => labels.value.find((label) => label.labelId === activePresetLabelId.value) ?? labels.value[0] ?? null);
const presetLabels = computed(() => labels.value.map((label, index) => ({
  label,
  shortcut: labelShortcut(index),
})),
);
const selectedLabelId = computed({
  get: () => selectedShape.value?.labelId ?? "",
  set: (value: string | number) => {
    if (!selectedShape.value) return;
    if (value === "") {
      updateShape(selectedShape.value.id, {
        labelId: null,
        label: selectedShape.value.label,
        source: "edited",
      });
      return;
    }
    const label = labels.value.find((item) => item.labelId === Number(value));
    if (label) selectPresetLabel(label);
  },
});
const selectedVideoCaptionBatch = computed(
  () => videoCaptionBatches.value.find((batch) => batch.batchId === selectedVideoCaptionBatchId.value) ?? null,
);
const selectedVideoCaptionVideo = computed(
  () => videoCaptionVideos.value.find((video) => video.videoId === selectedVideoCaptionVideoId.value) ?? null,
);
const selectedVideoCaptionFrame = computed(
  () => videoCaptionFrames.value.find((frame) => frame.frameId === selectedVideoCaptionFrameId.value) ?? null,
);
const canReview = computed(() => currentUser.value?.role === "reviewer" || currentUser.value?.role === "admin");
const isAdmin = computed(() => currentUser.value?.role === "admin");
const resultLocked = computed(() => currentState.value.reviewStatus === "approved");
const editingDisabled = computed(() => resultLocked.value || currentUser.value?.role === "reviewer");

watch(selectedShape, (shape) => {
  if (shape?.labelId !== null && shape?.labelId !== undefined && labels.value.some((label) => label.labelId === shape.labelId)) {
    activePresetLabelId.value = shape.labelId;
  }
});

onMounted(async () => {
  window.addEventListener("keydown", handleGlobalKeydown);
  await restoreLoginState();
});

onBeforeUnmount(() => {
  window.removeEventListener("keydown", handleGlobalKeydown);
  stopPanelResize();
});

async function restoreLoginState() {
  await runBusy(async () => {
    currentUser.value = await getCurrentUser();
    if (currentUser.value) await initializeAuthenticatedWorkspace();
  }, "登录状态检查失败");
  authReady.value = true;
}

async function initializeAuthenticatedWorkspace() {
  await Promise.all([loadLabels(), loadImageHistory()]);
  const first = imageHistoryList.value[0];

```

```

    if (first) await selectImage(first);
}

async function submitAuth() {
    await runBusy(async () => {
        currentUser.value =
            authMode.value === "login"
                ? await login({ username: authForm.username.trim(), password: authForm.password })
                : await register({
                    username: authForm.username.trim(),
                    displayName: authForm.displayName.trim(),
                    password: authForm.password,
                });
        authForm.password = "****";
        await initializeAuthenticatedWorkspace();
        message.value = `欢迎, ${currentUser.value.displayName}`;
    }, authMode.value === "login" ? "登录失败" : "注册失败");
}

async function signOut() {
    await runBusy(async () => {
        await logout();
        currentUser.value = null;
        imageHistoryList.value = [];
        labels.value = [];
        for (const state of Object.values(taskStates)) Object.assign(state, createDefaultTaskState());
        authForm.password = "****";
        message.value = "已退出登录";
    }, "退出登录失败");
}

async function setTaskType(taskType: ImageTaskType) {
    currentTaskType.value = taskType;
    activeTool.value = "select";
    selectedShapeId.value = null;
    message.value = "";
    await loadImageHistory();
    const state = currentState.value;
    if (!state.currentImageId && imageHistoryList.value[0]) await selectImage(imageHistoryList.value[0]);
}

async function setPage(page: PageMode) {
    currentPage.value = page;
    message.value = "";
    if (page === "videoCaptions" && videoCaptionBatches.value.length === 0) {
        await loadVideoCaptionBatches();
    }
    if (page === "settings" && isAdmin.value) await loadUsers();
}

async function handleUpload(event: Event) {
    const input = event.target as HTMLInputElement;
    const file = input.files?.[0];
    input.value = "";
    if (!file) return;
    const error = validateImage(file);
    if (error) {
        message.value = error;
        errorDialog.value = error;
        return;
    }
    await runBusy(async () => {
        const image = await uploadImage(file, currentTaskType.value, defaultSessionId);
        await bindImageToCurrentTask(image);
        await loadImageHistory();
        message.value = "上传完成, 原图已保存";
    }, "图片上传失败, 请稍后重试");
}

async function loadImageHistory() {
    await runBusy(async () => {

```

```

const records: ImageAsset[] = [];
let page = 1;
let total = 0;
do {
  const data = await listImages({ taskType: currentTaskType.value, page, pageSize: 200 });
  records.push(...data.records);
  total = data.total;
  page += 1;
} while (records.length < total);
imageHistoryList.value = records;
}, "历史图片加载失败");
}

async function selectImage(image: ImageAsset) {
  await runBusy(async () => {
    await bindImageToCurrentTask(image);
    const result = await getTaskResult({
      imageId: image.imageId,
      taskType: currentTaskType.value,
      sessionId: image.sessionId || defaultSessionId,
    });
    applyTaskResult(result);
    await loadCurrentVersions();
    message.value = result ? "任务结果已加载" : "当前图片暂无该任务结果";
    resetView();
  }, "图片或任务结果加载失败");
}

async function runCurrentTask() {
  if (!currentState.value.currentImageId) {
    message.value = "请先上传或选择图片";
    return;
  }
  const taskTitle = currentConfig.value.title;
  const modelName = selectedModelByTask[currentTaskType.value];
  await runBusy(async () => {
    processingOverlayText.value = `${taskTitle}正在处理中, 请稍候`;
    currentState.value.status = "processing";
    const result = await inferImageTask({
      imageId: currentState.value.currentImageId,
      taskType: currentTaskType.value,
      sessionId: currentImageSessionId(),
      modelName,
    });
    applyTaskResult(result);
    await loadCurrentVersions();
    await loadImageHistory();
    message.value = "AI 结果已生成并保存, 可继续人工校验";
  }, "AI 任务执行失败");
}

async function saveCurrentResult() {
  const state = currentState.value;
  if (!state.currentImageId) {
    message.value = "请先上传或选择图片";
    return;
  }
  await runBusy(async () => {
    const task = state.taskId
      ? { taskId: state.taskId }
      : await createImageTask({
          imageId: state.currentImageId!,
          taskType: currentTaskType.value,
          sessionId: currentImageSessionId(),
        });
    state.taskId = task.taskId;
    ensureAnnotationJson();
    await saveTaskResult({
      taskId: state.taskId,
      imageId: state.currentImageId,
      taskType: currentTaskType.value,
    });
  });
}

```

```

        resultImageUrl: state.resultImageUrl,
        resultJson: state.resultJson,
        annotationJson: state.annotationJson,
        descriptionText: state.descriptionText,
        source: "edited",
        modelId: selectedModelByTask[currentTaskType.value],
    });
    await loadCurrentVersions();
    await loadImageHistory();
    state.reviewStatus = "draft";
    state.reviewComment = "";
    state.reviewedAt = null;
    message.value = "当前任务结果已保存";
  }, "保存当前结果失败");
}

```

```

async function saveAnnotationSnapshot() {
  const state = currentState.value;
  if (!state.currentImageId) {
    message.value = "请先上传或选择图片";
    return;
  }
  await runBusy(async () => {
    if (!state.taskId) {
      const task = await createImageTask({
        imageId: state.currentImageId!,
        taskType: currentTaskType.value,
        sessionId: currentImageSessionId(),
      });
      state.taskId = task.taskId;
    }
    ensureAnnotationJson();
    const result = await updateAnnotation({
      taskId: state.taskId!,
      imageId: state.currentImageId!,
      taskType: currentTaskType.value,
      annotationJson: state.annotationJson!,
    });
    applyTaskResult(result);
    await saveTaskResult({
      taskId: state.taskId!,
      imageId: state.currentImageId!,
      taskType: currentTaskType.value,
      resultImageUrl: state.resultImageUrl,
      resultJson: state.resultJson,
      annotationJson: state.annotationJson,
      descriptionText: state.descriptionText,
      source: "edited",
      modelId: selectedModelByTask[currentTaskType.value],
    });
    await loadCurrentVersions();
    message.value = "标注已保存";
  }, "标注保存失败");
}

```

```

function applyTaskResult(result: ImageTaskResult | null) {
  const state = currentState.value;
  if (!result) {
    state.taskId = null;
    state.resultImageUrl = "";
    state.resultJson = null;
    state.annotationJson = emptyAnnotation();
    state.descriptionText = "";
    state.status = "idle";
    state.activeVersionId = null;
    state.latestVersionNo = null;
    state.isViewingHistory = false;
    state.reviewStatus = "draft";
    state.reviewComment = "";
    state.submittedAt = null;
    state.reviewedAt = null;
  }
}

```

```

    return;
  }
  state.taskId = result.taskId;
  state.resultImageUrl = result.resultImageUrl || "";
  state.resultJson = (result.resultJson as Record<string, unknown> | null) ?? null;
  state.annotationJson = normalizeAnnotation(result.annotationJson as Record<string, unknown> | null,
state.resultJson);
  state.descriptionText = result.descriptionText || "";
  state.status = result.status || "success";
  state.activeVersionId = result.latestVersionId || null;
  state.latestVersionNo = result.latestVersionNo || null;
  state.isViewingHistory = false;
  state.reviewStatus = result.reviewStatus || "draft";
  state.reviewComment = result.reviewComment || "";
  state.submittedAt = result.submittedAt || null;
  state.reviewedAt = result.reviewedAt || null;
  reviewCommentInput.value = result.reviewComment || "";
  selectedShapeId.value = null;
}

async function loadCurrentVersions() {
  const state = currentState.value;
  if (!state.currentImageId) {
    state.versions = [];
    return;
  }
  state.versions = await listTaskResultVersions({
    imageId: state.currentImageId,
    taskType: currentTaskType.value,
    sessionId: currentImageSessionId(),
  });
  if (!state.activeVersionId && state.versions[0]) {
    state.activeVersionId = state.versions[0].versionId;
    state.latestVersionNo = state.versions[0].versionNo;
  }
}

async function viewVersion(versionId: string) {
  await runBusy(async () => {
    const version = await getTaskResultVersion(versionId);
    const state = currentState.value;
    state.taskId = version.taskId;
    state.resultImageUrl = version.resultImageUrl || "";
    state.resultJson = (version.resultJson as Record<string, unknown> | null) ?? null;
    state.annotationJson = normalizeAnnotation(version.annotationJson as Record<string, unknown> | null,
state.resultJson);
    state.descriptionText = version.descriptionText || "";
    state.status = "success";
    state.activeVersionId = version.versionId;
    state.isViewingHistory = state.latestVersionNo !== version.versionNo;
    selectedShapeId.value = null;
    message.value = `正在查看历史版本 v${version.versionNo}`;
  }, "历史版本加载失败");
}

async function restoreVersion(versionId: string) {
  await runBusy(async () => {
    const result = await restoreTaskResultVersion(versionId);
    applyTaskResult(result);
    await loadCurrentVersions();
    await loadImageHistory();
    message.value = `已恢复为新版本 v${currentState.value.latestVersionNo || ""}`;
  }, "恢复历史版本失败");
}

async function bindImageToCurrentTask(image: ImageAsset) {
  const state = currentState.value;
  state.currentImageId = image.imageId;
  state.imageUrl = image.imageUrl || image.fileUrl;
  state.imageInfo = image;
  state.resultImageUrl = "";
}

```

```

state.resultJson = null;
state.annotationJson = emptyAnnotation();
state.descriptionText = "";
state.status = "idle";
state.versions = [];
state.activeVersionId = null;
state.latestVersionNo = null;
state.isViewingHistory = false;
selectedShapeId.value = null;
}

async function loadLabels() {
  await runBusy(async () => {
    labels.value = await listLabels();
    if (!labels.value.some((label) => label.labelId === activePresetLabelId.value)) {
      activePresetLabelId.value = labels.value[0]?.labelId ?? null;
    }
  }, "标签加载失败");
}

async function loadUsers() {
  if (!isAdmin.value) return;
  await runBusy(async () => {
    users.value = await listUsers();
  }, "用户列表加载失败");
}

async function changeUserRole(userId: string, role: "student" | "reviewer" | "admin") {
  await runBusy(async () => {
    await updateUserRole(userId, role);
    await loadUsers();
    message.value = "用户角色已更新";
  }, "用户角色更新失败");
}

function handleRoleChange(userId: string, event: Event) {
  const role = (event.target as HTMLSelectElement).value;
  if (role === "student" || role === "reviewer" || role === "admin") void changeUserRole(userId, role);
}

async function submitForReview() {
  if (!currentState.value.taskId) return;
  await runBusy(async () => {
    applyTaskResult(await submitTaskResult(currentState.value.taskId!));
    message.value = "已提交审核";
  }, "提交审核失败");
}

async function reviewCurrentResult(status: "approved" | "rejected") {
  if (!currentState.value.taskId) return;
  await runBusy(async () => {
    applyTaskResult(await reviewTaskResult(currentState.value.taskId!, status, reviewCommentInput.value));
    message.value = status === "approved" ? "审核已通过，结果已锁定" : "已驳回并返回修改";
  }, "审核操作失败");
}

async function loadVideoCaptionBatches() {
  await runBusy(async () => {
    videoCaptionBatches.value = await listVideoCaptionBatches();
    if (!selectedVideoCaptionBatchId.value && videoCaptionBatches.value[0]) {
      selectedVideoCaptionBatchId.value = videoCaptionBatches.value[0].batchId;
    }
    if (selectedVideoCaptionBatchId.value) await loadVideoCaptionVideos();
    message.value = "视频解帧批次已加载";
  }, "视频解帧批次加载失败");
}

async function loadVideoCaptionVideos() {
  if (!selectedVideoCaptionBatchId.value) {
    videoCaptionVideos.value = [];
    videoCaptionFrames.value = [];
  }
}

```

```

        selectedVideoCaptionVideoId.value = "";
        selectedVideoCaptionFrameId.value = "";
        return;
    }
    videoCaptionVideos.value = await listVideoCaptionVideos(selectedVideoCaptionBatchId.value);
    if (!videoCaptionVideos.value.some((video) => video.videoId === selectedVideoCaptionVideoId.value)) {
        selectedVideoCaptionVideoId.value = videoCaptionVideos.value[0]?.videoId ?? "";
    }
    await loadVideoCaptionFrames();
}

async function selectVideoCaptionBatch(batchId: string) {
    selectedVideoCaptionBatchId.value = batchId;
    selectedVideoCaptionVideoId.value = "";
    selectedVideoCaptionFrameId.value = "";
    await runBusy(loadVideoCaptionVideos, "视频列表加载失败");
}

async function selectVideoCaptionVideo(videoId: string) {
    selectedVideoCaptionVideoId.value = videoId;
    selectedVideoCaptionFrameId.value = "";
    await runBusy(loadVideoCaptionFrames, "视频帧加载失败");
}

async function loadVideoCaptionFrames() {
    if (!selectedVideoCaptionVideoId.value) {
        videoCaptionFrames.value = [];
        selectedVideoCaptionFrameId.value = "";
        videoCaptionTotal.value = 0;
        return;
    }
    const data = await listVideoCaptionFrames({
        videoId: selectedVideoCaptionVideoId.value,
        page: 1,
        pageSize: 500,
        query: videoCaptionQuery.value.trim(),
    });
    videoCaptionFrames.value = data.records;
    videoCaptionTotal.value = data.total;
    if (!videoCaptionFrames.value.some((frame) => frame.frameId === selectedVideoCaptionFrameId.value)) {
        selectedVideoCaptionFrameId.value = videoCaptionFrames.value[0]?.frameId ?? "";
    }
}

async function searchVideoCaptionFrames() {
    await runBusy(loadVideoCaptionFrames, "帧描述检索失败");
}

function selectVideoCaptionFrame(frameId: string) {
    selectedVideoCaptionFrameId.value = frameId;
}

async function submitLabel() {
    await runBusy(async () => {
        await createLabel({
            englishName: labelForm.englishName.trim(),
            chineseName: labelForm.chineseName.trim(),
            description: labelForm.description.trim(),
        });
        labelForm.englishName = "";
        labelForm.chineseName = "";
        labelForm.description = "";
        await loadLabels();
        message.value = "标签已新增";
    }, "新增标签失败");
}

function startEditLabel(label: LabelConfig) {
    editingLabelId.value = label.labelId;
    editingLabel.englishName = label.englishName;
    editingLabel.chineseName = label.chineseName;

```

```

    editingLabel.description = label.description;
}

async function duplicateLabel(labelId: number) {
    await runBusy(async () => {
        await copyLabel(labelId);
        await loadLabels();
        message.value = "标签已复制";
    }, "复制标签失败");
}

async function submitEditLabel(labelId: number) {
    await runBusy(async () => {
        await updateLabel(labelId, {
            englishName: editingLabel.englishName.trim(),
            chineseName: editingLabel.chineseName.trim(),
            description: editingLabel.description.trim(),
        });
        editingLabelId.value = null;
        await loadLabels();
        message.value = "标签已更新";
    }, "更新标签失败");
}

async function removeLabel(labelId: number) {
    await runBusy(async () => {
        await deleteLabel(labelId);
        if (selectedShape.value?.labelId === labelId) selectedShapeId.value = null;
        await loadLabels();
        message.value = "标签已删除";
    }, "删除标签失败");
}

function setTool(tool: AnnotationTool) {
    if (editingDisabled.value && ["box", "polygon", "line"].includes(tool)) {
        message.value = resultLocked.value ? "审核通过的结果已锁定" : "审核员只能查看和审核任务";
        return;
    }
    activeTool.value = tool;
    draftShape.value = null;
    polygonDraft.value = [];
    drawing.active = false;
}

function onStagePointerDown(event: PointerEvent) {
    if (activeTool.value !== "pan" || !currentState.value.imageUrl) return;
    view.dragging = true;
    view.startX = event.clientX;
    view.startY = event.clientY;
    view.originX = view.x;
    view.originY = view.y;
    (event.currentTarget as HTMLElement).setPointerCapture(event.pointerId);
}

function onStagePointerMove(event: PointerEvent) {
    if (!view.dragging) return;
    view.x = view.originX + event.clientX - view.startX;
    view.y = view.originY + event.clientY - view.startY;
}

function stopStageDrag() {
    view.dragging = false;
}

function startPanelResize(event: PointerEvent, side: ResizeSide) {
    if ((side === "history" && historyCollapsed.value) || (side === "thumbnail" && thumbnailCollapsed.value)) return;
    event.preventDefault();
    panelLayout.resizing = side;
    window.addEventListener("pointermove", onPanelResize);
    window.addEventListener("pointerup", stopPanelResize);
}

```

```

function onPanelResize(event: PointerEvent) {
  const workspace = workspaceBodyRef.value;
  if (!workspace || !panelLayout.resizing) return;
  const rect = workspace.getBoundingClientRect();
  if (panelLayout.resizing === "history") {
    panelLayout.historyWidth = clampPanelWidth(event.clientX - rect.left - 12);
    return;
  }
  panelLayout.thumbnailWidth = clampPanelWidth(rect.right - event.clientX - 12);
}

function stopPanelResize() {
  panelLayout.resizing = null;
  window.removeEventListener("pointermove", onPanelResize);
  window.removeEventListener("pointerup", stopPanelResize);
}

function clampPanelWidth(value: number) {
  return Math.min(520, Math.max(180, Math.round(value)));
}

function onOverlayPointerDown(event: PointerEvent) {
  if (!currentState.value.imageInfo || activeTool.value === "pan") return;
  const point = svgPoint(event);
  if (!point) return;
  if (activeTool.value === "select") {
    selectedShapeId.value = null;
    return;
  }
  if (activeTool.value === "polygon") {
    polygonDraft.value = [...polygonDraft.value, point];
    return;
  }
  draftShape.value = makeShape(activeTool.value === "line" ? "line" : "box", [point, point]);
  drawing.active = true;
  drawing.mode = "draw";
  drawing.start = point;
}

function onOverlayPointerMove(event: PointerEvent) {
  const point = svgPoint(event);
  if (!point) return;
  if (draftShape.value && drawing.mode === "draw") {
    draftShape.value = { ...draftShape.value, points: [draftShape.value.points[0], point] };
    return;
  }
  if (drawing.mode === "move" && drawing.start) moveSelectedShape(point);
  if (drawing.mode === "resize" && drawing.start) resizeSelectedBox(point);
}

function onOverlayPointerUp() {
  if (draftShape.value && drawing.mode === "draw") {
    const shape = normalizeShapePoints(draftShape.value);
    if (shapeSize(shape) > 6) addShape(shape);
    draftShape.value = null;
  }
  drawing.active = false;
  drawing.mode = "";
  drawing.start = null;
}

function finishPolygon() {
  if (polygonDraft.value.length < 3) {
    message.value = "多边形至少需要 3 个点";
    return;
  }
  addShape(makeShape("polygon", polygonDraft.value));
  polygonDraft.value = [];
  activeTool.value = "select";
}

```

```

function cancelPolygon() {
  polygonDraft.value = [];
}

function startMoveShape(event: PointerEvent, shapeId: string) {
  if (activeTool.value !== "select") return;
  const point = svgPoint(event);
  if (!point) return;
  selectedShapeId.value = shapeId;
  drawing.active = true;
  drawing.mode = "move";
  drawing.shapeId = shapeId;
  drawing.start = point;
}

function startResizeBox(event: PointerEvent, shapeId: string) {
  const point = svgPoint(event);
  if (!point) return;
  selectedShapeId.value = shapeId;
  drawing.active = true;
  drawing.mode = "resize";
  drawing.shapeId = shapeId;
  drawing.start = point;
}

function moveSelectedShape(point: Point) {
  const shape = annotationShapes.value.find((item) => item.id === drawing.shapeId);
  if (!shape || !drawing.start) return;
  const dx = point.x - drawing.start.x;
  const dy = point.y - drawing.start.y;
  drawing.start = point;
  updateShape(shape.id, {
    points: shape.points.map((item) => clampPoint({ x: item.x + dx, y: item.y + dy })),
    source: "edited",
  });
}

function resizeSelectedBox(point: Point) {
  const shape = annotationShapes.value.find((item) => item.id === drawing.shapeId);
  if (!shape || shape.type !== "box") return;
  updateShape(shape.id, { points: [shape.points[0], clampPoint(point)], source: "edited" });
}

function addShape(shape: AnnotationShape) {
  if (editingDisabled.value) return;
  setShapes([...annotationShapes.value, shape]);
  selectedShapeId.value = shape.id;
  activeTool.value = "select";
}

function updateShape(shapeId: string, patch: Partial<AnnotationShape>) {
  if (editingDisabled.value) return;
  setShapes(annotationShapes.value.map((shape) => (shape.id === shapeId ? { ...shape, ...patch } : shape)));
}

function removeSelectedShape() {
  if (editingDisabled.value) return;
  if (!selectedShapeId.value) return;
  setShapes(annotationShapes.value.filter((shape) => shape.id !== selectedShapeId.value));
  selectedShapeId.value = null;
}

function selectShape(shapeId: string) {
  selectedShapeId.value = shapeId;
  activeTool.value = "select";
}

function selectPresetLabel(label: LabelConfig) {
  activePresetLabelId.value = label.labelId;
  if (selectedShape.value) {

```

```

        updateShape(selectedShape.value.id, {
            labelId: label.labelId,
            label: `${label.englishName} / ${label.chineseName}`,
            source: "edited",
        });
    }
}

function labelShortcut(index: number) {
    if (index < 0 || index >= 20) return "";
    const slot = index % 10;
    const key = slot === 9 ? "0" : String(slot + 1);
    return index >= 10 ? `Shift+${key}` : key;
}

function shortcutLabel(event: KeyboardEvent) {
    const match = event.code.match(/^Digit([0-9])$/);
    if (!match || event.altKey || event.ctrlKey || event.metaKey) return null;
    const digit = Number(match[1]);
    const slot = digit === 0 ? 9 : digit - 1;
    return labels.value[(event.shiftKey ? 10 : 0) + slot] ?? null;
}

function handleGlobalKeydown(event: KeyboardEvent) {
    const target = event.target as HTMLElement | null;
    if (target && ["INPUT", "TEXTAREA", "SELECT"].includes(target.tagName)) return;
    if (currentPage.value !== "workspace") return;
    if ((event.ctrlKey || event.metaKey) && event.key.toLowerCase() === "s") {
        event.preventDefault();
        if (!editingDisabled.value) void saveAnnotationSnapshot();
        return;
    }
    const presetLabel = shortcutLabel(event);
    if (presetLabel) {
        event.preventDefault();
        selectPresetLabel(presetLabel);
        return;
    }
    const toolByKey: Partial<Record<string, AnnotationTool>> = { v: "select", h: "pan", b: "box", p: "polygon", l:
"line" };
    const tool = toolByKey[event.key.toLowerCase()];
    if (tool) {
        event.preventDefault();
        setTool(tool);
        return;
    }
    if (event.key === "Enter" && polygonDraft.value.length >= 3) {
        event.preventDefault();
        finishPolygon();
        return;
    }
    if (event.key === "Escape") {
        event.preventDefault();
        cancelPolygon();
        selectedShapeId.value = null;
        setTool("select");
        return;
    }
    if (event.key.toLowerCase() === "f") {
        event.preventDefault();
        fitView();
        return;
    }
    if (event.key.toLowerCase() === "a" || event.key.toLowerCase() === "d") {
        event.preventDefault();
        void selectAdjacentImage(event.key.toLowerCase() === "a" ? -1 : 1);
        return;
    }
    if (event.key === "+" || event.key === "=") view.zoom = Math.min(5, view.zoom + 0.1);
    if (event.key === "-" || event.key === "_") view.zoom = Math.max(0.2, view.zoom - 0.1);
    if ((event.key === "Delete" || event.key === "Backspace") && selectedShapeId.value) {

```

```

    event.preventDefault();
    removeSelectedShape();
  }
}

async function selectAdjacentImage(direction: -1 | 1) {
  const currentIndex = imageHistoryList.value.findIndex((image) => image.imageId ===
currentState.value.currentImageId);
  const nextIndex = Math.min(imageHistoryList.value.length - 1, Math.max(0, currentIndex + direction));
  const image = imageHistoryList.value[nextIndex];
  if (image && nextIndex !== currentIndex) await selectImage(image);
}

function setShapes(shapes: AnnotationShape[]) {
  currentState.value.annotationJson = {
    ...(currentState.value.annotationJson ?? emptyAnnotation()),
    shapes: shapes.map((shape) => serializeShape(shape)),
    updatedAt: new Date().toISOString(),
  };
}

function ensureAnnotationJson() {
  currentState.value.annotationJson = normalizeAnnotation(currentState.value.annotationJson,
currentState.value.resultJson);
}

function emptyAnnotation() {
  return { shapes: [], updatedAt: new Date().toISOString() };
}

function normalizeAnnotation(annotation: Record<string, unknown> | null, result: Record<string, unknown> | null) {
  const shapes = readShapes(annotation);
  if (shapes.length > 0) return { ...(annotation ?? {}), shapes: shapes.map((shape) => serializeShape(shape)) };
  return { ...(annotation ?? {}), shapes: resultToShapes(result).map((shape) => serializeShape(shape)), updatedAt:
new Date().toISOString() };
}

function resultToShapes(result: Record<string, unknown> | null): AnnotationShape[] {
  const boxes = ((result?.boxes as Array<Record<string, unknown>> | undefined) ?? []).map((box) => {
    const label = String(box.label ?? "object");
    return makeShape(
      "box",
      [
        { x: Number(box.x ?? 0), y: Number(box.y ?? 0) },
        { x: Number(box.x ?? 0) + Number(box.width ?? 0), y: Number(box.y ?? 0) + Number(box.height ?? 0) },
      ],
      label,
      Number(box.score ?? 0),
      "ai",
    );
  });
  const segments = ((result?.segments as Array<Record<string, unknown>> | undefined) ?? []).map((segment) => {
    const points = ((segment.polygon as number[][] | undefined) ?? []).map(([x, y]) => ({ x: Number(x), y:
Number(y) }));
    return makeShape("polygon", points, String(segment.label ?? "segment"), Number(segment.score ?? 0), "ai");
  });
  return [...boxes, ...segments].filter((shape) => shape.points.length >= 2);
}

function readShapes(annotation: Record<string, unknown> | null): AnnotationShape[] {
  const raw = ((annotation?.shapes as Array<Record<string, unknown>> | undefined) ?? []) as Array<Record<string,
unknown>>;
  return raw
    .map((shape) => ({
      id: String(shape.id ?? newId("shape")),
      type: (shape.type === "polygon" || shape.type === "Line" ? shape.type : "box") as ShapeType,
      labelId: shape.labelId === null || shape.labelId === undefined ? null : Number(shape.labelId),
      label: String(shape.label ?? "object"),
      points: ((shape.points as Array<Record<string, number>> | undefined) ?? []).map((point) => ({ x:
Number(point.x), y: Number(point.y) })),
      score: shape.score === undefined ? undefined : Number(shape.score),
    }

```

```

        source: shape.source === "ai" ? "ai" : shape.source === "manual" ? "manual" : "edited",
    )))
    .filter((shape) => shape.points.length >= 2 || shape.type === "polygon");
}

function serializeShape(shape: AnnotationShape) {
    return {
        id: shape.id,
        type: shape.type,
        labelId: shape.labelId,
        label: shape.label,
        points: shape.points.map((point) => ({ x: round(point.x), y: round(point.y) })),
        score: shape.score,
        source: shape.source,
    };
}

function makeShape(type: ShapeType, points: Point[], label = defaultLabelText(), score?: number, source:
AnnotationShape["source"] = "manual") {
    const labelConfig = source === "ai" ? labels.value.find((item) => item.englishName === label) :
activePresetLabel.value;
    return {
        id: newId("shape"),
        type,
        labelId: labelConfig?.labelId ?? null,
        label: labelConfig ? `${labelConfig.englishName} / ${labelConfig.chineseName}` : label,
        points: points.map(clampPoint),
        score,
        source,
    };
}

function normalizeShapePoints(shape: AnnotationShape) {
    if (shape.type !== "box") return { ...shape, points: shape.points.map(clampPoint) };
    const [a, b] = shape.points;
    return { ...shape, points: [clampPoint({ x: Math.min(a.x, b.x), y: Math.min(a.y, b.y) }), clampPoint({ x:
Math.max(a.x, b.x), y: Math.max(a.y, b.y) })] };
}

function shapeSize(shape: AnnotationShape) {
    if (shape.type === "line") return distance(shape.points[0], shape.points[1]);
    const box = shapeBounds(shape);
    return Math.max(box.width, box.height);
}

function shapeBounds(shape: AnnotationShape) {
    const xs = shape.points.map((point) => point.x);
    const ys = shape.points.map((point) => point.y);
    const x = Math.min(...xs);
    const y = Math.min(...ys);
    return { x, y, width: Math.max(...xs) - x, height: Math.max(...ys) - y };
}

function svgPoint(event: PointerEvent): Point | null {
    const svg = event.currentTarget instanceof SVGSVGElement ? event.currentTarget : (event.currentTarget as
Element).closest("svg");
    if (!svg || !currentState.value.imageInfo?.width || !currentState.value.imageInfo?.height) return null;
    const rect = svg.getBoundingClientRect();
    return clampPoint({
        x: ((event.clientX - rect.left) / rect.width) * currentState.value.imageInfo.width,
        y: ((event.clientY - rect.top) / rect.height) * currentState.value.imageInfo.height,
    });
}

function clampPoint(point: Point) {
    const width = currentState.value.imageInfo?.width ?? Number.MAX_SAFE_INTEGER;
    const height = currentState.value.imageInfo?.height ?? Number.MAX_SAFE_INTEGER;
    return { x: Math.min(width, Math.max(0, point.x)), y: Math.min(height, Math.max(0, point.y)) };
}

function boxRect(shape: AnnotationShape) {

```

```

const box = shapeBounds(shape);
return { ...box, labelY: Math.max(box.y - 8, 16), handleX: box.x + box.width, handleY: box.y + box.height };
}

function polygonPoints(points: Point[]) {
  return points.map((point) => `${point.x},${point.y}`).join(" ");
}

function linePoints(points: Point[]) {
  return points.length >= 2 ? { x1: points[0].x, y1: points[0].y, x2: points[1].x, y2: points[1].y } : { x1: 0, y1: 0, x2: 0, y2: 0 };
}

function shapeTypeText(type: ShapeType) {
  return { box: "矩形框", polygon: "多边形", line: "线段" }[type];
}

function resultKeypoints(): PoseKeypoint[] {
  const keypoints = (currentState.value.resultJson?.keypoints as Array<Record<string, number | string>> | undefined) ?? [];
  return keypoints.map((point) => ({
    name: String(point.name),
    x: Number(point.x),
    y: Number(point.y),
    score: Number(point.score ?? 1),
  }));
}

function resultSkeleton(): PoseSkeletonLine[] {
  const skeleton = (currentState.value.resultJson?.skeleton as Array<Record<string, number | string>> | undefined) ?? [];
  if (skeleton.length > 0) {
    return skeleton.map((line, index) => ({
      name: String(line.name ?? `skeleton_${index}`),
      x1: Number(line.x1),
      y1: Number(line.y1),
      x2: Number(line.x2),
      y2: Number(line.y2),
      score: Number(line.score ?? 1),
    }));
  }
  return fallbackSkeletonFromKeypoints(resultKeypoints());
}

function fallbackSkeletonFromKeypoints(keypoints: PoseKeypoint[]): PoseSkeletonLine[] {
  const grouped = new Map<string, Map<string, PoseKeypoint>>>();
  for (const point of keypoints) {
    const match = point.name.match(/^person_(\d+)_(.+)$/);
    if (!match || point.score <= 0.05 || (point.x <= 0 && point.y <= 0)) continue;
    const [, personIndex, keypointName] = match;
    const group = grouped.get(personIndex) ?? new Map<string, PoseKeypoint>();
    group.set(keypointName, point);
    grouped.set(personIndex, group);
  }

  const lines: PoseSkeletonLine[] = [];
  for (const [personIndex, points] of grouped) {
    for (const [fromName, toName] of poseSkeletonPairs) {
      const from = points.get(fromName);
      const to = points.get(toName);
      if (!from || !to) continue;
      lines.push({
        name: `person_${personIndex}_${fromName}_to_${toName}`,
        x1: from.x,
        y1: from.y,
        x2: to.x,
        y2: to.y,
        score: Math.min(from.score, to.score),
      });
    }
  }
}

```

```

    return lines;
}

function resultLabels() {
    return ((currentState.value.resultJson?.labels as Array<Record<string, number | string>> | undefined) ?? []) as
Array<{
    label: string;
    score: number;
}>;
}

function onWheel(event: WheelEvent) {
    if (!currentState.value.imageUrl) return;
    event.preventDefault();
    const direction = event.deltaY > 0 ? -0.12 : 0.12;
    view.zoom = Math.min(5, Math.max(0.2, Number((view.zoom + direction).toFixed(2))));
}

function resetView() {
    view.zoom = 1;
    view.x = 0;
    view.y = 0;
}

function fitView() {
    view.zoom = 0.9;
    view.x = 0;
    view.y = 0;
}

function originalView() {
    resetView();
}

async function runBusy(action: () => Promise<void>, fallback: string) {
    activeRequestCount.value += 1;
    message.value = "";
    try {
        await action();
    } catch (error) {
        const detail = error instanceof Error && error.message ? error.message : fallback;
        if (error instanceof ApiRequestError && error.status === 401) {
            currentUser.value = null;
        }
        message.value = detail;
        errorDialog.value = detail;
    } finally {
        activeRequestCount.value = Math.max(0, activeRequestCount.value - 1);
        processingOverlayText.value = "";
    }
}

function currentImageSessionId() {
    return currentState.value.imageInfo?.sessionId || defaultSessionId;
}

function playCaption() {
    const text = currentState.value.descriptionText?.trim();
    if (!text) {
        message.value = "暂无可播放的图像描述";
        return;
    }
    if (!("speechSynthesis" in window)) {
        message.value = "当前浏览器不支持语音播放";
        console.error("[Speech] speechSynthesis is not supported");
        return;
    }
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.lang = "zh-CN";
    utterance.rate = 0.95;
}

```

```

utterance.onerror = (event) => {
  console.error("[Speech] Caption playback failed", event);
  message.value = "语音播放失败";
};
window.speechSynthesis.speak(utterance);
}

function validateImage(file: File): string {
  const allowed = ["image/jpeg", "image/png", "image/webp", "image/bmp"];
  const extAllowed = /\.?(jpe?g|png|webp|bmp)$/i.test(file.name);
  if (!allowed.includes(file.type) && !extAllowed) return "图片格式不支持, 请上传 jpg、jpeg、png、webp 或 bmp";
  if (file.size > 20 * 1024 * 1024) return "图片大小超过 20MB 限制";
  return "";
}

function createDefaultTaskState(): TaskState {
  return {
    currentImageId: null,
    imageUrl: "",
    imageInfo: null,
    taskId: null,
    resultImageUrl: "",
    resultJson: null,
    annotationJson: emptyAnnotation(),
    descriptionText: "",
    status: "idle",
    versions: [],
    activeVersionId: null,
    latestVersionNo: null,
    isViewingHistory: false,
    reviewStatus: "draft",
    reviewComment: "",
    submittedAt: null,
    reviewedAt: null,
  };
}

function defaultLabelText() {
  return "object / 对象";
}

function newId(prefix: string) {
  return `${prefix}_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 8)}`;
}

function round(value: number) {
  return Math.round(value * 100) / 100;
}

function distance(a: Point, b: Point) {
  return Math.hypot(a.x - b.x, a.y - b.y);
}

function formatBytes(value?: number | null) {
  if (!value) return "-";
  if (value < 1024 * 1024) return `${Math.round(value / 1024)} KB`;
  return `${(value / 1024 / 1024).toFixed(1)} MB`;
}

function formatTime(value?: string) {
  if (!value) return "-";
  return new Date(value).toLocaleString();
}

function formatDuration(value?: number | null) {
  if (!value) return "-";
  return formatTimestamp(value);
}

function formatTimestamp(value?: number | null) {
  if (!value) return "00:00";
}

```

```

const totalSeconds = Math.floor(value / 1000);
const hours = Math.floor(totalSeconds / 3600);
const minutes = Math.floor((totalSeconds % 3600) / 60);
const seconds = totalSeconds % 60;
const base = `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(2, "0")}`;
return hours > 0 ? `${String(hours).padStart(2, "0")}:${base}` : base;
}

function statusText(status: ImageTaskStatus) {
  return { idle: "未处理", pending: "待处理", processing: "处理中", success: "已完成", failed: "失败" }[status];
}

function reviewStatusText(status: ImageReviewStatus) {
  return { draft: "草稿", pending_review: "待审核", approved: "已通过", rejected: "已驳回" }[status];
}
</script>

<template>
  <main v-if="currentUser" class="app-shell image-task-workspace">
    <header class="topbar">
      <div class="brand">
        
        <div>
          <h1>铁路多目标识别平台</h1>
          <span>{{ loading ? "处理中" : "就绪" }}<template v-if="message"> • {{ message }}</template></span>
        </div>
      </div>
      <div class="topbar-actions">
        <nav class="task-tabs" aria-label="工作区">
          <button :class="{ active: currentPage === 'workspace' }" @click="setPage('workspace')"><Boxes :size="16" />标注工作台</button>
          <button :class="{ active: currentPage === 'videoCaptions' }" @click="setPage('videoCaptions')"><Film :size="16" />视频解帧</button>
          <button :class="{ active: currentPage === 'liveGesture' }" @click="setPage('liveGesture')"><Camera :size="16" />实时手势</button>
          <button :class="{ active: currentPage === 'settings' }" @click="setPage('settings')"><Settings :size="16" />系统配置</button>
        </nav>
        <div class="user-session">
          <UserRound :size="17" />
          <span><strong>{{ currentUser.displayName }}</strong><small>{{ currentUser.username }}</small></span>
          <button title="退出登录" aria-label="退出登录" @click="signOut"><LogOut :size="16" /></button>
        </div>
      </div>
    </header>

    <nav v-if="currentPage === 'workspace'" class="task-strip" aria-label="功能页面">
      <button v-for="task in taskConfigs" :key="task.type" :class="{ active: currentTaskType === task.type }" @click="setTaskType(task.type)">
        {{ task.title }}
      </button>
    </nav>

    <section
      v-if="currentPage === 'workspace'"
      ref="workspaceBodyRef"
      class="workspace-body"
      :class="{ 'is-resizing': panelLayout.resizing }"
      :style="workspaceGridStyle"
    >
      <aside class="history-panel" :class="{ collapsed: historyCollapsed }">
        <div class="panel-head">
          <strong v-if="!historyCollapsed">历史图片</strong>
          <button :title="historyCollapsed ? '展开历史列表' : '收起历史列表'" @click="historyCollapsed = !historyCollapsed">
            <ChevronRight v-if="historyCollapsed" :size="16" />
            <ChevronLeft v-else :size="16" />
          </button>
        </div>
        <template v-if="!historyCollapsed">
          <label class="upload-zone compact">

```

```

        <Upload :size="18" />
        <span>上传图片</span>
        <input type="file" accept="image/jpeg,image/png,image/webp,image/bmp" @change="handleUpload" />
    </label>
    <button class="ghost-button" :disabled="loading" @click="loadImageHistory"><RefreshCw :size="16" /> 刷新
历史</button>
    <div class="history-list">
        <button
            v-for="image in imageHistoryList"
            :key="image.imageId"
            class="history-item"
            :class="{ selected: image.imageId === currentState.currentImageId }"
            @click="selectImage(image)"
        >
            
            <span>
                <strong>{{ image.originalName }}</strong>
                <small>{{ formatTime(image.createdAt) }}</small>
                <small>{{ statusText(image.taskStatus) }} • {{ image.hasCurrentTaskResult ? "已有结果" : "暂无结
果" }}</small>
            </span>
        </button>
        <div v-if="imageHistoryList.length === 0" class="empty-state">暂无历史图片</div>
    </div>
</template>
</aside>

<button
    class="panel-resizer history-resizer"
    :disabled="historyCollapsed"
    title="拖动调整历史图片宽度"
    aria-label="拖动调整历史图片宽度"
    @pointerdown="startPanelResize($event, 'history')"
></button>

<section class="editor-panel">
    <div class="editor-toolbar">
        <div>
            <strong>{{ currentConfig.title }}</strong>
            <span>{{ currentConfig.description }}</span>
        </div>
        <div class="toolbar-actions">
            <label class="model-select">
                <span>模型</span>
                <select v-model="selectedModelByTask[currentTaskType]" :disabled="loading">
                    <option v-for="model in currentConfig.models" :key="model" :value="model">{{ model }}</option>
                </select>
            </label>
            <button :disabled="!currentState.currentImageId || loading || editingDisabled" @click="runCurrentTask">
                <Loader2 v-if="loading" class="spin" :size="16" />
                <Sparkles v-else :size="16" />
                开始处理
            </button>
            <button :disabled="!currentState.currentImageId || loading || editingDisabled"
@click="saveCurrentResult"><Save :size="16" />保存结果</button>
            <button :disabled="!currentState.currentImageId || loading || editingDisabled"
@click="saveAnnotationSnapshot"><Download :size="16" />保存标注</button>
        </div>
    </div>

    <div class="annotation-toolbar">
        <button :class="{ active: activeTool === 'select' }" title="选择" @click="setTool('select')">
<MousePointer2 :size="16" /> (选择) </button>
        <button :class="{ active: activeTool === 'pan' }" title="移动画布" @click="setTool('pan')"><Move
:size="16" /> (移动) </button>
        <button :disabled="editingDisabled" :class="{ active: activeTool === 'box' }" title="矩形框 (B) "
@click="setTool('box')"><Boxes :size="16" /> (矩形框) </button>
        <button :disabled="editingDisabled" :class="{ active: activeTool === 'polygon' }" title="多边形 (P) "
@click="setTool('polygon')"><Pentagon :size="16" /> (多边形) </button>
        <button :disabled="editingDisabled" :class="{ active: activeTool === 'line' }" title="线段 (L) "
@click="setTool('line')"><Waypoints :size="16" /> (线段) </button>
    </div>

```

```

        <button :disabled="polygonDraft.length < 3" @click="finishPolygon"><Check :size="16" /> (完成) </button>
        <button :disabled="polygonDraft.length === 0" @click="cancelPolygon"><RotateCcw :size="16" /> (取消)
    </button>
  </div>

  <div
    class="canvas-stage"
    :class="`tool-${activeTool}`"
    @wheel="onWheel"
    @pointerdown="onStagePointerDown"
    @pointermove="onStagePointerMove"
    @pointerup="stopStageDrag"
    @pointerleave="stopStageDrag"
  >
    <div v-if="!currentState.imageUrl" class="empty-canvas">
      <ImageIcon :size="40" />
      <label class="upload-zone">
        <Upload :size="20" />
        <span>选择图片开始</span>
        <input type="file" accept="image/jpeg, image/png, image/webp, image/bmp" @change="handleUpload" />
      </label>
    </div>

    <div v-else class="canvas-content" :style="{ transform: `translate(${view.x}px, ${view.y}px)
scale(${view.zoom})` }">
      
      <svg
        v-if="currentState.imageInfo?.width && currentState.imageInfo?.height"
        class="overlay-layer interactive"
        :viewBox="`0 0 ${currentState.imageInfo.width} ${currentState.imageInfo.height}`"
        @pointerdown.stop="onOverlayPointerDow

```

frontend/src/components/LiveGesturePage.vue

```

<script setup lang="ts">
import { computed, nextTick, onBeforeUnmount, ref } from "vue";
import { Camera, CameraOff, Hand, Loader2, RefreshCw, ShieldCheck } from "@lucide/vue";
import {
  FilesetResolver,
  HandLandmarker,
  type HandLandmarkerResult,
  type NormalizedLandmark,
} from "@mediapipe/tasks-vision";

type CameraState = "idle" | "loading" | "running" | "error";
type HandReading = {
  side: string;
  confidence: number;
  fingers: number;
  gesture: string;
};

const videoRef = ref<HTMLVideoElement | null>(null);
const canvasRef = ref<HTMLCanvasElement | null>(null);
const cameraState = ref<CameraState>("idle");
const statusMessage = ref("摄像头尚未启动");
const readings = ref<HandReading[]>([]);
const fps = ref(0);

let stream: MediaStream | null = null;
let landmarker: HandLandmarker | null = null;
let animationFrame = 0;
let lastVideoTime = -1;
let frameCount = 0;
let fpsStartedAt = 0;

const isSecureCameraContext = computed(
  () => window.isSecureContext || ["localhost", "****", ".*"].includes(window.location.hostname),
);

```

```

const stateText = computed(() => {
  if (cameraState.value === "loading") return "模型与摄像头初始化中";
  if (cameraState.value === "running") return "实时识别中";
  if (cameraState.value === "error") return "摄像头不可用";
  return "未启动";
});

async function startCamera() {
  if (cameraState.value === "loading" || cameraState.value === "running") return;
  if (!isSecureCameraContext.value || !navigator.mediaDevices?.getUserMedia) {
    cameraState.value = "error";
    statusMessage.value = "浏览器仅允许在 HTTPS 或 localhost 页面调用摄像头，请改用安全访问地址";
    return;
  }

  cameraState.value = "loading";
  statusMessage.value = "正在加载手部关键点模型";
  readings.value = [];
  try {
    await ensureLandmarker();
    statusMessage.value = "正在请求摄像头权限";
    stream = await navigator.mediaDevices.getUserMedia({
      audio: false,
      video: { facingMode: "user", width: { ideal: 1280 }, height: { ideal: 720 }, frameRate: { ideal: 30 } },
    });
    await nextTick();
    const video = videoRef.value;
    if (!video) throw new Error("视频组件尚未就绪");
    video.srcObject = stream;
    await video.play();
    cameraState.value = "running";
    statusMessage.value = "请将手掌置于画面中央";
    lastVideoTime = -1;
    frameCount = 0;
    fpsStartedAt = performance.now();
    animationFrame = requestAnimationFrame(predictFrame);
  } catch (error) {
    stopCamera(false);
    cameraState.value = "error";
    statusMessage.value = cameraErrorMessage(error);
  }
}

async function ensureLandmarker() {
  if (landmarker) return;
  const vision = await FilesetResolver.forVisionTasks("/mediapipe/wasm");
  const options = {
    baseOptions: { modelAssetPath: "/mediapipe/models/hand_landmarker.task", delegate: "GPU" as const },
    runningMode: "VIDEO" as const,
    numHands: 2,
    minHandDetectionConfidence: 0.55,
    minHandPresenceConfidence: 0.55,
    minTrackingConfidence: 0.5,
  };
  try {
    landmarker = await HandLandmarker.createFromOptions(vision, options);
  } catch {
    landmarker = await HandLandmarker.createFromOptions(vision, {
      ...options,
      baseOptions: { modelAssetPath: "/mediapipe/models/hand_landmarker.task", delegate: "CPU" },
    });
  }
}

function predictFrame(now: number) {
  if (cameraState.value !== "running") return;
  const video = videoRef.value;
  const canvas = canvasRef.value;
  if (video && canvas && landmarker && video.readyState >= HTMLMediaElement.HAVE_CURRENT_DATA) {
    if (video.currentTime !== lastVideoTime) {

```

```

        lastVideoTime = video.currentTime;
        const result = landmarker.detectForVideo(video, now);
        renderResult(result, canvas, video.videoWidth, video.videoHeight);
        updateFps(now);
    }
}
animationFrame = requestAnimationFrame(predictFrame);
}

function renderResult(result: HandLandmarkerResult, canvas: HTMLCanvasElement, width: number, height: number) {
    if (!width || !height) return;
    if (canvas.width !== width || canvas.height !== height) {
        canvas.width = width;
        canvas.height = height;
    }
    const context = canvas.getContext("2d");
    if (!context) return;
    context.clearRect(0, 0, width, height);
    readings.value = result.landmarks.map((points, index) => {
        drawHand(context, points, width, height, index);
        const category = result.handedness[index]?.[0];
        return {
            side: category?.categoryName === "Left" ? "左手" : category?.categoryName === "Right" ? "右手" : "手部",
            confidence: category?.score ?? 0,
            fingers: extendedFingers(points).length,
            gesture: classifyGesture(points),
        };
    });
    statusMessage.value = readings.value.length ? `已检测到 ${readings.value.length} 只手` : "未检测到手部，请调整距离和光线";
}

function drawHand(context: CanvasRenderingContext2D, points: NormalizedLandmark[], width: number, height: number, handIndex: number) {
    const color = handIndex === 0 ? "#36d399" : "#ffb84d";
    context.lineWidth = Math.max(3, width / 320);
    context.strokeStyle = color;
    context.lineCap = "round";
    for (const connection of HandLandmarker.HAND_CONNECTIONS) {
        const start = points[connection.start];
        const end = points[connection.end];
        context.beginPath();
        context.moveTo(start.x * width, start.y * height);
        context.lineTo(end.x * width, end.y * height);
        context.stroke();
    }
    for (const [index, point] of points.entries()) {
        context.beginPath();
        context.fillStyle = [4, 8, 12, 16, 20].includes(index) ? "#ffffff" : color;
        context.strokeStyle = "#10232f";
        context.lineWidth = 2;
        context.arc(point.x * width, point.y * height, Math.max(4, width / 210), 0, Math.PI * 2);
        context.fill();
        context.stroke();
    }
}

function extendedFingers(points: NormalizedLandmark[]) {
    const extended: string[] = [];
    const wrist = points[0];
    const fingerJoints = [
        { name: "食指", tip: 8, pip: 6 },
        { name: "中指", tip: 12, pip: 10 },
        { name: "无名指", tip: 16, pip: 14 },
        { name: "小指", tip: 20, pip: 18 },
    ];
    for (const finger of fingerJoints) {
        if (pointDistance(points[finger.tip], wrist) > pointDistance(points[finger.pip], wrist) * 1.16)
            extended.push(finger.name);
    }
    if (pointDistance(points[4], points[5]) > pointDistance(points[3], points[5]) * 1.15) extended.unshift("拇指");
}

```

```

    return extended;
}

function classifyGesture(points: NormalizedLandmark[]) {
    const fingers = extendedFingers(points);
    const has = (name: string) => fingers.includes(name);
    if (pointDistance(points[4], points[8]) < 0.055 && has("中指") && has("无名指") && has("小指")) return "OK";
    if (fingers.length === 0) return "握拳";
    if (fingers.length >= 4) return "张开手掌";
    if (fingers.length === 1 && has("食指")) return "食指指向";
    if (fingers.length === 1 && has("拇指")) return points[4].y < points[2].y ? "点赞" : "拇指手势";
    if (fingers.length === 2 && has("食指") && has("中指")) return "胜利手势";
    return `${fingers.length} 指伸展`;
}

function pointDistance(a: NormalizedLandmark, b: NormalizedLandmark) {
    return Math.hypot(a.x - b.x, a.y - b.y, (a.z ?? 0) - (b.z ?? 0));
}

function updateFps(now: number) {
    frameCount += 1;
    const elapsed = now - fpsStartedAt;
    if (elapsed >= 1000) {
        fps.value = Math.round((frameCount * 1000) / elapsed);
        frameCount = 0;
        fpsStartedAt = now;
    }
}

function stopCamera(resetState = true) {
    cancelAnimationFrame(animationFrame);
    animationFrame = 0;
    stream?.getTracks().forEach((track) => track.stop());
    stream = null;
    if (videoRef.value) videoRef.value.srcObject = null;
    const context = canvasRef.value?.getContext("2d");
    if (context && canvasRef.value) context.clearRect(0, 0, canvasRef.value.width, canvasRef.value.height);
    readings.value = [];
    fps.value = 0;
    if (resetState) {
        cameraState.value = "idle";
        statusMessage.value = "摄像头已停止";
    }
}

function cameraErrorMessage(error: unknown) {
    const name = error instanceof DOMException ? error.name : "";
    if (name === "NotAllowedError") return "摄像头权限被拒绝, 请在浏览器地址栏中允许访问后重试";
    if (name === "NotFoundError") return "未检测到可用摄像头";
    if (name === "NotReadableError") return "摄像头正被其他程序占用";
    return error instanceof Error ? `启动失败: ${error.message}` : "摄像头启动失败";
}

onBeforeUnmount(() => {
    stopCamera(false);
    landmarker?.close();
    landmarker = null;
});
</script>

<template>
    <section class="gesture-page">
        <div class="gesture-toolbar">
            <div>
                <h2>实时手势识别</h2>
                <p>{{ statusMessage }}</p>
            </div>
            <div class="gesture-actions">
                <span class="camera-state" :class="`state-${cameraState}`"><i></i>{{ stateText }}</span>
                <button v-if="cameraState !== 'running'" :disabled="cameraState === 'loading'" @click="startCamera">
                    <Loader2 v-if="cameraState === 'loading'" class="spin" :size="17" />
                </button>
            </div>
        </div>
    </section>
</template>

```

```

    <Camera v-else :size="17" />
    {{ cameraState === "error" ? "重新启动" : "启动摄像头" }}
  </button>
  <button v-else class="ghost-button" @click="stopCamera()"><CameraOff :size="17" />停止</button>
</div>
</div>

<div class="gesture-layout">
  <div class="camera-stage">
    <video ref="videoRef" muted playsinline></video>
    <canvas ref="canvasRef"></canvas>
    <div v-if="cameraState !== 'running'" class="camera-placeholder">
      <Loader2 v-if="cameraState === 'loading'" class="spin" :size="48" />
      <CameraOff v-else :size="48" />
      <strong>{{ stateText }}</strong>
      <span>{{ statusMessage }}</span>
    </div>
    <div v-if="cameraState === 'running'" class="camera-metrics">
      <span>{{ fps }} FPS</span><span>{{ readings.length }} HANDS</span>
    </div>
  </div>

  <aside class="gesture-results">
    <div class="result-panel-heading">
      <span><Hand :size="18" />识别结果</span>
      <small>最多同时识别 2 只手</small>
    </div>
    <div v-if="readings.length === 0" class="gesture-empty">
      <Hand :size="34" />
      <strong>等待手部进入画面</strong>
      <span>保持手掌完整可见，并避免逆光</span>
    </div>
    <article v-for="(reading, index) in readings" :key="index" class="hand-result">
      <header><span>手部 {{ index + 1 }}</span><strong>{{ reading.gesture }}</strong></header>
      <dl>
        <dt>左右手</dt><dd>{{ reading.side }}</dd>
        <dt>置信度</dt><dd>{{ Math.round(reading.confidence * 100) }}%</dd>
        <dt>伸展手指</dt><dd>{{ reading.fingers }} 根</dd>
        <dt>关键点</dt><dd>21 个</dd>
      </dl>
    </article>
    <div class="privacy-note"><ShieldCheck :size="17" /><span>视频仅在当前浏览器中实时处理，不会上传或保存。</span></div>
    <div v-if="!isSecureCameraContext" class="https-warning">
      当前地址不是 HTTPS。公网摄像头识别需要配置 HTTPS 后才能使用。
    </div>
    <button v-if="cameraState === 'error'" class="retry-button" @click="startCamera"><RefreshCw :size="16" />重试</button>
  </aside>
</div>
</section>
</template>

```

frontend/src/env.d.ts

```

/// <reference types="vite/client" />

declare module "*.vue" {
  import type { DefineComponent } from "vue";

  const component: DefineComponent<object, object, unknown>;
  export default component;
}

```

frontend/src/main.ts

```

import { createApp } from "vue";
import App from "./App.vue";

```

```
import "./styles.css";

createApp(App).mount("#root");
```

frontend/src/styles.css

```
:root {
  color: #17202a;
  background: #f4f6f8;
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 36px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
  cursor: pointer;
}

button:disabled {
  cursor: not-allowed;
  opacity: 0.55;
}

button.active {
  border-color: #286eac;
  color: #145995;
  background: #eef6ff;
}

.app-shell {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.topbar {
  min-height: 76px;
  padding: 12px 20px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  border-bottom: 1px solid #d7dee6;
```

```
    background: #ffffff;
}

.brand {
  min-width: 260px;
  display: flex;
  align-items: center;
  gap: 10px;
}

.brand img {
  width: 38px;
  height: 38px;
  flex: 0 0 auto;
}

.topbar h1 {
  margin: 0;
  font-size: 20px;
  line-height: 1.2;
  font-weight: 700;
}

.topbar span,
.editor-toolbar span,
.empty-state,
.history-item small,
.view-controls span {
  color: #66717d;
  font-size: 13px;
}

.image-task-workspace {
  height: 100vh;
  overflow: hidden;
}

.task-tabs,
.task-strip,
.task-mini-tabs,
.annotation-toolbar,
.toolbar-actions,
.view-controls {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
}

.model-select {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  min-height: 36px;
  color: #405160;
  font-size: 13px;
}

.model-select select {
  min-width: 190px;
  height: 36px;
  padding: 0 10px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
}

.task-tabs {
  flex: 0 0 auto;
}
```

```
.topbar-actions,
.user-session {
  display: flex;
  align-items: center;
  gap: 10px;
}

.topbar-actions {
  min-width: 0;
}

.user-session {
  min-width: 168px;
  padding-left: 12px;
  border-left: 1px solid #d7dee6;
}

.user-session > span {
  min-width: 0;
  display: grid;
  flex: 1;
}

.user-session strong,
.user-session small {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.user-session strong {
  color: #17202a;
  font-size: 13px;
}

.user-session small {
  color: #66717d;
  font-size: 11px;
}

.user-session button {
  width: 34px;
  min-width: 34px;
  padding: 0;
}

.task-tabs button {
  min-width: 86px;
}

.task-strip {
  min-height: 48px;
  padding: 8px 12px;
  border-bottom: 1px solid #d7dee6;
  background: #fbfcfe;
}

.task-strip button {
  min-width: 112px;
  white-space: nowrap;
}

.workspace-body {
  flex: 1;
  display: grid;
  grid-template-columns: 280px 6px minmax(360px, 1fr) 6px 300px;
  gap: 8px;
  min-height: 0;
  padding: 12px;
```

```
}

.workspace-body.is-resizing {
  user-select: none;
}

.history-panel,
.thumbnail-panel,
.editor-panel {
  min-height: 0;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  background: #ffffff;
}

.history-panel,
.thumbnail-panel {
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.history-panel.collapsed,
.thumbnail-panel.collapsed {
  width: 48px;
}

.thumbnail-panel:not(.collapsed) {
  overflow-y: auto;
}

.thumbnail-panel:not(.collapsed) > .panel-head {
  position: sticky;
  top: 0;
  z-index: 3;
  flex: 0 0 auto;
  background: #ffffff;
}

.panel-resizer {
  align-self: stretch;
  min-width: 6px;
  width: 6px;
  min-height: 0;
  padding: 0;
  border: 0;
  border-radius: 4px;
  background: transparent;
  cursor: col-resize;
}

.panel-resizer::before {
  content: "";
  display: block;
  width: 2px;
  height: 100%;
  margin: 0 auto;
  border-radius: 2px;
  background: #d7dee6;
}

.panel-resizer:hover::before,
.workspace-body.is-resizing .panel-resizer::before {
  background: #286eac;
}

.panel-resizer:disabled {
  cursor: default;
  opacity: 0.35;
}
```

```
.panel-head {
  min-height: 48px;
  padding: 8px 10px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid #e1e7ee;
}

.upload-zone {
  position: relative;
  min-height: 120px;
  margin: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 8px;
  border: 1px dashed #a8b7c6;
  border-radius: 8px;
  color: #405160;
  background: #f8fafc;
  cursor: pointer;
}

.task-mini-tabs {
  padding: 10px;
  align-items: stretch;
}

.task-mini-tabs button {
  flex: 1 1 45%;
  min-width: 0;
  font-size: 12px;
}

.upload-zone.compact {
  min-height: 72px;
}

.upload-zone input {
  position: absolute;
  inset: 0;
  opacity: 0;
  cursor: pointer;
}

.ghost-button {
  margin: 0 10px 10px;
  width: calc(100% - 20px);
}

.history-list {
  min-height: 0;
  padding: 0 10px 10px;
  display: grid;
  gap: 8px;
  overflow-y: auto;
}

.history-item {
  width: 100%;
  min-height: 72px;
  padding: 8px;
  display: grid;
  grid-template-columns: 58px minmax(0, 1fr);
  gap: 9px;
  align-items: center;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}
```

```
}

.history-item.selected {
  border-color: #286eac;
  background: #eef6ff;
}

.history-item img {
  width: 58px;
  height: 50px;
  object-fit: cover;
  border-radius: 6px;
  background: #e8edf2;
}

.history-item span {
  min-width: 0;
  display: grid;
  gap: 3px;
}

.history-item strong {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}

.editor-panel {
  min-width: 0;
  display: grid;
  grid-template-rows: auto auto minmax(0, 1fr) auto;
  overflow: hidden;
}

.editor-toolbar {
  padding: 10px 12px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  border-bottom: 1px solid #e1e7ee;
}

.editor-toolbar > div:first-child {
  display: grid;
  gap: 3px;
}

.annotation-toolbar {
  min-height: 46px;
  padding: 6px 12px;
  border-bottom: 1px solid #e1e7ee;
  background: #fbfcfe;
  flex-wrap: nowrap;
  overflow-x: auto;
}

.annotation-toolbar button {
  min-width: 36px;
  padding: 0 10px;
  white-space: nowrap;
}

.canvas-stage {
  position: relative;
  min-width: 0;
  min-height: 0;
  overflow: hidden;
  background:
    linear-gradient(45deg, #eef2f6 25%, transparent 25%),
```

```
    linear-gradient(-45deg, #eef2f6 25%, transparent 25%),
    linear-gradient(45deg, transparent 75%, #eef2f6 75%),
    linear-gradient(-45deg, transparent 75%, #eef2f6 75%);
background-position: 0 0, 0 10px, 10px -10px, -10px 0;
background-size: 20px 20px;
cursor: default;
}
```

```
.canvas-stage.tool-pan {
  cursor: grab;
}
```

```
.canvas-stage.tool-pan:active {
  cursor: grabbing;
}
```

```
.canvas-stage.tool-box,
.canvas-stage.tool-line,
.canvas-stage.tool-polygon {
  cursor: crosshair;
}
```

```
.processing-mask {
  position: absolute;
  inset: 0;
  z-index: 12;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 24px;
  background: rgba(15, 23, 42, 0.56);
  backdrop-filter: blur(2px);
}
```

```
.processing-card {
  width: min(360px, 100%);
  min-height: 180px;
  padding: 28px 24px;
  display: grid;
  justify-items: center;
  align-content: center;
  gap: 12px;
  border: 1px solid rgba(255, 255, 255, 0.22);
  border-radius: 8px;
  color: #ffffff;
  text-align: center;
  background: rgba(23, 32, 42, 0.92);
  box-shadow: 0 18px 45px rgba(15, 23, 42, 0.35);
}
```

```
.processing-card strong {
  font-size: 18px;
  line-height: 1.35;
}
```

```
.processing-card span {
  max-width: 100%;
  color: #d7dee6;
  font-size: 13px;
  overflow-wrap: anywhere;
}
```

```
.processing-spinner {
  color: #7dd3fc;
  animation: spin 900ms linear infinite;
}
```

```
.empty-canvas {
  position: absolute;
  inset: 0;
  display: flex;
}
```

```
align-items: center;
justify-content: center;
flex-direction: column;
gap: 12px;
color: #66717d;
}

.canvas-content {
position: absolute;
inset: 24px;
display: flex;
align-items: center;
justify-content: center;
transform-origin: center center;
transition: transform 120ms ease;
}

.base-image {
max-width: 100%;
max-height: 100%;
object-fit: contain;
user-select: none;
pointer-events: none;
}

.overlay-layer {
position: absolute;
max-width: 100%;
max-height: 100%;
width: auto;
height: auto;
inset: 0;
margin: auto;
overflow: visible;
pointer-events: none;
}

.overlay-layer.interactive {
pointer-events: auto;
}

.box-shape {
fill: rgba(225, 29, 72, 0.04);
stroke: #e11d48;
stroke-width: 4;
vector-effect: non-scaling-stroke;
}

.box-label {
fill: #e11d48;
font-size: 28px;
font-weight: 700;
paint-order: stroke;
stroke: #ffffff;
stroke-width: 4;
vector-effect: non-scaling-stroke;
pointer-events: none;
}

.segment-shape {
fill: rgba(40, 110, 172, 0.26);
stroke: #286eac;
stroke-width: 4;
vector-effect: non-scaling-stroke;
}

.line-shape {
stroke: #7c3aed;
stroke-width: 5;
stroke-linecap: round;
vector-effect: non-scaling-stroke;
}
```

```
}

.shape-group {
  cursor: pointer;
  pointer-events: visiblePainted;
}

.shape-group.selected .box-shape,
.shape-group.selected .segment-shape,
.shape-group.selected .line-shape {
  stroke: #f59e0b;
}

.resize-handle,
.draft-point {
  fill: #ffffff;
  stroke: #f59e0b;
  stroke-width: 4;
  cursor: nwse-resize;
  vector-effect: non-scaling-stroke;
}

.draft-shape {
  fill: rgba(245, 158, 11, 0.12);
  stroke: #f59e0b;
  stroke-width: 4;
  stroke-dasharray: 10 8;
  vector-effect: non-scaling-stroke;
}

.pose-bone {
  stroke: #0ea5e9;
  stroke-width: 5;
  stroke-linecap: round;
  vector-effect: non-scaling-stroke;
  pointer-events: none;
}

.keypoint {
  fill: #f59e0b;
  stroke: #ffffff;
  stroke-width: 3;
  vector-effect: non-scaling-stroke;
  pointer-events: none;
}

.view-controls {
  min-height: 48px;
  padding: 8px 12px;
  border-top: 1px solid #e1e7ee;
}

.thumbnail-card {
  margin: 10px;
  padding: 0;
  display: block;
  overflow: hidden;
  background: #f8fafc;
}

.thumbnail-card img {
  display: block;
  width: 100%;
  max-height: 160px;
  object-fit: contain;
}

.image-info {
  margin: 0;
  padding: 0 12px 12px;
  display: grid;
```

```
grid-template-columns: 70px minmax(0, 1fr);
gap: 8px;
font-size: 13px;
}

.image-info dt {
  color: #66717d;
}

.image-info dd {
  margin: 0;
  min-width: 0;
  overflow-wrap: anywhere;
}

.field-label {
  display: grid;
  gap: 6px;
  color: #405160;
  font-size: 13px;
}

.field-label input,
.field-label select,
.field-label textarea,
.label-row input,
.label-row textarea {
  width: 100%;
  min-width: 0;
  padding: 8px 10px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
}

.field-label textarea,
.label-row textarea {
  resize: vertical;
}

.danger-button {
  width: 100%;
  margin-top: 10px;
  border-color: #fecdd3;
  color: #be123c;
  background: #fff1f2;
}

.result-summary {
  margin: 0 10px 10px;
  padding-top: 10px;
  border-top: 1px solid #e1e7ee;
}

.result-summary h2 {
  margin: 0 0 10px;
  font-size: 15px;
}

.review-heading,
.review-actions,
.user-admin-row,
.user-admin-row > span {
  display: flex;
  align-items: center;
}

.review-heading,
.user-admin-row {
  justify-content: space-between;
```

```
    gap: 10px;
}

.review-heading h2 {
    margin: 0;
}

.review-summary {
    display: grid;
    gap: 10px;
}

.review-status {
    padding: 3px 7px;
    border-radius: 4px;
    color: #405160;
    background: #e8edf2;
    font-size: 11px;
    font-weight: 700;
}

.review-status.status-pending_review {
    color: #7a5200;
    background: #fff1c7;
}

.review-status.status-approved {
    color: #17613b;
    background: #dff3e8;
}

.review-status.status-rejected {
    color: #9b2d27;
    background: #fbe4e2;
}

.review-comment {
    margin: 0;
    color: #405160;
    font-size: 12px;
    line-height: 1.5;
}

.review-summary textarea {
    width: 100%;
    resize: vertical;
}

.review-actions {
    gap: 8px;
}

.review-actions button {
    flex: 1;
}

.user-admin-header {
    margin-top: 24px;
}

.user-admin-list {
    border: 1px solid #d7dee6;
    border-radius: 8px;
    background: #ffffff;
}

.user-admin-row {
    min-height: 58px;
    padding: 9px 12px;
    border-bottom: 1px solid #e1e7ee;
}
```

```
.user-admin-row:last-child {
  border-bottom: 0;
}

.user-admin-row > span {
  min-width: 0;
  flex: 1;
  align-items: flex-start;
  flex-direction: column;
}

.user-admin-row small {
  color: #66717d;
}

.user-admin-row select {
  min-width: 120px;
  height: 36px;
}

.preset-heading {
  margin-bottom: 8px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
}

.preset-heading h2 {
  margin: 0;
}

.preset-heading > span {
  color: #66717d;
  font-size: 12px;
}

.label-preset-grid {
  max-height: 260px;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
  overflow-y: auto;
}

.preset-label-button {
  width: 100%;
  min-width: 0;
  min-height: 48px;
  padding: 5px 6px;
  display: grid;
  grid-template-columns: 30px minmax(0, 1fr);
  gap: 6px;
  justify-content: stretch;
  text-align: left;
  background: #f8fafc;
}

.preset-label-button kbd {
  width: 30px;
  min-height: 24px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #bdc8d4;
  border-bottom-width: 2px;
  border-radius: 4px;
  color: #405160;
  background: #ffffff;
  font-family: inherit;
```

```
font-size: 11px;
font-weight: 700;
}

.preset-label-button > span {
  min-width: 0;
  display: grid;
  align-content: center;
  gap: 2px;
}

.preset-label-button strong,
.preset-label-button small {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.preset-label-button strong {
  font-size: 12px;
}

.preset-label-button small {
  color: #66717d;
  font-size: 10px;
}

.preset-label-button.active kbd {
  border-color: #286eac;
  color: #145995;
  background: #ffffff;
}

.result-row {
  min-height: 36px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  border-bottom: 1px solid #edf1f5;
}

.result-row strong,
.result-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.annotated-summary {
  max-height: 180px;
  overflow-y: auto;
}

.annotation-row {
  width: 100%;
  min-height: 34px;
  padding: 6px 8px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 8px;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}

.annotation-row.selected {
  border-color: #f59e0b;
  background: #fff7df;
}
```

```
}

.annotation-row strong,
.annotation-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.annotation-row span {
  color: #66717d;
  font-size: 12px;
}

.caption-actions {
  margin-bottom: 8px;
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}

.caption-actions button {
  flex: 1 1 116px;
  min-width: 0;
  min-height: 32px;
  padding: 0 8px;
  white-space: nowrap;
}

.caption-text {
  margin: 0;
  line-height: 1.6;
  color: #405160;
}

.history-viewing {
  margin-bottom: 8px;
  padding: 8px;
  border: 1px solid #f5c56b;
  border-radius: 6px;
  color: #7a4b00;
  background: #fff7df;
  font-size: 12px;
}

.version-summary {
  padding-top: 10px;
  max-height: 170px;
  overflow-y: auto;
}

.version-list {
  display: grid;
  gap: 4px;
}

.version-row {
  width: 100%;
  min-height: 34px;
  padding: 6px 8px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 8px;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}

.version-row.selected {
  border-color: #286eac;
```

```
    background: #eef6ff;
}

.version-row span {
  min-width: 0;
  display: flex;
  align-items: center;
  justify-content: flex-start;
  gap: 8px;
  overflow: hidden;
}

.version-row small {
  color: #66717d;
  font-size: 12px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.restore-button {
  margin: 10px 0 0;
  width: 100%;
}

.settings-page {
  flex: 1;
  min-height: 0;
  padding: 18px;
  overflow: auto;
}

.gesture-page {
  flex: 1;
  min-height: 0;
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 14px;
  overflow: auto;
  background: #edf1f4;
}

.gesture-toolbar {
  min-height: 66px;
  padding: 10px 14px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  border: 1px solid #d3dce4;
  border-radius: 8px;
  background: #ffffff;
}

.gesture-toolbar h2 {
  margin: 0 0 4px;
  font-size: 18px;
  letter-spacing: 0;
}

.gesture-toolbar p {
  margin: 0;
  color: #66717d;
  font-size: 13px;
}

.gesture-actions,
.camera-state,
.result-panel-heading > span,
.privacy-note {
```

```
display: flex;
align-items: center;
gap: 8px;
}

.camera-state {
  min-width: 122px;
  color: #52606d;
  font-size: 13px;
}

.camera-state i {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #8895a2;
}

.camera-state.state-running i {
  background: #1f9d67;
  box-shadow: 0 0 0 4px rgba(31, 157, 103, 0.13);
}

.camera-state.state-loading i {
  background: #d28b16;
}

.camera-state.state-error i {
  background: #c33d3d;
}

.gesture-layout {
  flex: 1;
  min-height: 440px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) 300px;
  gap: 14px;
}

.camera-stage {
  position: relative;
  min-height: 0;
  overflow: hidden;
  border: 1px solid #25343e;
  border-radius: 8px;
  background: #101820;
}

.camera-stage video,
.camera-stage canvas {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: contain;
  transform: scaleX(-1);
}

.camera-stage canvas {
  pointer-events: none;
}

.camera-placeholder {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 10px;
  color: #dce6ec;
```

```
    text-align: center;
}

.camera-placeholder span {
    max-width: 460px;
    color: #9fb0bc;
    font-size: 13px;
}

.camera-metrics {
    position: absolute;
    top: 12px;
    left: 12px;
    display: flex;
    gap: 6px;
    pointer-events: none;
}

.camera-metrics span {
    padding: 5px 8px;
    border: 1px solid rgba(255, 255, 255, 0.25);
    border-radius: 4px;
    color: #ffffff;
    background: rgba(10, 20, 27, 0.72);
    font: 600 11px ui-monospace, SFMono-Regular, Menlo, monospace;
}

.gesture-results {
    min-height: 0;
    padding: 14px;
    display: flex;
    flex-direction: column;
    gap: 12px;
    overflow-y: auto;
    border: 1px solid #d3dce4;
    border-radius: 8px;
    background: #ffffff;
}

.result-panel-heading {
    padding-bottom: 12px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    border-bottom: 1px solid #e1e7ee;
}

.result-panel-heading span {
    font-weight: 700;
}

.result-panel-heading small {
    color: #7a8793;
}

.gesture-empty {
    min-height: 190px;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-direction: column;
    gap: 8px;
    color: #7b8994;
    text-align: center;
}

.gesture-empty span {
    font-size: 12px;
}

.hand-result {
```

```
padding: 12px;
border: 1px solid #d8e1e8;
border-left: 4px solid #1f9d67;
border-radius: 6px;
background: #f9fbfc;
}

.hand-result header {
margin-bottom: 10px;
display: flex;
align-items: center;
justify-content: space-between;
gap: 8px;
}

.hand-result header span {
color: #62717d;
font-size: 12px;
}

.hand-result header strong {
color: #116645;
font-size: 16px;
}

.hand-result dl {
margin: 0;
display: grid;
grid-template-columns: 1fr auto;
gap: 6px 12px;
font-size: 13px;
}

.hand-result dt {
color: #66717d;
}

.hand-result dd {
margin: 0;
font-weight: 600;
}

.privacy-note,
.https-warning {
padding: 10px;
border-radius: 6px;
font-size: 12px;
line-height: 1.5;
}

.privacy-note {
margin-top: auto;
color: #275d49;
background: #edf8f3;
}

.https-warning {
color: #8a4d0f;
border: 1px solid #ecd2a7;
background: #fff8e9;
}

.retry-button {
width: 100%;
}

.video-caption-page {
flex: 1;
min-height: 0;
padding: 12px;
display: grid;
```

```
grid-template-columns: 320px minmax(420px, 1fr) 380px;
gap: 12px;
overflow: hidden;
}

.video-list-panel,
.frame-browser,
.frame-detail-panel {
  min-height: 0;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  background: #ffffff;
  overflow: hidden;
}

.video-list-panel,
.frame-detail-panel {
  display: flex;
  flex-direction: column;
}

.video-list-panel {
  display: grid;
  grid-template-rows: auto auto minmax(120px, 0.34fr) auto minmax(260px, 0.66fr);
}

.batch-summary {
  margin: 10px;
  padding: 10px;
  display: grid;
  gap: 5px;
  border: 1px solid #d7dee6;
  border-radius: 6px;
  background: #f8fafc;
}

.batch-summary span,
.video-row span,
.frame-toolbar span,
.frame-card span {
  color: #66717d;
  font-size: 12px;
}

.sub-head {
  border-top: 1px solid #e1e7ee;
}

.video-list {
  min-height: 0;
  padding: 0 10px 10px;
  display: grid;
  align-content: start;
  gap: 8px;
  overflow-y: auto;
}

.video-row {
  width: 100%;
  min-height: 82px;
  padding: 8px 10px;
  display: grid;
  justify-items: stretch;
  gap: 4px;
  text-align: left;
  background: #f8fafc;
}

.video-row.selected {
  border-color: #286eac;
  background: #eef6ff;
```

```
}

.video-row strong {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}

.batch-row {
  min-height: 52px;
}

.video-original-name {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.keyword-list {
  display: flex;
  align-items: center;
  gap: 5px;
  flex-wrap: wrap;
}

.keyword-list small {
  padding: 2px 6px;
  border-radius: 999px;
  color: #145995;
  background: #eef6ff;
  font-size: 11px;
}

.frame-browser {
  display: grid;
  grid-template-rows: auto minmax(0, 1fr);
}

.frame-toolbar {
  min-height: 58px;
  padding: 10px 12px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  border-bottom: 1px solid #e1e7ee;
}

.frame-toolbar > div {
  min-width: 0;
  display: grid;
  gap: 3px;
}

.frame-toolbar strong {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.frame-search {
  min-width: min(360px, 48%);
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
  align-items: center;
  gap: 8px;
}
```

```
.frame-search input {
  width: 100%;
  min-width: 0;
  height: 36px;
  padding: 0 10px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
}

.frame-grid {
  min-height: 0;
  padding: 12px;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(144px, 1fr));
  align-content: start;
  gap: 10px;
  overflow-y: auto;
}

.frame-card {
  min-width: 0;
  padding: 6px;
  display: grid;
  gap: 6px;
  background: #f8fafc;
}

.frame-card.selected {
  border-color: #286eac;
  background: #eef6ff;
}

.frame-card img {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  border-radius: 4px;
  background: #e8edf2;
}

.frame-preview {
  margin: 10px;
  min-height: 220px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #e1e7ee;
  border-radius: 6px;
  background: #f8fafc;
  overflow: hidden;
}

.frame-preview img {
  display: block;
  width: 100%;
  max-height: 360px;
  object-fit: contain;
}

.detail-empty {
  margin: auto;
}

.settings-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 16px;
}
```

```
.settings-header h2 {
  margin: 0;
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 22px;
}

.settings-header p {
  margin: 6px 0 0;
  color: #66717d;
}

.label-form {
  display: grid;
  grid-template-columns: minmax(160px, 1fr) minmax(160px, 1fr) minmax(240px, 1.6fr) auto;
  gap: 12px;
  align-items: end;
  padding: 14px;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  background: #ffffff;
}

.label-table {
  margin-top: 14px;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  overflow: hidden;
  background: #ffffff;
}

.label-table-head,
.label-row {
  display: grid;
  grid-template-columns: 72px 1fr 1fr 1.5fr 170px 170px 132px;
  gap: 10px;
  align-items: center;
  padding: 10px 12px;
  border-bottom: 1px solid #edf1f5;
  font-size: 13px;
}

.label-table-head {
  color: #405160;
  font-weight: 700;
  background: #f8fafc;
}

.label-row > span {
  min-width: 0;
  overflow-wrap: anywhere;
}

.row-actions {
  display: flex;
  align-items: center;
  gap: 6px;
}

.row-actions button {
  min-width: 32px;
  width: 32px;
  min-height: 32px;
  padding: 0;
}

.table-empty {
  padding: 24px;
}
```

```
.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgba(15, 23, 42, 0.62);
}

.system-loading-backdrop,
.system-error-backdrop {
  position: fixed;
  inset: 0;
  z-index: 40;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  background: rgba(15, 23, 42, 0.5);
}

.system-loading-dialog,
.system-error-dialog {
  width: min(420px, 100%);
  padding: 20px;
  border: 1px solid #c9d2dc;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 18px 48px rgba(15, 23, 42, 0.24);
}

.system-loading-dialog {
  display: grid;
  justify-items: center;
  gap: 10px;
  text-align: center;
}

.system-loading-dialog svg {
  color: #1769aa;
}

.system-loading-dialog span,
.system-error-dialog span {
  color: #66717d;
  font-size: 13px;
}

.system-error-dialog {
  display: grid;
  gap: 18px;
  border-top: 4px solid #c53b32;
}

.system-error-dialog > div {
  display: grid;
  gap: 8px;
}

.system-error-dialog button {
  justify-self: end;
  min-width: 88px;
}

.auth-shell {
  min-height: 100vh;
  padding: 32px 20px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
    background: #eef1f4;
}

.auth-panel {
  width: min(420px, 100%);
  padding: 28px;
  border: 1px solid #cfd7df;
  border-top: 4px solid #1769aa;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 16px 42px rgba(23, 32, 42, 0.14);
}

.auth-brand {
  display: flex;
  align-items: center;
  gap: 12px;
}

.auth-brand img {
  width: 44px;
  height: 44px;
}

.auth-brand h1 {
  margin: 0 0 4px;
  font-size: 20px;
}

.auth-brand span {
  color: #66717d;
  font-size: 13px;
}

.auth-mode {
  margin: 24px 0 18px;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
}

.auth-form,
.auth-form label {
  display: grid;
  gap: 8px;
}

.auth-form {
  gap: 16px;
}

.auth-form label > span {
  color: #405160;
  font-size: 13px;
  font-weight: 600;
}

.auth-form input {
  width: 100%;
  min-width: 0;
  height: 42px;
  padding: 0 12px;
  border: 1px solid #bdc8d4;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
}

.auth-form input:focus {
  border-color: #1769aa;
  outline: 2px solid rgba(23, 105, 170, 0.14);
}
```

```
}

.auth-submit {
  min-height: 42px;
  margin-top: 4px;
  border-color: #1769aa;
  color: #ffffff;
  background: #1769aa;
}

.original-modal {
  max-width: min(90vw, 1200px);
  max-height: 90vh;
  padding: 12px;
  display: grid;
  gap: 10px;
  border-radius: 8px;
  background: #ffffff;
}

.original-modal img {
  max-width: 100%;
  max-height: 76vh;
  object-fit: contain;
}

.spin {
  animation: spin 900ms linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (max-width: 980px) {
  .gesture-page {
    padding: 10px;
  }

  .gesture-toolbar {
    align-items: stretch;
    flex-direction: column;
  }

  .gesture-actions {
    justify-content: space-between;
    flex-wrap: wrap;
  }

  .gesture-layout {
    grid-template-columns: 1fr;
    grid-template-rows: minmax(340px, 60vh) auto;
  }

  .gesture-results {
    overflow: visible;
  }

  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .topbar-actions {
    width: 100%;
    align-items: flex-start;
    flex-wrap: wrap;
  }
}
```

```

.user-session {
  min-width: min(220px, 100%);
  padding-left: 0;
  border-left: 0;
}

.image-task-workspace {
  height: auto;
  min-height: 100vh;
  overflow: visible;
}

.workspace-body,
.workspace-body[style] {
  grid-template-columns: 1fr;
  grid-template-rows: minmax(220px, auto) minmax(520px, 1fr) minmax(240px, auto);
  min-height: calc(100vh - 121px);
  overflow-y: auto;
}

.panel-resizer {
  display: none;
}

.history-panel,
.thumbnail-panel {
  min-height: 220px;
}

.history-panel.collapsed,
.thumbnail-panel.collapsed {
  width: auto;
  min-height: 48px;
}

.label-form,
.label-table-head,
.label-row {
  grid-template-columns: 1fr;
}

.label-table-head {
  display: none;
}
}

```

frontend/tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["DOM", "DOM.Iterable", "ES2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "resolveJsonModule": true,
    "types": ["vite/client"],
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "preserve"
  },
  "include": ["src"],
}

```

```
"references": [{ "path": "./tsconfig.node.json" }]  
}
```

frontend/tsconfig.node.json

```
{  
  "compilerOptions": {  
    "composite": true,  
    "skipLibCheck": true,  
    "module": "ESNext",  
    "moduleResolution": "Bundler",  
    "allowSyntheticDefaultImports": true  
  },  
  "include": ["vite.config.ts"]  
}
```

frontend/vite.config.d.ts

```
declare const _default: import("vite").UserConfigFnObject;  
export default _default;
```

frontend/vite.config.js

```
import { defineConfig, loadEnv } from "vite";  
import vue from "@vitejs/plugin-vue";  
export default defineConfig(function (_a) {  
  var mode = _a.mode;  
  var backendTarget = loadEnv(mode, ".", "").VITE_PROXY_TARGET || "***  
  return {  
    plugins: [vue()],  
    server: {  
      port: 4004,  
      proxy: {  
        "/api": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
        "/uploads": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
      },  
    },  
  };  
});
```

frontend/vite.config.ts

```
import { defineConfig, loadEnv } from "vite";  
import vue from "@vitejs/plugin-vue";  
  
export default defineConfig(({ mode }) => {  
  const backendTarget = loadEnv(mode, ".", "").VITE_PROXY_TARGET || "***  
  return {  
    plugins: [vue()],  
    server: {  
      port: 4004,  
      proxy: {  
        "/api": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
        "/uploads": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
      },  
    },  
  };  
});
```

```
    },
  },
},
};
});
```

scripts/dev-backend.sh

```
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")/../../backend"
uv sync --dev
DATA_DIR="${DATA_DIR:-./data}" \
INFERENCE_BACKEND="${INFERENCE_BACKEND:-ultralytics}" \
MODEL_PATH="${MODEL_PATH:-./yolo11n.pt}" \
DEVICE="${DEVICE:-0}" \
RUN_TASKS_INLINE="${RUN_TASKS_INLINE:-true}" \
TASK_WORKER_ENABLED="${TASK_WORKER_ENABLED:-false}" \
uv run uvicorn app.main:app --reload --host *** --port "${PORT:-8010}"
```

scripts/dev-frontend.sh

```
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")/../../frontend"
npm install
VITE_API_BASE_URL="${VITE_API_BASE_URL:-***}" \
npm run dev -- --host *** --port "${PORT:-4004}"
```

scripts/smoke.sh

```
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")/../../backend"
uv sync --dev
DATA_DIR="$(mktemp -d)" INFERENC
```

docker-compose.yml

```
services:
  backend:
    image: python:3.11-slim
    working_dir: /app/backend
    command: >
      sh -c "pip install -e . &&
        uvicorn app.main:app --host *** --port 8000"
    environment:
      DATA_DIR: /app/data
      DATABASE_URL: ${DATABASE_URL}
      INFERENCE_BACKEND: mock
      TASK_WORKER_ENABLED: "true"
      CORS_ORIGINS: ***
    ports:
      - "8010:8000"
    volumes:
      - ./app

  frontend:
    image: node:24
    working_dir: /app/frontend
    command: >
      sh -c "npm install &&
```

```
npm run dev -- --host *** --port 4004"
environment:
  VITE_API_BASE_URL: ""
  VITE_PROXY_TARGET: ***
ports:
  - "4004:4004"
volumes:
  - ./app
depends_on:
  - backend
```

games/competition-positioning.md

2026 工业互联网技术与应用挑战赛参赛定位

赛事识别

赛事页面：<***

公开索引显示该赛事为“2026年第二届工业互联网技术与应用挑战赛——数智应用创新赛道”，主办方为中国互联网协会智慧教育工作委员会。移动端通知页显示有“关于举办2026年第二届工业互联网技术与应用挑战赛的通知.pdf”官方盖章文件。

第三方赛事索引显示报名/赛程区间为 2026.03.17-2026.07.24。当前日期为 2026-07-20，按该信息推算剩余时间很短，应优先准备可演示系统、作品说明和应用价值材料。

推荐赛道

推荐选择：数智应用创新赛道。

项目理由：

- 项目核心是“多模态铁路目标识别与图像/视频理解”，属于 AI + 工业场景应用。
- 已具备视频解帧、目标检测、DeepSeek-VL2 图像描述、数据库存储、前端检索展示等完整链路。
- 场景可落到铁路基础设施巡检、施工安全识别、设备状态记录、人员与风险识别，契合工业企业数智化转型。
- 面向东南亚铁路发展，可包装成跨区域、跨语言、低成本部署的铁路运维数智平台。

不建议的定位

不建议只叫“铁路多目标识别系统”。

原因：这个名字偏算法功能，比赛更看重工业互联网应用价值、场景闭环和推广性。应该把重点从“识别”提升到“巡检、运维、安全、国际化服务”。

推荐题目

首选题目：

面向东南亚铁路运维的多模态智能巡检与双语风险识别系统

英文题目：

Multimodal Intelligent Inspection and Bilingual Risk Recognition System for Southeast Asian Railway Operations

备选题目

1. 面向“一带一路”铁路基础设施的多模态智能巡检与安全预警平台
2. 面向东南亚铁路场景的视觉多模态设备识别与运维辅助系统
3. 基于视觉大模型的跨语言铁路巡检解译与安全风险识别平台
4. 面向国际铁路运维的多模态目标识别、语义解译与风险管理系统

作品定位一句话

本项目面向东南亚铁路建设与运维场景，构建集视频解帧、铁路目标检测、视觉语言描述、风险关键词提取、双语展示与数据管理于一体的多模态智能巡检平台，为跨语言铁路运维、施工安全监管和设备巡检提供低成本、可部署的数智化解决方案。

建议突出功能

- 视频批量解帧：无人机/移动巡检视频自动拆解为关键帧。
- 多模态识别：结合 YOLO 检测与 DeepSeek-VL2 图像描述。

- 铁路语义关键词：接触网、轨道、人员、设备、施工、安全风险等标签化展示。
- 数据库存储：批次、视频、帧、描述、关键词全量入库，便于追溯。
- 前端可视化：按视频浏览关键帧，查看语义描述与关键词。
- 国际化双语：中文/英文界面与报告，后续可扩展东南亚本地语言。

后续材料建议

- PPT 主线：痛点 -> 场景 -> 技术方案 -> 系统演示 -> 东南亚推广价值。
- 演示重点：打开视频解帧页面，展示 24 个 DJI 视频、1716 帧、关键词和描述。
- 数据亮点：真实无人机铁路视频、实际抽帧入库、可检索可追溯。
- 应用价值：降低巡检人工成本，提高跨语言协同效率，服务铁路出海和区域基础设施运维。

games/competition/analysis/评分对照与差距分析.md

评分对照与差距分析

一、结论

本项目符合“工业企业数智化转型综合解决方案”赛题，可选择官方列出的“设备智能运维与健康管理”方向。铁路基础设施属于典型工业装备与生产运营基础设施，当前系统已经形成视频采集、解帧、视觉语言描述、关键词抽取、PostgreSQL 入库和 Web 复核的真实闭环。

当前材料的主要问题不是方向不符，而是证据与文档结构没有完全围绕评分标准组织。按现有初稿保守估计约为 **65-72 分**。完成双语最小闭环、性能与价值量化、实施和投入产出章节后，材料才具备冲击 **85 分以上** 的基础。

二、官方任务要求对照

官方要求	当前符合情况	已有内容或证据	主要缺口
--- --- --- ---			
项目背景与痛点分析	基本符合	东南亚铁路场景、视频回看成本、多源数据和跨语言协同痛点	缺少明确的目标用户、真实作业流程、痛点基线和需求指标
总体方案设计	较好符合	分层架构、采集架构、识别流程、数据库 ER 图、部署架构	缺少系统边界图、完整业务流程、数据流向和外部系统接口
创新点说明	部分符合	多模态、术语归一化、多语种预留、视频语义命名	没有独立按技术/模式/应用/价值四类论证，也缺少竞品或传统方案对比
落地可行性	部分符合	FastAPI、Vue、PostgreSQL、本地 DeepSeek-VL2、边缘中心部署设想	缺少分阶段实施步骤、硬件清单、软件依赖、网络与安全设计、预算和风险控制
推广价值	较弱符合	提到跨区域部署和东南亚多语种扩展	缺少复制对象、标准化交付包、商业模式、收费对象和规模化路线

三、评分项与当前分值预估

评分项	满分	当前预估	判断依据	冲分动作
--- ---: ---: ---				
需求与场景契合度	15	12-13	场景真实、方向明确，有 24 段 DJI 铁路视频	增加角色、作业流程、业务边界和可量化需求目标
方案创新性	25	14-17	技术组合完整，但多语种目前主要是架构预留	做出中英双语最小闭环；增加四类创新和横向对比表
技术先进性与可行性	25	19-21	已有可运行前后端、数据库和本地模型	补部署规格、接口/数据流、实施计划、实测吞吐与失败恢复机制
应用价值与成效	20	8-11	只有 24 视频、1716 帧的处理规模，没有效果和成本指标	补效率、准确性、人工复核时间、成本节约和安全提升测算
文档完整性与规范性	15	11-12	图较多、逻辑清楚，但仍标“初稿”，参考文献不完整	改成方案说明书，增加目录、图表清单、证据附录和完整引用
合计	100	64-74	中位估计约 69	先补 45 分权重对应的创新与量化成效

分值是依据当前材料的内部预评，不代表组委会最终评分。

四、必须纠正的表述风险

1. “双语风险识别”目前主要是架构预留。正式稿应写成“已实现中英术语映射和双语展示”之前，必须先做出可操作界面、术语表和样例输出；否则改为“面向双语扩展的风险识别系统”。
2. 当前 1716 帧处理结果主要证明 DeepSeek-VL2 图像描述与入库链路。若没有同批次 YOLO 检测记录，不应把 YOLO 的效果与这批验证数据合并表述。
3. “降低成本、提高效率、提升安全”必须给出基线、测量口径、公式和样本量，不能只写定性结论。
4. 东南亚适配不能只依赖标题。至少需要覆盖热带环境、弱网络、跨语言协作、本地术语和边缘部署中的两个真实约束。
5. 参考文献目前存在机构报告或规范名称过于笼统的问题。正式稿应补完整题名、年份、发布机构和可核验来源。

五、最高优先级补强项

P0：提交前必须完成

- 将正式 PDF 从“学术论文初稿”改为“工业互联网数智应用解决方案说明书”。
- 增加四类创新点对照表：技术创新、模式创新、应用创新、价值创新。
- 增加实施路线、软硬件配置、部署拓扑、数据安全和故障恢复。
- 增加投入产出模型，至少提供保守/基准两种测算，并声明数据来源和假设。
- 使用真实数据补充处理耗时、单视频平均时长、帧处理吞吐、人工复核耗时和关键词抽检结果。
- 完成最小中英双语闭环：术语表、接口字段、前端切换或双语结果、人工审校状态。

P1：显著增分

- 增加传统人工巡检、单一 YOLO、通用视觉大模型和本方案的对比矩阵。
- 选取 3 个端到端案例，展示原视频、关键帧、中文描述、英文描述、风险级别和复核结果。
- 增加系统边界图、业务流程图、数据流图、边缘-中心部署图和价值闭环图。
- 补充可复制交付包与商业模式：软件许可、项目实施、模型/术语库定制、年度运维服务。

P2：有余力再做

- 增加泰语、老挝语或越南语中的一种术语样例，但不能用少量机器翻译样例冒充完整能力。
- 增加弱网缓存、断点续传或边缘推理演示。
- 对关键词抽取进行小规模标注评估，给出准确率、召回率和错误案例。

六、量化证据建议

指标	建议口径	当前状态
数据规模	视频数、总时长、抽帧数、场景类别数	已有视频数和抽帧数，缺总时长与类别分布
处理效率	总处理时间、帧/秒、单分钟视频处理时间、GPU 型号	缺失
描述质量	随机抽检 100 帧，人工评定正确/部分正确/错误	缺失
关键词质量	人工标注集上的 Precision、Recall、F1	缺失
双语质量	术语一致率、人工审校通过率、典型错误	未实现
人工节省	人工全量回看时间与系统筛查+复核时间对比	缺失
经济价值	年视频量、人工单价、算力与维护成本、回收期	缺失
安全价值	风险发现提前量、漏检下降或复核覆盖率	缺失

所有效果数字都应来自实测或明确标注为“测算值”，并附计算公式，避免将预测值写成既成成效。

games/research/ecrl-background.md

马东铁项目背景：事实依据与写作边界

更新日期：2026-07-21

本文档用于支撑参赛论文《面向东南亚铁路运维的多模态智能巡检与双语风险识别系统》的项目背景、答辩陈述和后续资料核验。优先采用马来西亚政府、中国政府及项目相关机构公开资料。

可直接引用的公开事实

主题	已核验信息	来源
项目定位	马东铁连接哥打巴鲁与巴生港；MRL持有项目与资产，CCCC承担工程、采购、施工和调试	[马来西亚交通部项目概况](***)
最新进度	线路全长665公里；截至2026年1月整体进度91.70%；哥打巴鲁至鹅唛段计划2026年底完成、2027年1月商业运营	[马来西亚交通部2026年2月新闻稿](***)
联合运维	MRL与CCCC按50:50股比成立运营维护合资公司	[“一带一路”网转新华社报道](***)
培训规模	运营维护阶段总体培养目标为3200人；2025年计划安排210名马来西亚学员赴柳州开展一年强化培训，首批102人启程	[马来西亚交通部PLKI-ECRL O&M讲话](***)
培训岗位	车站助理、信号技术员、机车副司机、接触网技术员、电力机车与动车组维护技术员	[马来西亚交通部PLKI-ECRL O&M讲话](***)
本地化目标	马方明确将培养本地能力、避免长期依赖外国专业力量作为人才计划目标	[马来西亚交通部PLKI-ECRL O&M讲话](***)
中马合作方向	双方将马东铁、铁路运输、职业教育、数字教育、人工智能和技术转移列为合作领域，并强调相互信任和互利	[马来西亚外交部中马联合声明](***)
东盟区域方向	中国与东盟行动计划提出职业技术教育交流、知识分享、技能发展、数字教育资源共建共享和语言交流	[中国外交部：中国—东盟行动计划（2026—2030）](***)

基于事实提出的方案判断

以下内容属于本项目对公开事实的工程化推导，不应表述为有关部门已经确认的既有事实：

1. 公开资料能够证明培训和联合运维正在推进，但不能据此断言马东铁“没有运维系统”。
2. 多岗位、跨语言和联合运维会产生培训资料、现场视频、双语术语、风险案例与复核记录的统一管理需求，因此存在建设数字化知识承载平台的合理空间。
3. “培训解决人会不会，平台解决知识能否持续沉淀”是本项目的价值主张，不是官方原文。
4. 对中马合作宜使用“技术认可、互利合作、能力共建、本地化运营”等表达，不使用“东盟依赖中国技术”作为事实结论。
5. 当前原型已实现中文多模态解帧、描述、关键词、数据库和前端浏览；中英双语、马来语与其他东南亚语种属于架构预留和后续建设内容。

答辩推荐表述

> 马东铁已进入从工程建设向运营准备转换的关键阶段。中马联合运维和大规模赴华培训正在建立本地专业队伍，但人员回国后仍需要一个能把培训知识、现场巡检视频、风险术语和处置记录持续沉淀下来的数字平台。本项目以真实铁路巡检视频为基础，先完成多模态识别、结构化入库和前端复核，再逐步扩展中英及马来语知识服务，服务的是技术转移后的本地能力固化，而不是替代专业人员。

games/submission/参赛方案目录.md

正式参赛方案建议目录

建议保留作品题目《面向东南亚铁路运维的多模态智能巡检与双语风险识别系统》，将文档副标题改为“工业互联网数智应用解决方案说明书”。正文建议控制在 22-30 页；赛题未规定页数，以证据完整、图表可读为准。

封面与摘要

- 封面：作品名称、赛道、方向、团队信息、日期。
- 执行摘要：用一页说明服务对象、核心痛点、方案闭环、已实现成果和量化价值。
- 目录、图目录和表目录。

1 项目背景与典型场景（对应需求契合度 15 分）

- 1.1 东南亚铁路建设与运维背景
- 1.2 目标客户与用户角色
- 1.3 现有巡检业务流程和核心痛点
- 1.4 数智化需求、建设范围和量化目标
- 1.5 典型应用场景：接触网、轨道、人员作业、设备间与环境风险

建议图表：现状业务流程图、用户角色图、痛点-需求-目标映射表。

2 总体解决方案

- 2.1 建设思路与设计原则
- 2.2 系统边界及外部参与方
- 2.3 总体技术架构
- 2.4 巡检业务闭环
- 2.5 视频与语义数据流程
- 2.6 功能模块和权限边界

建议图表：系统上下文图、总体架构图、业务流程图、数据流图。

3 核心技术与实现路径（对应技术先进性与可行性 25 分）

- 3.1 多源巡检数据接入与标准化
- 3.2 视频解帧、时序索引和关键帧组织
- 3.3 YOLO 目标检测及人工标注闭环
- 3.4 DeepSeek-VL2 场景描述
- 3.5 铁路关键词抽取与风险分级
- 3.6 中英双语术语库、生成与人工审校
- 3.7 PostgreSQL 数据模型与可追溯机制
- 3.8 Web 可视化和人工复核
- 3.9 边缘-中心协同部署、弱网缓存与安全设计

建议图表：模型协同图、风险识别流程、ER 图、部署拓扑、接口时序图。

4 方案创新性（对应创新性 25 分）

- 4.1 技术创新：检测、视觉语言和铁路知识融合

- 4.2 模式创新：机器初筛、双语解释、人工复核闭环
- 4.3 应用创新：面向跨语言铁路运维的语义化视频资产
- 4.4 价值创新：从“视频留档”升级为“风险知识服务”
- 4.5 与人工方案、单模型方案和通用平台的对比

本章必须用对比表和已实现证据支撑，不要只列口号。

5 系统功能与真实案例

- 5.1 已实现功能清单
- 5.2 24 段视频与 1716 帧数据概况
- 5.3 案例一：接触网与轨道巡检
- 5.4 案例二：施工人员与安全风险
- 5.5 案例三：设备间与环境风险
- 5.6 中英双语输出与人工审校案例

每个案例采用“输入-模型结果-风险解释-人工复核-处置建议”的一致结构。

6 系统验证与应用成效（对应应用价值与成效 20 分）

- 6.1 验证环境和评价方法
- 6.2 处理性能和稳定性
- 6.3 描述、关键词和双语质量
- 6.4 人工巡检效率对比
- 6.5 安全、质量和管理价值
- 6.6 投入产出与回收期测算
- 6.7 局限性和适用边界

建议图表：测试环境表、指标结果表、人工与系统对比图、成本收益瀑布图。所有数值注明“实测”或“测算”。

7 落地实施与运行保障

- 7.1 单站点部署方案
- 7.2 软件、硬件和网络配置清单
- 7.3 四阶段实施计划：调研、试点、验收、推广
- 7.4 数据安全、权限、日志与备份
- 7.5 模型更新、术语维护和人工审核机制
- 7.6 项目风险与应对措施

建议图表：实施甘特图、软硬件清单、运维闭环图、风险矩阵。

8 推广价值与商业模式

- 8.1 东南亚不同国家和线路的复制方法
- 8.2 从铁路向轨道交通、港口专用线和工业园区的扩展
- 8.3 标准化交付包与生态合作
- 8.4 软件许可、项目实施、定制与年度服务模式
- 8.5 三年推广路线

9 结论

只总结已验证成果、可量化价值和下一步落地，不重复背景。

附录

- A. 核心术语中英对照表。
- B. 接口与数据库字段摘要。
- C. 测试样本、计算公式和原始统计说明。
- D. 系统截图、演示访问说明或视频二维码。
- E. 原创性、数据授权和开源组件说明。
- F. 参考文献。

与当前论文目录的调整关系

当前章节	调整方式
--- ---	
引言、需求分析	合并并增强为第 1 章，补角色、流程和指标
系统总体架构	扩展为第 2 章，补系统边界、业务流和数据流
多模态识别方法	纳入第 3 章，并严格区分已实现与规划能力
多语种知识服务	纳入第 3.6 节，并用真实双语样例验证
数据管理与系统实现	分拆到第 3 章和第 5 章

系统验证与结果分析 | 重写为第 5、6 章，增加效果和价值指标 |
跨区域部署与应用前景 | 拆为第 7、8 章，补实施、预算和商业模式 |
结论与展望 | 删除泛化展望，改为已验证结论与近期落地计划 |